

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU PHÁT TRIỂN HỆ THỐNG THÔNG TIN
HỖ TRỢ QUẢN LÝ VĂN BẰNG CHỨNG CHỈ
DỰA TRÊN NỀN TẢNG BLOCKCHAIN**

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:

Phạm Đức Khải – MSSV: AT190226

Người hướng dẫn:

TS. Trần Trung

Khoa CNTT – Trường Đại học Điện lực

Hà Nội, 2026

MỤC LỤC

CHƯƠNG 1. Cơ sở lý thuyết về công nghệ Blockchain.....	1
1.1. Tổng quan về công nghệ Blockchain.....	1
1.1.1. Khái niệm và mục tiêu hình thành.....	1
1.1.2. Quá trình phát triển.....	1
1.1.3. Đặc trưng, giới hạn và những cách hiểu cần tránh.....	2
1.1.4. Phân biệt với DLT và cơ sở dữ liệu phân tán.....	3
1.2. Cấu trúc dữ liệu và nền tảng mật mã.....	3
1.2.1. Giao dịch, khối và liên kết băm.....	3
1.2.2. Hàm băm mật mã và cây Merkle.....	4
1.2.3. Mật mã khóa công khai và chữ ký số.....	6
1.2.4. Định danh, dấu thời gian và nonce.....	6
1.3. Kiến trúc và nguyên lý hoạt động.....	7
1.3.1. Mạng ngang hàng và vai trò của nút.....	7
1.3.2. Vòng đời giao dịch.....	7
1.3.3. Sao chép trạng thái và thực thi xác định.....	8
1.3.4. Hợp đồng thông minh và dữ liệu ngoài hệ thống.....	8
1.4. Cơ chế đồng thuận.....	8
1.4.1. Bài toán đồng thuận và mô hình lỗi.....	9
1.4.2. Proof of Work và Proof of Stake.....	9
1.4.3. Đồng thuận CFT và BFT trong mạng định danh.....	10
1.4.4. Tính hoàn tất và đánh đổi thiết kế.....	10
1.5. Phân loại Blockchain và điều kiện lựa chọn.....	12
1.5.1. Các trục phân loại.....	12
1.5.2. So sánh các mô hình triển khai.....	13
1.5.3. Khi nên và không nên sử dụng Blockchain.....	13
1.6. An toàn, quyền riêng tư và khả năng mở rộng.....	14
1.6.1. Mô hình đe dọa và bảo vệ nhiều lớp.....	14
1.6.2. Quản lý khóa, hợp đồng thông minh và nguồn dữ liệu ngoài.....	15
1.6.3. Quyền riêng tư và dữ liệu cá nhân.....	15
1.6.4. Khả năng mở rộng và khả năng tương tác.....	15
1.7. Định hướng ứng dụng trong quản lý văn bản, chứng chỉ.....	16
1.7.1. Bài toán, tác nhân và yêu cầu kiểm chứng.....	16
1.7.2. Mô hình dữ liệu khái niệm trên chuỗi và ngoài chuỗi.....	17
1.7.3. Giá trị kỳ vọng, giới hạn và ranh giới trách nhiệm.....	17
1.8. Tổng kết Chương 1.....	17
1.8.1. Kết quả lý luận chính.....	17
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	19
2.1. Bối cảnh số hóa văn bản và nhu cầu xác minh.....	19
2.2. Phân tích bài toán và phạm vi hệ thống.....	20
2.2.1. Quy trình nghiệp vụ hiện tại.....	20
2.2.2. Các bất cập cần giải quyết.....	21
2.2.3. Quy trình đề xuất.....	21

2.2.4. Ranh giới, giả định và ràng buộc.....	22
2.3. Thuật ngữ, tác nhân và quyền hạn.....	23
2.3.1. Quy ước thuật ngữ.....	23
2.3.2. Phân tích tác nhân.....	24
2.3.3. Mô hình use case tổng quát.....	26
2.4. Yêu cầu chức năng và đặc tả use case.....	27
2.4.1. Danh sách yêu cầu chức năng.....	27
2.4.2. UC-01 – Đăng nhập.....	28
2.4.3. UC-02 – Tạo yêu cầu phát hành.....	28
2.4.4. UC-03 – Phê duyệt hoặc từ chối.....	29
2.4.5. UC-04 – Xử lý phát hành bất đồng bộ.....	29
2.4.6. UC-05 – Xem và tải chứng thư của Student.....	30
2.4.7. UC-06 – Xác minh công khai.....	31
2.4.8. UC-07 – Thu hồi chứng thư.....	32
2.4.9. UC-08 – Quản trị và công bố thông tin hỗ trợ.....	32
2.5. Yêu cầu phi chức năng.....	33
2.6. Mô hình đe dọa.....	35
2.6.1. Phương pháp đánh giá.....	35
2.6.2. Ma trận tài sản–đe dọa–kiểm soát.....	35
2.7. Phân tích và lựa chọn kiến trúc.....	37
2.7.1. Blockchain công khai và Blockchain cấp quyền.....	37
2.7.2. Bitcoin, Ethereum và Hyperledger Fabric.....	37
2.7.3. Lưu toàn bộ dữ liệu và chỉ lưu cam kết mật mã.....	38
2.7.4. Đồng bộ, bất đồng bộ, phát hành đơn và phát hành theo lô.....	38
2.8. Kiến trúc và các luồng xử lý.....	39
2.8.1. Kiến trúc thành phần và ranh giới tin cậy.....	39
2.8.2. Trình tự phát hành theo lô.....	39
2.8.3. Máy trạng thái yêu cầu phát hành.....	40
2.8.4. Trình tự xác minh công khai.....	41
2.9. Thiết kế dữ liệu.....	42
2.9.1. Mô hình dữ liệu đang triển khai.....	42
2.9.2. Mô hình chuẩn hóa đề xuất cho giai đoạn sau.....	43
2.9.3. Ràng buộc, chỉ mục và chính sách dữ liệu.....	43
2.9.4. Nguồn sự thật đối với thu hồi.....	44
2.10. Thiết kế API và phân quyền.....	44
2.10.1. Quy ước chung.....	44
2.10.2. Đặc tả endpoint chính.....	44
2.10.3. Ma trận phân quyền.....	46
2.11. Thiết kế ba cổng giao diện.....	47
2.11.1. Nguyên tắc thiết kế.....	47
2.11.2. Sơ đồ điều hướng.....	48
2.11.3. Đặc tả màn hình và trạng thái.....	48
2.12. An toàn triển khai và vận hành.....	49
2.13. Tiêu chí chấp nhận và ma trận truy vết.....	50
2.13.1. Tiêu chí chấp nhận.....	50

2.13.2. Ma trận FR–use case–thành phần–API–dữ liệu–kiểm thử.....	51
2.14. Kết luận Chương 2.....	53
CHƯƠNG 3. XÂY DỰNG, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG.....	54
3.1. Môi trường và công nghệ triển khai.....	54
3.1.1. Môi trường thực nghiệm.....	54
3.1.2. Tổ chức triển khai.....	55
3.2. Hướng dẫn tái lập triển khai từ Ubuntu mới.....	55
3.2.1. Chuẩn bị máy Ubuntu và kiểm tra tài nguyên.....	55
3.2.2. Lấy mã nguồn và cố định phiên bản.....	56
3.2.3. Tạo cấu hình runtime và bảo vệ bí mật.....	57
3.2.4. Khởi động hạ tầng và áp dụng migration.....	57
3.2.5. Cài phụ thuộc, kiểm thử và build.....	58
3.2.6. Khởi tạo Blockcerts regtest và chạy preflight.....	59
3.2.7. Triển khai backend, frontend và kiểm tra dịch vụ.....	60
3.2.8. Cơ chế khôi phục Worker cần kiểm tra sau triển khai.....	60
3.3. Hiện thực backend và cơ sở dữ liệu.....	61
3.3.1. Các module.....	61
3.3.2. Migration và transaction.....	61
3.3.3. Audit và verification log.....	62
3.4. Hiện thực quy trình phát hành.....	62
3.4.1. Phát hành đơn và Maker–Checker.....	62
3.4.2. Tối ưu phát hành theo lô.....	63
3.4.3. Cấu trúc chứng thư đầu ra.....	64
3.5. Hiện thực xác minh và thu hồi.....	64
3.5.1. Xác minh công khai.....	64
3.5.2. Thu hồi.....	64
3.6. Hiện thực ba frontend.....	65
3.7. Gia cố an toàn, định danh và độ bền Worker.....	65
3.8. Phương pháp kiểm thử.....	66
3.8.1. Mục tiêu và nguyên tắc.....	66
3.8.2. Kiểm thử tự động và chất lượng mã.....	66
3.9. Kịch bản 1 – Quy trình cấp văn bằng đầu cuối.....	67
3.10. Kịch bản 2 – Xác minh, toàn vẹn và kiểm soát an toàn.....	72
3.11. Kịch bản 3 – Hiệu năng, phát hành theo lô và khả năng phục hồi.....	78
3.11.1. Kết quả từng lần.....	79
3.11.2. Giá trị tổng hợp.....	79
3.11.3. Tài nguyên.....	81
3.11.4. Kiểm toán artifact.....	81
3.12. Đánh giá kết quả.....	82
3.12.1. Mức đáp ứng.....	82
3.12.2. Hạn chế.....	82
3.12.3. Hướng phát triển.....	83
3.13. Quy trình thao tác chi tiết để tái hiện ba kịch bản.....	84
3.13.1. Bước quy trình 1 – Maker tạo yêu cầu cấp chứng thư.....	84
3.13.2. Bước quy trình 2 – Checker phê duyệt và Worker phát hành.....	86

3.13.3. Bước quy trình 3 – Student xem và tải chứng thư.....	88
3.13.4. Bước quy trình 4 – Xác minh công khai chứng thư hợp lệ.....	90
3.13.5. Bước quy trình 5 – Phát hiện chứng thư bị sửa.....	92
3.13.6. Bước quy trình 6 – Checker thu hồi và xác minh lại.....	93
3.13.7. Bước quy trình 7 – Kiểm tra RBAC, isolation và SSRF.....	95
3.13.8. Bước quy trình 8 – Đối chiếu trạng thái và bằng chứng sau thử nghiệm.....	95
3.14. Hỏi đáp và tự phản biện.....	96
3.14.1. Chi phí, Bitcoin và mô hình vận hành.....	96
3.14.2. Lý do chọn Blockchain và Bitcoin.....	97
3.14.3. Tính đúng đắn mật mã và xác minh.....	98
3.14.4. Khóa, danh tính và quyền sở hữu.....	99
3.14.5. Dữ liệu cá nhân, pháp lý và quyền riêng tư.....	100
3.14.6. Thu hồi, sửa sai và vòng đời văn bằng.....	101
3.14.7. Bảo mật ứng dụng và mô hình đe dọa.....	102
3.14.8. Độ tin cậy, mở rộng và vận hành production.....	103
3.14.9. Kiểm thử, bằng chứng và giới hạn thực nghiệm.....	104
3.14.10. Trải nghiệm người dùng và khả năng tương tác.....	105
3.14.11. Quản trị, đạo đức và hướng phát triển.....	106
3.15. Kết luận Chương 3.....	107
KẾT LUẬN VÀ KIẾN NGHỊ.....	108
1. Kết quả của đồ án.....	23
2. Đóng góp chính.....	86
3. Kiến nghị.....	86
TÀI LIỆU THAM KHẢO.....	109
PHỤ LỤC.....	111
Phụ lục A. Nhóm API chính.....	111
Phụ lục B. Bằng chứng thực nghiệm chính thức.....	111
Phụ lục C. Hướng dẫn tái lập tóm tắt.....	111

DANH MỤC KÝ HIỆU VÀ TỪ VIẾT TẮT

Từ viết tắt	Từ đầy đủ
API	Application Programming Interface
BFT	Byzantine Fault Tolerance
CFT	Crash Fault Tolerance
DLT	Distributed Ledger Technology
HSM	Hardware Security Module
JSON-LD	JavaScript Object Notation for Linked Data
JWT	JSON Web Token
P2P	Peer-to-Peer
PKI	Public Key Infrastructure
PoS	Proof of Stake
PoW	Proof of Work
RBAC	Role-Based Access Control
RPC	Remote Procedure Call
SHA	Secure Hash Algorithm
URI	Uniform Resource Identifier
UUID	Universally Unique Identifier
VBCC	Văn bản, chứng chỉ
VC	Verifiable Credential
W3C	World Wide Web Consortium

DANH MỤC HÌNH VẼ

Hình 1.1. Cấu trúc khối và liên kết mật mã giữa các khối liên tiếp.....	4
Hình 1.2. Cây Merkle và đường dẫn kiểm chứng một giao dịch.....	5
Hình 1.3. Vòng đời giao dịch Blockchain ở mức khái quát.....	7
Hình 1.4. So sánh khái quát các nhóm cơ chế đồng thuận.....	11
Hình 1.5. Phân loại Blockchain theo phạm vi tham gia và quyền truy cập.....	12
Hình 1.6. Các lớp bảo vệ trong một hệ thống Blockchain.....	14
Hình 1.7. Mô hình khái niệm ứng dụng Blockchain trong quản lý văn bản. .	16
Hình 2.1. Quy trình quản lý và xác minh văn bản theo mô hình hiện tại.....	21
Hình 2.2. Quy trình đề xuất với kiểm soát Maker–Checker và neo Bitcoin....	22
Hình 2.3. Tác nhân và nhóm use case trong ranh giới hệ thống.....	26
Hình 2.4. Kiến trúc thành phần và ranh giới tin cậy.....	39
Hình 2.5. Trình tự phát hành chứng thư theo lô.....	39
Hình 2.6. Máy trạng thái yêu cầu phát hành.....	40
Hình 2.7. Trình tự xác minh công khai và các nhánh kết quả.....	41
Hình 2.8. Mô hình dữ liệu vật lý hiện tại và liên kết logic.....	42
Hình 2.9. Sơ đồ điều hướng ba cổng giao diện.....	48
Hình 3.1. Kiến trúc triển khai.....	55
Hình 3.2. Kiểm tra hệ điều hành, tài nguyên và phiên bản công cụ trên máy mới	56
Hình 3.3. Trạng thái hạ tầng và các migration đã áp dụng.....	58
Hình 3.4. Kết quả unit test backend và kiểm thử SSRF của verifier.....	59
Hình 3.5. Preflight Blockcerts và giao dịch neo trên Bitcoin regtest.....	60
Hình 3.6. Triển khai hoàn tất và health check toàn hệ thống.....	60
Hình 3.7. Luồng phát hành một văn bản.....	63
Hình 3.8. Luồng phát hành theo lô.....	63

Hình 3.9. Luồng thu hồi và xác minh.....	65
Hình 3.10. Maker đối chiếu hồ sơ Student chỉ đọc và nhập đầy đủ dữ liệu văn bằng	68
Hình 3.11. Phiếu chứa dữ liệu nghiệp vụ được tạo thành công và chờ duyệt.	69
Hình 3.12. Checker xem đầy đủ snapshot hồ sơ và thông tin văn bằng.....	70
Hình 3.13. Records hiển thị văn bằng đã được Worker phát hành.....	71
Hình 3.14. Holder xem thông tin văn bằng thuộc đúng tài khoản Student.....	72
Hình 3.15. Verify xác nhận chứng thư hợp lệ và hiển thị dữ liệu nghiệp vụ...	73
Hình 3.16. Verify phát hiện số hiệu văn bằng trong JSON đã bị sửa.....	74
Hình 3.17. Checker thu hồi văn bằng và ghi nhận giao dịch thu hồi.....	75
Hình 3.18. Verify trả trạng thái đã thu hồi cho chứng thư gốc.....	76
Hình 3.19. Tiến trình thực nghiệm Chương 3.....	77
Hình 3.20. Tổng hợp tỷ lệ đạt của ba kịch bản.....	78
Hình 3.21. Thời gian phát hành theo kích thước batch.....	80
Hình 3.22. Thông lượng và thời gian trên mỗi chứng thư.....	80
Hình 3.23. Mức sử dụng tài nguyên.....	81
Hình 3.24. Màn hình đăng nhập cổng quản trị.....	84
Hình 3.25. Maker nhập mã sinh viên và địa chỉ nhận của chứng thư.....	85
Hình 3.26. Yêu cầu mới được tạo và chờ Checker phê duyệt.....	86
Hình 3.27. Checker kiểm tra yêu cầu đang chờ phê duyệt.....	87
Hình 3.28. Chứng thư chuyển sang trạng thái đã cấp sau khi Worker xử lý...	88
Hình 3.29. Màn hình đăng nhập cổng Student.....	89
Hình 3.30. Student xem danh sách và tải chứng thư thuộc tài khoản.....	90
Hình 3.31. Cổng Verify công khai trước khi tải chứng thư.....	91
Hình 3.32. Kết quả xác minh hợp lệ trước khi chứng thư bị thu hồi.....	92
Hình 3.33. Verifier phát hiện nội dung chứng thư đã bị thay đổi.....	93

<i>Hình 3.34. Checker thu hồi chứng thư và hệ thống ghi nhận trạng thái.....</i>	<i>94</i>
<i>Hình 3.35. Cổng Verify trả trạng thái đã thu hồi cho chứng thư gốc.....</i>	<i>95</i>

DANH MỤC BẢNG

Bảng 1.1. So sánh khái quát các mô hình triển khai Blockchain.....	13
Bảng 2.1. Quy trình nghiệp vụ hiện tại và điểm yếu cần xử lý.....	20
Bảng 2.2. Vai trò và giới hạn của Blockchain theo công đoạn.....	23
Bảng 2.3. Quy ước thuật ngữ trong đồ án.....	23
Bảng 2.4. Mục tiêu, quyền hạn và giới hạn của tác nhân.....	24
Bảng 2.5. Danh sách yêu cầu chức năng.....	27
Bảng 2.6. Đặc tả UC-01 – Đăng nhập.....	28
Bảng 2.7. Đặc tả UC-02 – Tạo yêu cầu phát hành.....	28
Bảng 2.8. Đặc tả UC-03 – Phê duyệt hoặc từ chối.....	29
Bảng 2.9. Đặc tả UC-04 – Xử lý phát hành bất đồng bộ.....	29
Bảng 2.10. Đặc tả UC-05 – Xem và tải chứng thư của Student.....	30
Bảng 2.11. Đặc tả UC-06 – Xác minh công khai.....	31
Bảng 2.12. Đặc tả UC-07 – Thu hồi chứng thư.....	32
Bảng 2.13. Yêu cầu phi chức năng và trạng thái kiểm chứng.....	33
Bảng 2.14. Ma trận tài sản–đe dọa–kiểm soát.....	35
Bảng 2.15. So sánh Blockchain công khai và Blockchain cấp quyền.....	37
Bảng 2.16. So sánh nền tảng cho phạm vi đồ án.....	38
Bảng 2.17. Chuyển trạng thái yêu cầu phát hành.....	40
Bảng 2.18. Các bảng dữ liệu vật lý hiện tại.....	42
Bảng 2.19. Đặc tả các endpoint chính.....	44
Bảng 2.20. Ma trận phân quyền chức năng.....	46
Bảng 2.21. Đặc tả màn hình và trạng thái giao diện.....	48
Bảng 2.22. Tiêu chí chấp nhận.....	50
Bảng 2.23. Ma trận truy vết yêu cầu–thiết kế–kiểm thử.....	51
Bảng 3.1. Thành phần phần mềm.....	54
Bảng 3.2. Kết quả Kịch bản 1.....	76
Bảng 3.3. Kết quả Kịch bản 3.....	77
Bảng 3.4. Chín lần chạy hiệu năng chính thức.....	79
Bảng 3.5. Tổng hợp theo kích thước batch.....	79
Bảng 3.6. Tài nguyên hệ thống.....	81
Bảng 3.7. Kết quả kiểm toán.....	82

<i>Bảng 3.8. Đối chiếu yêu cầu và bằng chứng.....</i>	<i>82</i>
---	-----------

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp này, em đã nhận được sự giúp đỡ tận tình của cán bộ hướng dẫn TS. Trần Trung – Giảng viên Khoa CNTT Trường Đại học Điện lực; cán bộ quản lý ThS. Nguyễn Đức Ngân – Giảng viên Khoa ATTT Học viện Kỹ thuật Mật Mã, cùng với sự quan tâm sâu sát của Hệ quản lý sinh viên, sự động viên của người thân, bạn bè đã luôn kề bên hỗ trợ hết mình.

Xin cảm ơn tất cả mọi người đã tạo những điều kiện tốt nhất để em hoàn thành đồ án tốt nghiệp này.

SINH VIÊN THỰC HIỆN ĐỒ ÁN

Phạm Đức Khải

LỜI NÓI ĐẦU

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ, việc số hóa hồ sơ, tài liệu và văn bằng, chứng chỉ đóng vai trò thiết yếu trong công tác quản lý giáo dục. Tuy nhiên, sự phát triển của hệ thống thông tin số cũng kéo theo những thách thức lớn về bảo mật và an toàn dữ liệu. Hiện nay, các văn bằng, chứng chỉ chủ yếu được quản lý qua hồ sơ giấy hoặc các cơ sở dữ liệu tập trung truyền thống. Phương pháp này bộc lộ nhiều hạn chế: hồ sơ giấy tốn kém chi phí lưu trữ, dễ thất lạc; trong khi dữ liệu tập trung dễ trở thành mục tiêu tấn công, sửa đổi trái phép từ tin tặc. Bên cạnh đó, tình trạng làm giả văn bằng ngày càng tinh vi, quy trình tra cứu và xác minh thủ công lại rườm rà, mất nhiều thời gian, gây ảnh hưởng trực tiếp đến uy tín của các cơ sở đào tạo cũng như tổ chức tuyển dụng.

Để hỗ trợ giải quyết những hạn chế trên, Blockchain là một hướng tiếp cận đáng xem xét khi nhiều bên cần chia sẻ và kiểm chứng lịch sử dữ liệu. Sự kết hợp giữa mạng ngang hàng, hàm băm mật mã, chữ ký số và cơ chế đồng thuận cho phép xây dựng sổ cái phân tán có khả năng truy vết và phát hiện sửa đổi. Trong quản lý văn bằng, việc ghi bằng chứng mật mã cùng lịch sử trạng thái lên sổ cái có thể hỗ trợ phát hiện tài liệu bị thay đổi và giảm sự phụ thuộc vào một máy chủ đối soát duy nhất. Mức bảo đảm thực tế vẫn phụ thuộc chất lượng dữ liệu đầu vào, quản lý khóa, mô hình đồng thuận và quy trình quản trị hệ thống.

Xuất phát từ yêu cầu cấp thiết về việc bảo vệ tính toàn vẹn của dữ liệu học thuật và khắc phục những hạn chế của phương pháp quản lý hiện tại, em đã quyết định lựa chọn đề tài “Nghiên cứu phát triển hệ thống thông tin hỗ trợ quản lý văn bằng chứng chỉ dựa trên nền tảng Blockchain”. Mục tiêu của đề án là phân tích, thiết kế và triển khai thực nghiệm một hệ thống phần mềm ứng dụng công nghệ Blockchain nhằm số hóa quy trình cấp phát, quản lý và xác thực văn bằng chứng chỉ một cách an toàn, minh bạch.

Do thời gian thực hiện và phạm vi nghiên cứu có giới hạn, đề án chắc chắn không tránh khỏi những thiếu sót nhất định. Kính mong nhận được sự nhận xét, góp ý từ quý thầy cô và các bạn để hệ thống được hoàn thiện hơn trong tương lai.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VỀ CÔNG NGHỆ BLOCKCHAIN

1.1. Tổng quan về công nghệ Blockchain

1.1.1. Khái niệm và mục tiêu hình thành

Công nghệ chuỗi khối (Blockchain) là một dạng kiến trúc sổ cái phân tán (Distributed Ledger Technology - DLT), trong đó dữ liệu giao dịch được đóng gói thành các khối (blocks), liên kết tuần tự theo thời gian bằng các hàm băm mật mã và được đồng bộ giữa các nút mạng thông qua giao thức đồng thuận [1], [2]. Thay vì phụ thuộc vào một máy chủ trung tâm, sổ cái Blockchain được sao chép và duy trì độc lập tại nhiều nút tham gia. Khi dữ liệu quá khứ bị thay đổi, các liên kết băm phía sau không còn khớp; sự sai lệch vì thế có thể được phát hiện khi kiểm tra lại lịch sử. Kết hợp với sao chép sổ cái và đồng thuận, cơ chế này làm cho việc sửa đổi hồ sơ mà không để lại dấu vết trở nên khó khăn trong các giả định an toàn của hệ thống [1], [10].

Mục tiêu cốt lõi của Blockchain là giúp các thực thể thuộc nhiều miền quản trị thống nhất một lịch sử trạng thái dùng chung và giảm mức phụ thuộc vào một bên trung gian duy nhất [1]. Trong các quy trình nghiệp vụ truyền thống, việc đối soát liên tổ chức qua trung gian thường làm phát sinh chi phí, độ trễ và nguy cơ xuất hiện điểm lỗi đơn lẻ (Single Point of Failure). Blockchain giải quyết bài toán này nhờ kết hợp mật mã khóa công khai, cây Merkle, mạng ngang hàng và thuật toán đồng thuận để tạo ra nhật ký kiểm toán minh bạch, cho phép các bên tự kiểm chứng tính toàn vẹn dữ liệu [2], [10].

Tuy nhiên, Blockchain không nhằm thay thế các hệ thống lưu trữ dung lượng lớn hay cơ sở dữ liệu quan hệ truyền thống. Mọi hệ thống Blockchain trong thực tế đều đòi hỏi một khung quản trị xác định, bao gồm quy định về quyền truy cập, chính sách quản lý khóa mật mã, quy tắc phê duyệt giao dịch và cơ chế nâng cấp giao thức [1]. Công nghệ này phù hợp hơn với bài toán cộng tác đa bên, trong đó các thực thể cần chia sẻ trạng thái có thể kiểm chứng nhưng không muốn một bên duy nhất toàn quyền quyết định lịch sử dữ liệu [10].

1.1.2. Quá trình phát triển

Nền tảng lý thuyết của Blockchain là kết quả kế thừa từ nhiều thập kỷ nghiên cứu về mật mã học, cấu trúc dữ liệu và hệ thống phân tán. Từ cuối những năm 1970 và đầu thập niên 1980, các công trình về hệ mật mã khóa công khai, cấu trúc cây băm Merkle [3] và các giải pháp đóng dấu thời gian số (digital

timestamping) đã đặt nền móng cho việc xác thực tính toàn vẹn dữ liệu. Song song với đó, lý thuyết hệ thống phân tán phát triển các mô hình đồng thuận chịu lỗi dừng (CFT) và chịu lỗi tùy ý Byzantine (BFT), cung cấp cơ sở toán học để duy trì tính nhất quán trạng thái giữa các nút độc lập khi xảy ra sự cố mạng hoặc có nút sai hỏng [6], [7].

Năm 2008, Satoshi Nakamoto kết hợp chuỗi liên kết băm, mạng ngang hàng và cơ chế Bằng chứng công việc (Proof of Work) để giới thiệu Bitcoin, qua đó đề xuất một cơ chế hạn chế chi tiêu kép mà không cần máy chủ thanh toán trung tâm [1], [2]. Các nền tảng về sau mở rộng khả năng lập trình trạng thái thông qua hợp đồng thông minh (smart contract), cho phép biểu diễn và tự động thực thi một số quy tắc nghiệp vụ [1], [10]. Đồng thời, các kiến trúc Blockchain cấp quyền (permissioned) ra đời, ứng dụng các thuật toán CFT và BFT để tối ưu hóa thông lượng và đáp ứng yêu cầu bảo mật, quyền riêng tư nghiêm ngặt trong môi trường liên tổ chức [1], [8].

1.1.3. Đặc trưng, giới hạn và những cách hiểu cần tránh

Hệ thống Blockchain sở hữu các đặc trưng kỹ thuật cốt lõi gồm phân tán dữ liệu, khả năng kiểm chứng độc lập, tính truy vết giao dịch và cơ chế chống can thiệp dữ liệu hồi tố [1]. Mọi giao dịch hợp lệ đều gắn liền với định danh mật mã của bên khởi tạo và được neo chặt vào cấu trúc chuỗi khối, giúp người tham gia đối soát lịch sử một cách minh bạch mà không phụ thuộc một chủ thể duy nhất [2], [10]. Mặc dù vậy, việc nghiên cứu và ứng dụng đòi hỏi phải nhận diện rõ các giới hạn kỹ thuật và tránh ba ngộ nhận phổ biến:

Thứ nhất, dữ liệu ghi trên chuỗi không đồng nghĩa với chân lý thực tế; hệ thống chỉ bảo đảm tính toàn vẹn của dữ liệu sau khi đã được chấp nhận vào sổ cái. Nếu dữ liệu đầu vào bị sai lệch hoặc cố tình gian lận từ khâu thu thập ngoại chuỗi (off-chain), hệ thống vẫn lưu trữ và bảo vệ dữ liệu sai đó một cách bền vững theo nguyên lý "rác vào, rác ra" (Garbage-In, Garbage-Out) [1]. Thứ hai, không nên tuyệt đối hóa tính chất dữ liệu thành "bất biến vĩnh cửu". Về bản chất, dữ liệu trên chuỗi chỉ mang tính kháng sửa đổi dựa trên các giả định an toàn mật mã và ngưỡng tài nguyên đồng thuận; nếu kẻ xấu kiểm soát vượt ngưỡng năng lực tính toán hoặc vượt số nút lỗi cho phép của thuật toán đồng thuận, lịch sử giao dịch vẫn có nguy cơ bị phân nhánh hoặc đảo ngược [1], [6].

Thứ ba, Blockchain không đồng nghĩa với tính ẩn danh hoàn toàn hay việc loại bỏ trách nhiệm pháp lý. Đa phần các mạng lưới chỉ cung cấp tính năng bí danh

(pseudonymity) dựa trên địa chỉ khóa công khai; thông qua phân tích đồ thị luồng giao dịch và đối chiếu siêu dữ liệu ngoại vi, danh tính thực của thực thể vẫn có thể bị truy vết và định danh [1]. Ngoài ra, việc nhiều nút cùng kiểm tra và lưu bản sao thường làm tăng độ trễ, chi phí lưu trữ và chi phí phối hợp so với cơ sở dữ liệu tập trung được tối ưu cho cùng một bài toán [10].

1.1.4. Phân biệt với DLT và cơ sở dữ liệu phân tán

Công nghệ sổ cái phân tán (DLT) là khái niệm bao trùm, chỉ chung mọi kiến trúc hệ thống lưu trữ và đồng bộ hóa sổ cái ghi chép trạng thái trên nhiều thực thể mạng mà không phụ thuộc vào một máy chủ trung tâm [1]. DLT không bắt buộc dữ liệu phải tổ chức thành khối liên kết chuỗi tuyến tính, mà có thể sử dụng cấu trúc đồ thị có hướng không chu trình (DAG) hoặc các dạng cây phi tuyến khác. Do đó, Blockchain là một tập con đặc thù của DLT, nổi bật với cấu trúc phân khối tuần tự và các liên kết mật mã chặt chẽ giữa các khối [1], [10].

Cơ sở dữ liệu phân tán truyền thống (chẳng hạn các hệ quản trị RDBMS hoặc NoSQL phân tán) cũng thực hiện việc phân mảnh và nhân bản dữ liệu qua nhiều máy chủ nhằm tối ưu thông lượng, độ sẵn sàng (Availability) và tính chịu lỗi phần cứng (Fault Tolerance). Tuy nhiên, các hệ thống này vận hành trong cùng một miền quản trị tin cậy (Single Trust Domain) dưới sự kiểm soát của một tổ chức [10]. Quản trị viên hệ thống (DBA) có toàn quyền can thiệp, xóa bản ghi hoặc chỉnh sửa lược đồ dữ liệu trực tiếp mà không cần sự đồng thuận ngang hàng của các bên độc lập.

Blockchain hướng tới môi trường có nhiều chủ thể cùng duy trì lịch sử theo một bộ quy tắc đã thống nhất [1], [8]. Trong thiết kế liên tổ chức, quyền cập nhật và quản trị nên được phân bổ sao cho một tài khoản đơn lẻ không thể âm thầm sửa lịch sử đã xác nhận; tuy nhiên, mức phân quyền thực tế vẫn phụ thuộc số tổ chức vận hành, chính sách nâng cấp và khả năng thông đồng. Khác biệt cốt lõi so với cơ sở dữ liệu phân tán truyền thống vì thế nằm ở ranh giới tin cậy và quy trình chấp nhận thay đổi, không chỉ ở số lượng máy lưu bản sao [10].

1.2. Cấu trúc dữ liệu và nền tảng mật mã

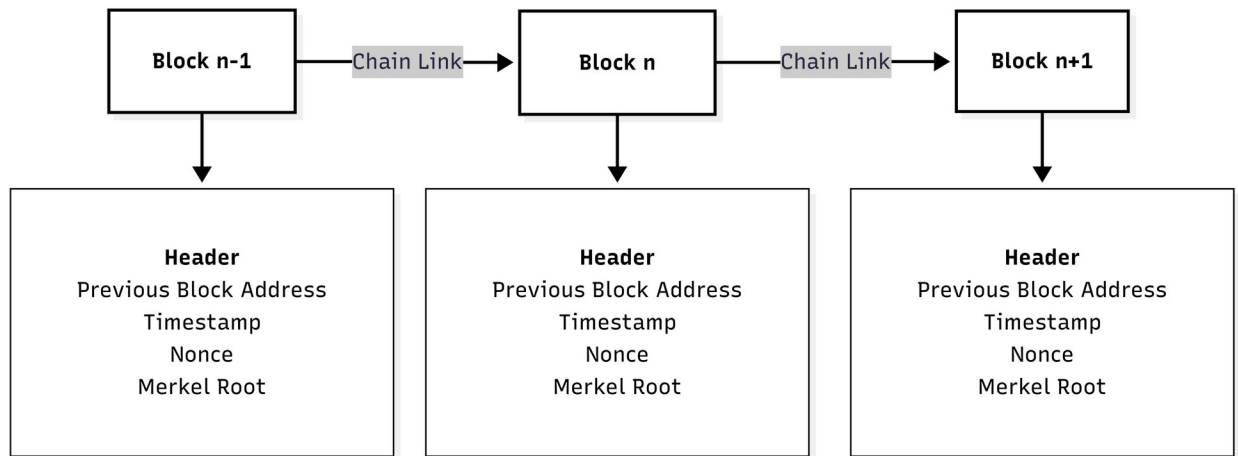
1.2.1. Giao dịch, khối và liên kết băm

Giao dịch (Transaction) là đơn vị nguyên tử biểu diễn một yêu cầu thay đổi trạng thái trong hệ thống sổ cái phân tán [1], [10]. Tùy nền tảng, giao dịch có thể chứa định danh hoặc địa chỉ bên gửi, dữ liệu đầu vào, số thứ tự chống phát lại, tham số phí và chữ ký số. Khi được phát tán vào mạng lưới, các nút tiếp nhận tiến

hành kiểm tra độc lập tính hợp lệ về mặt cú pháp, tính toàn vẹn chữ ký số và điều kiện logic trạng thái hiện hành trước khi chấp nhận đưa vào hàng đợi xử lý [1], [2].

Các giao dịch đã được sắp thứ tự có thể được đóng gói vào Khối (Block), gồm phần đầu chứa siêu dữ liệu giao thức và phần thân chứa giao dịch [1], [2]. Tùy thiết kế, phần đầu khối có thể bao gồm giá trị băm của khối trước (H_{i-1}), gốc Merkle ($MerkleRoot_i$), dấu thời gian ($Timestamp_i$) và dữ liệu phục vụ đồng thuận ($Metadata_i$). Một biểu diễn khái quát của liên kết băm là: Mỗi liên kết mật mã giữa các khối liên tiếp được thiết lập bằng cách tính toán giá trị băm H_i của toàn bộ phần đầu khối thông qua hàm băm mật mã H :

Trong biểu thức (1.1), toán tử // biểu thị phép nối chuỗi. Đây là mô hình minh họa; tập trường và quy tắc mã hóa byte cụ thể do từng giao thức quy định. Giá trị H_{i-1} tạo liên kết kiểm chứng giữa hai khối liên tiếp.



Hình 1.1. Cấu trúc khối và liên kết mật mã giữa các khối liên tiếp

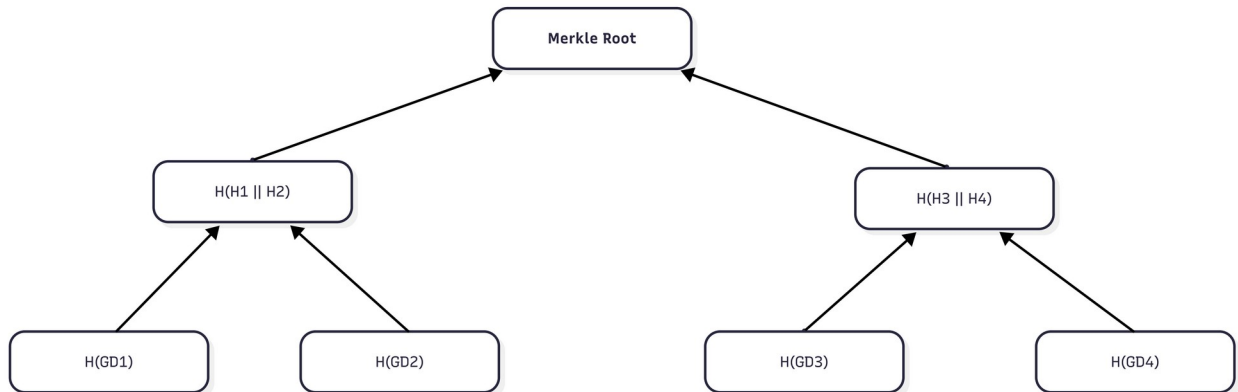
Nếu dữ liệu trong khối quá khứ k bị sửa, giá trị băm giao dịch, gốc Merkle và H_k thay đổi do tính nhạy của hàm băm [4]. Tham chiếu tại khối kế tiếp vì thế không còn khớp và sự can thiệp có thể được phát hiện. Để lịch sử đã sửa được các nút chấp nhận, đối phương còn phải đáp ứng quy tắc đồng thuận và chính sách quản trị của mạng; điều kiện cụ thể phụ thuộc loại Blockchain và mô hình đe dọa, không chỉ phụ thuộc liên kết băm [1], [2].

1.2.2. Hàm băm mật mã và cây Merkle

Hàm băm mật mã là công cụ toán học nền tảng đảm bảo tính toàn vẹn dữ liệu trong kiến trúc chuỗi khối [1], [4]. Một hàm băm mật mã H ánh xạ một thông điệp đầu vào M có độ dài tùy ý thành một chuỗi nhị phân cố định $h=H(M)$, điển hình là 256-bit đối với chuẩn SHA-256 (FIPS 180-4) [4]. Về mặt an toàn thông tin,

hàm băm mật mã phải thỏa mãn ba thuộc tính cốt lõi: tính kháng tiền ảnh (Pre-image Resistance: cho trước y , khó về mặt tính toán để tìm x sao cho $H(x)=y$), tính kháng tiền ảnh thứ hai (Second Pre-image Resistance: cho trước x_1 , khó tìm $x_2 \neq x_1$ sao cho $H(x_2)=H(x_1)$), và tính kháng va chạm mạnh (Collision Resistance: khó tìm bất kỳ cặp (x_1, x_2) phân biệt nào sao cho $H(x_1)=H(x_2)$). Đồng thời, hiệu ứng tuyết lở (avalanche effect) đảm bảo mọi biến đổi dù nhỏ nhất ở đầu vào cũng làm thay đổi ngẫu nhiên và toàn diện giá trị băm đầu ra [4].

Để quản lý và xác thực danh sách giao dịch trong khối với chi phí tối ưu, giao thức sử dụng cấu trúc Cây Merkle (Merkle Tree) [3]. Cây Merkle thường được biểu diễn dưới dạng cây nhị phân, trong đó mỗi nút lá lưu giá trị băm của một giao dịch ($H_j=H(T x_j)$), còn mỗi nút trung gian lưu giá trị băm của phép nối hai nút con trực tiếp i . Tiến trình băm cặp này thực hiện đệ quy từ các nút lá lên đỉnh để tạo ra một giá trị đại diện duy nhất là Gốc Merkle (*Merkle Root*) được ghi vào phần đầu khối [3]. Cây nhị phân là mô hình minh họa phổ biến; cấu trúc và cách xử lý số nút lẻ có thể khác giữa các nền tảng.



Hình 1.2. Cây Merkle và đường dẫn kiểm chứng một giao dịch

Cấu trúc cây Merkle cho phép kiểm tra một giao dịch thuộc tập giao dịch đã cam kết thông qua Bảng chứng Merkle (Merkle Proof). Với cây cân bằng, kích thước bằng chứng và số phép băm khi xác minh ở mức $O(\log_2 N)$, với N là tổng số lượng giao dịch trong khối [3], [10]. Một nút nhẹ (Light Node) chỉ cần lưu trữ phần đầu khối chứa *Merkle Root* và yêu cầu mạng lưới cung cấp chuỗi các nút băm trung gian nằm dọc theo đường dẫn kiểm chứng (Audit Path) từ giao dịch cần kiểm tra lên gốc cây. Bằng cách thực hiện tuần tự $\log_2 N$ phép băm, nút kiểm tra có thể tự xác thực tính hợp lệ của giao dịch mà không cần tải hay phân tích toàn bộ phần thân khối, giúp tối ưu hóa băng thông mạng và hiệu năng lưu trữ trong hệ thống phân tán [3], [10].

1.2.3. Mật mã khóa công khai và chữ ký số

Mật mã khóa công khai và chữ ký số thiết lập cơ chế xác thực nguồn gốc và kiểm soát ủy quyền đối với các giao dịch thay đổi trạng thái sổ cái [1], [5]. Theo chuẩn NIST FIPS 186-5 [5], một lược đồ chữ ký số an toàn được định nghĩa hình thức qua bộ ba thuật toán (*KeyGen*, *Sign*, *Verify*): thuật toán sinh khóa $KeyGen(1^\lambda) \rightarrow (sk, pk)$ nhận tham số an ninh λ để tạo cặp khóa bất đối xứng gồm khóa bí mật sk (dùng để ký) và khóa công khai pk (dùng để xác minh); thuật toán ký $Sign(sk, m) \rightarrow \sigma$ tạo chữ ký số σ từ thông điệp m và khóa sk ; thuật toán xác minh $Verify(pk, m, \sigma) \rightarrow Valid, Invalid$ kiểm tra tính hợp lệ của σ dựa trên pk và m . Mô hình tổng quát được biểu diễn theo công thức (1.2):

Không nên diễn giải chữ ký số nói chung như việc "mã hóa giá trị băm bằng khóa bí mật rồi giải mã bằng khóa công khai". Cách mô tả này không phản ánh mô hình của nhiều lược đồ hiện đại. Ở mức tổng quát, thuật toán *Verify* chỉ trả về kết quả hợp lệ hoặc không hợp lệ dựa trên khóa công khai, thông điệp và chữ ký; nó không khôi phục thông điệp bằng một phép giải mã [5].

Về phương diện an toàn thông tin, chữ ký số cung cấp ba thuộc tính cơ bản: xác thực nguồn gốc (Authentication), bảo toàn tính toàn vẹn (Integrity) và chống chối bỏ kỹ thuật (Non-repudiation) [1], [5]. Tuy nhiên, cần nhận thức rõ giới hạn biên của công nghệ: chữ ký số chỉ chứng minh tính hợp lệ về mặt toán học giữa bộ dữ liệu (m, σ, pk) , chứ không thể chứng minh thực thể hợp pháp thực sự trực tiếp ký nếu khóa bí mật sk đã bị lộ lọt, bị mã độc đánh cắp hoặc trích xuất trái phép từ thiết bị lưu trữ [1], [5]. Do đó, giá trị chứng minh của chữ ký trong thực tế phụ thuộc vào quản lý vòng đời khóa, bảo vệ thiết bị ký, gắn khóa với đúng danh tính, ghi nhật ký sử dụng và thu hồi kịp thời khi có sự cố [1], [5].

1.2.4. Định danh, dấu thời gian và nonce

Định danh trong Blockchain cho phép gắn giao dịch với một khóa hoặc tư cách thành viên có quyền [1]. Ở mạng không cấp quyền, địa chỉ thường được tạo từ khóa công khai hoặc dữ liệu dẫn xuất và chủ yếu mang tính bí danh. Ở mạng cấp quyền, khóa công khai có thể được gắn với danh tính tổ chức thông qua chứng thư số và hạ tầng khóa công khai [8]. Trong cả hai trường hợp, chữ ký hợp lệ chỉ chứng minh việc sử dụng khóa tương ứng; mối liên hệ giữa khóa và cá nhân hoặc tổ chức ngoài đời vẫn phụ thuộc quy trình đăng ký, cấp phát và quản trị danh tính.

Dấu thời gian hỗ trợ sắp xếp sự kiện và áp dụng các quy tắc có yếu tố thời hạn, nhưng không nên mặc nhiên coi thời gian do một nút cung cấp là bằng chứng

thời gian pháp lý. Mạng phân tán không có đồng hồ vật lý chung tuyệt đối; từng giao thức phải quy định nguồn thời gian, khoảng sai lệch chấp nhận và cách xử lý nút có đồng hồ sai [1], [2]. Khi nghiệp vụ cần bằng chứng thời gian có thẩm quyền, hệ thống phải kết hợp cơ chế xác nhận phù hợp ngoài Blockchain.

Nonce là giá trị được giao thức sử dụng để tạo tính duy nhất hoặc thứ tự. Trong PoW, nút khai thác thay đổi nonce để tìm giá trị băm thỏa điều kiện độ khó [2]. Trong mô hình tài khoản, nonce thường là bộ đếm tăng dần để các nút phát hiện giao dịch cũ hoặc sai thứ tự. Cơ chế này hỗ trợ chống phát lại khi được kiểm tra cùng chữ ký và ngữ cảnh giao dịch; nó không thay thế toàn bộ biện pháp bảo vệ phiên và giao thức [1], [10].

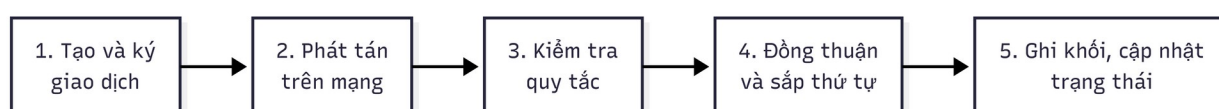
1.3. Kiến trúc và nguyên lý hoạt động

1.3.1. Mạng ngang hàng và vai trò của nút

Mạng ngang hàng (Peer-to-Peer - P2P) cho phép các nút trao đổi trực tiếp giao dịch, khối và thông tin trạng thái mà không bắt buộc mọi thông điệp phải đi qua một máy chủ trung tâm [1], [2]. Mỗi nút kiểm tra dữ liệu nhận được theo quy tắc giao thức trước khi chuyển tiếp cho các nút lân cận. Cách tổ chức này hỗ trợ nhân bản dữ liệu, tăng khả năng tiếp tục hoạt động khi một số kết nối gặp sự cố và làm giảm phụ thuộc vào một điểm truyền tin duy nhất. Tuy nhiên, P2P không đồng nghĩa với phân quyền tuyệt đối: các nút vẫn có thể tập trung tại cùng nhà cung cấp hạ tầng, cùng miền quản trị hoặc chịu ảnh hưởng của một nhóm vận hành nhỏ [1], [10].

Vai trò của nút phụ thuộc nền tảng. Nút đầy đủ thường lưu đủ dữ liệu cần thiết để tự kiểm tra lịch sử; nút nhẹ chỉ giữ một phần thông tin và dựa vào bằng chứng mật mã; nút đề xuất hoặc xác thực tham gia đồng thuận; một số nền tảng còn tách riêng nhiệm vụ sắp thứ tự, thực thi và lưu trữ [1], [8]. Vì vậy, các tên gọi như nút khai thác, nút xác thực hay nút lưu trữ không phải bộ phân loại áp dụng cho mọi Blockchain. Khi đánh giá kiến trúc, cần xác định rõ mỗi nút lưu gì, kiểm tra gì, thuộc tổ chức nào và có quyền tác động đến trạng thái ra sao.

1.3.2. Vòng đời giao dịch



Hình 1.3. Vòng đời giao dịch Blockchain ở mức khái quát

Vòng đời giao dịch bắt đầu khi chủ thể tạo yêu cầu thay đổi trạng thái và ký bằng khóa bí mật tương ứng. Nút tiếp nhận kiểm tra cấu trúc, chữ ký, số thứ tự chống phát lại, quyền thao tác và điều kiện trạng thái hiện tại [1]. Giao dịch hợp lệ được truyền tới các thành phần tham gia sắp thứ tự hoặc đồng thuận. Trong mạng công khai, giao dịch có thể được giữ trong vùng chờ trước khi nút đề xuất lựa chọn; mạng cấp quyền có thể sử dụng quy trình đề xuất, chứng thực và sắp thứ tự khác mà không có hoạt động khai thác [2], [8].

Sau khi đạt điều kiện chấp nhận của giao thức, giao dịch được đưa vào khối hoặc lô dữ liệu, truyền tới các nút và kiểm tra lần cuối trước khi cập nhật trạng thái. Ứng dụng chỉ nên thông báo thành công khi đã nhận bằng chứng giao dịch được cam kết hợp lệ, thay vì coi việc tạo mã giao dịch hay phát tán yêu cầu là hoàn tất. Mức độ chắc chắn của kết quả phụ thuộc cơ chế đồng thuận: một số mạng cần nhiều khối xác nhận, trong khi mạng dựa trên tíc số có thể đạt hoàn tất xác định dưới các giả định vận hành đã nêu [1], [6].

1.3.3. Sao chép trạng thái và thực thi xác định

Blockchain có thể được phân tích như một dạng sao chép máy trạng thái. Gọi S là trạng thái, T là giao dịch và δ là hàm chuyển trạng thái; cùng trạng thái đầu vào và cùng thứ tự giao dịch phải tạo cùng kết quả $S' = \delta(S, T)$ tại các nút trung thực [10]. Điều kiện xác định này đòi hỏi chương trình tránh phụ thuộc trực tiếp vào đồng hồ cục bộ, số ngẫu nhiên không được thống nhất, phản hồi mạng bên ngoài hoặc cách làm tròn khác nhau giữa các máy.

Nhiều nền tảng tách nhật ký giao dịch nối tiếp khỏi phần biểu diễn trạng thái mới nhất để vừa bảo toàn lịch sử vừa truy vấn hiệu quả. Nhật ký cho phép kiểm toán và tái dựng; trạng thái hiện thời phục vụ nghiệp vụ đọc thường xuyên. Đây là mô hình kiến trúc phổ biến nhưng cách tổ chức dữ liệu, xử lý giao dịch không hợp lệ và phục hồi trạng thái thay đổi theo từng nền tảng [8], [10].

1.3.4. Hợp đồng thông minh và dữ liệu ngoài hệ thống

Hợp đồng thông minh là chương trình biểu diễn quy tắc chuyển trạng thái và được các nút thực thi theo cơ chế của nền tảng [1], [10]. Nó có thể chuẩn hóa điều kiện nghiệp vụ và tạo dấu vết nhất quán, nhưng không tự bảo đảm logic được viết đúng. Lỗi phân quyền, kiểm tra đầu vào, xử lý trạng thái hoặc cơ chế nâng cấp có thể gây hậu quả trên toàn mạng. Vì vậy, mã cần được kiểm thử, rà soát an toàn, quản lý phiên bản và triển khai theo quy trình phê duyệt phù hợp.

Môi trường thực thi xác định không thể tùy ý lấy dữ liệu từ Internet vì các nút có thể nhận kết quả khác nhau. Dữ liệu ngoài hệ thống thường được đưa vào qua nguồn cung cấp có thẩm quyền hoặc cơ chế oracle. Chữ ký của nguồn chỉ chứng minh dữ liệu đến từ khóa tương ứng và không bị sửa sau khi ký; nó không chứng minh nội dung phản ánh đúng sự thật ngoài đời. Thiết kế an toàn cần quy định nguồn tin cậy, thời hạn hiệu lực, cách xử lý sai lệch và cơ chế dừng an toàn khi dữ liệu ngoài không đáng tin [1].

1.4. Cơ chế đồng thuận

1.4.1. Bài toán đồng thuận và mô hình lỗi

Đồng thuận giúp các nút thống nhất thứ tự giao dịch và trạng thái được chấp nhận trong điều kiện thông điệp có thể đến không đồng thời hoặc một số nút gặp sự cố [1]. Hai thuộc tính thường được xem xét là tính an toàn, nghĩa là các nút trung thực không chấp nhận hai quyết định mâu thuẫn trong cùng ngữ cảnh, và tính sống, nghĩa là hệ thống có thể tiếp tục đưa ra quyết định khi các giả định về mạng và số nút lỗi còn được thỏa mãn [6], [7]. Không giao thức nào bảo đảm mọi thuộc tính trong mọi điều kiện; mỗi thiết kế phải nêu mô hình mạng, loại lỗi và ngưỡng đối phương mà nó chịu được.

Mô hình chịu lỗi dừng (Crash Fault Tolerance - CFT) giả định nút lỗi chỉ dừng hoặc mất kết nối, không chủ động tạo thông điệp sai. Mô hình chịu lỗi Byzantine (Byzantine Fault Tolerance - BFT) mạnh hơn vì cho phép nút bị chiếm quyền gửi thông điệp khác nhau tới các bên, thông đồng hoặc cố ý vi phạm giao thức [6], [7]. Việc lựa chọn CFT hay BFT phải dựa trên ranh giới tin cậy: nút trong cùng miền quản trị và được bảo vệ tốt có thể chấp nhận giả định lỗi dừng, còn mạng liên tổ chức có nguy cơ hành vi tùy ý cần đánh giá BFT hoặc biện pháp kiểm soát bổ sung.

1.4.2. Proof of Work và Proof of Stake

Trong mạng không cấp quyền, một thực thể có thể tạo nhiều định danh với chi phí thấp để thao túng biểu quyết. Proof of Work (PoW) gắn quyền đề xuất khối với công việc tính toán, còn Proof of Stake (PoS) gắn quyền tham gia với lượng tài sản cam kết và các quy tắc khuyến khích hoặc xử phạt [1], [2]. Hai nhóm cơ chế này tạo chi phí cho tấn công Sybil mà không cần danh sách thành viên cố định.

PoW có lịch sử vận hành rõ trong Bitcoin nhưng tiêu thụ nhiều năng lượng và thường đạt hoàn tất theo xác suất [2]. PoS giảm nhu cầu tính toán lặp lại, song độ an toàn phụ thuộc cách chọn người đề xuất, cơ chế xử lý nhánh cạnh tranh,

phân phối cổ phần và quản trị giao thức. Không phải mọi PoS đều có cùng mô hình hoàn tất hoặc hiệu năng. Đối với mạng gồm các tổ chức đã định danh, PoW và PoS thường không phải lựa chọn đầu tiên vì quyền tham gia có thể được kiểm soát trực tiếp bằng danh tính và chính sách [1], [8].

1.4.3. Đồng thuận CFT và BFT trong mạng định danh

Raft là thuật toán CFT sử dụng nút dẫn dắt để sắp thứ tự và sao chép nhật ký; một mục được cam kết khi đạt đa số theo quy tắc của giao thức [7]. PBFT cổ điển hướng tới mô hình Byzantine và sử dụng các pha trao đổi thông điệp để các bản sao thống nhất thứ tự yêu cầu ngay cả khi có một số nút hành xử tùy ý [6]. So với PoW hoặc PoS, các giao thức này dựa trên tập thành viên đã biết và cơ chế tức số, nên phù hợp hơn với nhiều mạng cấp quyền.

Các ngưỡng chịu lỗi phụ thuộc giả định và giao thức, không phải công thức phổ quát cho mọi thuật toán CFT/BFT. Với cấu hình đa số CFT và mô hình PBFT cổ điển, quan hệ số nút thường được viết:

$$n \geq 2f + 1 \text{ (CFT)}; n \geq 3f + 1 \text{ (BFT)} \quad (1.3)$$

Trong đó n là tổng số nút và f là số nút lỗi tối đa. Với $n=3f+1$, một tíc số BFT thường có $2f+1$ phiếu; hai tíc số như vậy giao nhau ít nhất $f+1$ nút, nên phần giao chứa ít nhất một nút trung thực khi có tối đa f nút Byzantine [6]. PBFT cổ điển còn có chi phí truyền thông bậc hai $O(n^2)$ ở các pha nhiều nút trao đổi chéo, do đó chi phí tăng nhanh theo quy mô mạng [6].

1.4.4. Tính hoàn tất và đánh đổi thiết kế

Tính hoàn tất (finality) biểu thị mức độ chắc chắn rằng một giao dịch đã được chấp nhận sẽ không bị đảo ngược trong điều kiện an toàn giả định. Trong cơ chế hoàn tất theo xác suất, khả năng tổ chức lại lịch sử giảm dần khi có thêm khối xác nhận nhưng không bằng không về mặt tuyệt đối [2]. Trong cơ chế hoàn tất xác định dựa trên tức số, giao dịch được coi là hoàn tất sau khi đạt đủ phiếu theo giao thức; kết luận này vẫn phụ thuộc vào giả định số nút lỗi, an toàn khóa và quy tắc quản trị [6], [7].

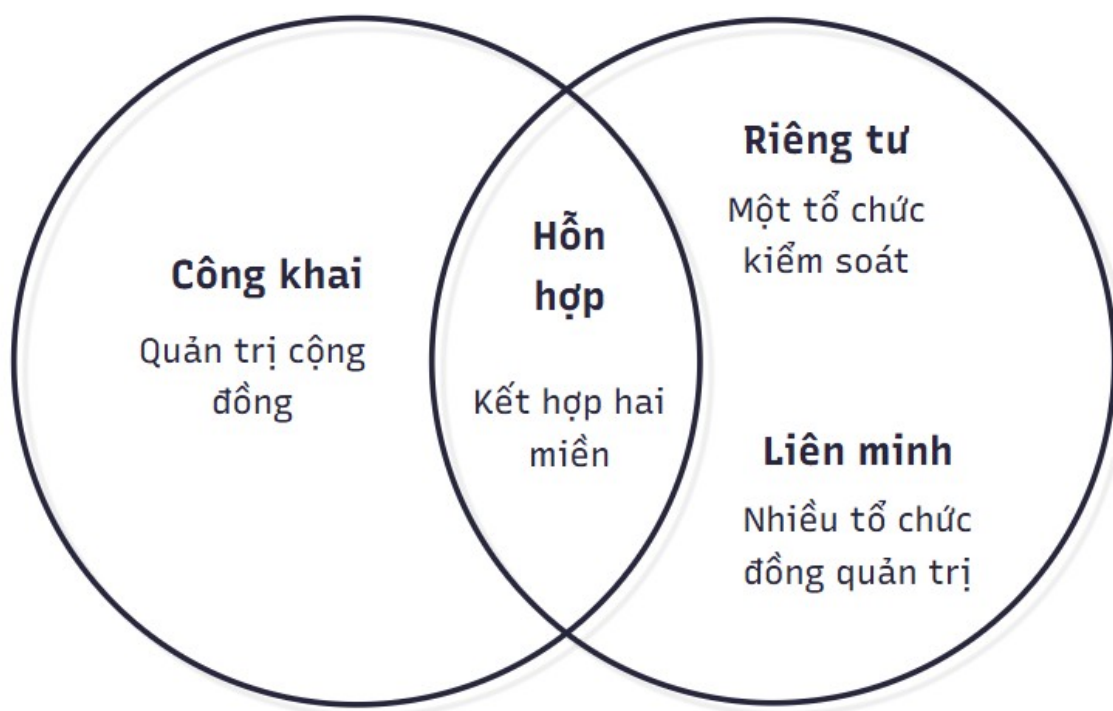
Tiêu chí	 Proof of Work	 Proof of Stake	 Mạng định danh
Căn cứ	Tính toán	Tài sản đặt cọc	Danh tính, biểu quyết
Ưu điểm	Mở, dễ kiểm chứng	Ít năng lượng hơn	Độ trễ thấp
Đánh đổi	Tốn năng lượng	Phức tạp kinh tế	Cần quản trị thành viên
Kết thúc	Thường xác suất	Xác suất/xác định	Thường xác định

Hình 1.4. So sánh khái quát các nhóm cơ chế đồng thuận

Lựa chọn đồng thuận là bài toán đánh đổi giữa mức phân tán quyền quyết định, khả năng chịu lỗi, độ trễ, thông lượng và chi phí vận hành [10]. “Bộ ba phân quyền – an toàn – khả năng mở rộng” có thể dùng như một khung trực giác, không phải định luật buộc mọi hệ thống chỉ được chọn hai thuộc tính. Cơ chế phù hợp phải được đánh giá trên tài thực tế, số tổ chức vận hành, loại hành vi đối phương và yêu cầu phục hồi, thay vì dựa trên tên gọi hoặc tuyên bố hiệu năng của nền tảng.

1.5. Phân loại Blockchain và điều kiện lựa chọn

1.5.1. Các trục phân loại



Hình 1.5. Phân loại Blockchain theo phạm vi tham gia và quyền truy cập

Blockchain có thể được phân loại dựa trên các trục tiêu chí hoàn toàn độc lập với nhau. Trước hết là tiêu chí về quyền tham gia: hệ thống được chia thành mạng "không cấp quyền" (bất kỳ ai cũng có thể tham gia theo quy tắc mở) và mạng "cấp quyền" (người tham gia phải được phê duyệt danh tính và vai trò từ trước) [1]. Bên cạnh đó, xét về phạm vi quản trị, hệ thống bao gồm mạng công khai (không thuộc sở hữu độc quyền của bất kỳ thực thể nào), mạng riêng (do một tổ chức duy nhất kiểm soát) và mạng liên minh (do một nhóm các tổ chức hợp tác cùng quản lý).

Ngoài ra, quyền truy cập và đọc dữ liệu là một yếu tố tách biệt hoàn toàn. Một mạng blockchain "cấp quyền", dù kiểm soát chặt chẽ người tham gia, vẫn có thể cho phép công chúng bên ngoài xem các dữ liệu hoặc bằng chứng tối thiểu. Ngược lại, một mạng "công khai" vẫn có quyền áp dụng các cơ chế mã hóa để bảo vệ nội dung riêng tư của người dùng. Chính vì sự độc lập của các tiêu chí này, không nên đồng nhất khái niệm "công khai" với "không cấp quyền", hay đánh đồng "riêng tư" với "cấp quyền"[1], [8]

1.5.2. So sánh các mô hình triển khai

Bảng 1.1. So sánh khái quát các mô hình triển khai Blockchain

Tiêu chí	Công khai, không cấp quyền	Riêng, cấp quyền	Liên minh, cấp quyền
Quản trị	Cộng đồng và quy tắc giao thức	Một tổ chức hoặc một miền quản trị	Nhiều tổ chức theo chính sách chung
Thành viên xác thực	Mở theo điều kiện chống Sybil	Danh tính được phê duyệt nội bộ	Danh tính thuộc các tổ chức thành viên
Khả năng kiểm chứng	Rộng, thường công khai	Chủ yếu trong nội bộ	Giữa các thành viên và bên được cấp quyền
Quyền riêng tư	Khó kiểm soát phạm vi phổ biến dữ liệu	Dễ giới hạn hơn theo chính sách nội bộ	Có thể phân vùng theo tổ chức và nghiệp vụ
Hiệu năng	Phụ thuộc mạng mở và cơ chế kinh tế	Có thể tối ưu trong phạm vi nhỏ	Phụ thuộc số tổ chức, tức số và hạ tầng
Bài toán phù hợp	Hệ sinh thái mở cần kiểm chứng rộng	Nhật ký và quy trình nội bộ	Phối hợp và kiểm toán liên tổ chức

Mạng mở tạo khả năng kiểm chứng rộng và chống Sybil bằng nguồn lực kinh tế, nhưng thường phát sinh phí, độ trễ và khó kiểm soát phạm vi phổ biến dữ liệu. Mạng riêng dễ quản trị và tối ưu hiệu năng, song giá trị phân tán niềm tin bị hạn chế nếu một tổ chức vẫn quyết định toàn bộ lịch sử. Mạng liên minh phân bổ quyền giữa nhiều tổ chức đã định danh, thuận lợi cho kiểm toán liên tổ chức và chính sách dữ liệu có kiểm soát, nhưng đòi hỏi thỏa thuận quản trị, vận hành nút và xử lý tranh chấp [1], [8], [10].

Không mô hình nào mặc nhiên tốt hơn. Hiệu năng thực tế phụ thuộc thuật toán đồng thuận, số nút, chính sách xác nhận, hạ tầng mạng, kích thước giao dịch và cách tổ chức dữ liệu. Vì vậy, các con số thông lượng hoặc độ trễ chỉ có ý nghĩa khi được đo trên cấu hình và tải xác định [10].

1.5.3. Khi nên và không nên sử dụng Blockchain

Nên xem xét Blockchain khi nhiều tổ chức cần cùng cập nhật trạng thái, lịch sử phải được kiểm chứng độc lập và không bên nào được chấp nhận làm chủ thể duy nhất có quyền sửa dữ liệu. Quy trình cũng phải đủ rõ để biểu diễn thành giao dịch, các bên có động lực vận hành chung và dữ liệu có thể được tối thiểu hóa phù hợp với yêu cầu riêng tư [1], [10].

Không nên sử dụng Blockchain nếu chỉ một tổ chức chịu trách nhiệm và được các bên tin cậy, nếu dữ liệu phải sửa hoặc xóa thường xuyên, hoặc nếu tải và độ trễ vượt khả năng của cơ chế đồng thuận. Một cơ sở dữ liệu truyền thống có nhật ký kiểm toán và chữ ký số có thể đơn giản, rẻ và dễ quản trị hơn trong các

trường hợp đó. Blockchain cũng không sửa được một quy trình nghiệp vụ mơ hồ, dữ liệu đầu vào kém chất lượng hay tranh chấp trách nhiệm giữa các tổ chức.

1.6. An toàn, quyền riêng tư và khả năng mở rộng

1.6.1. Mô hình đe dọa và bảo vệ nhiều lớp



Hình 1.6. Các lớp bảo vệ trong một hệ thống Blockchain

An toàn Blockchain không chỉ phụ thuộc hàm băm hoặc đồng thuận mà phải được xem xét theo nhiều lớp: mật mã, mạng, đồng thuận, hợp đồng thông minh, ứng dụng, lưu trữ ngoài chuỗi và quản trị vận hành [1], [10]. Tài sản cần bảo vệ gồm khóa bí mật, danh tính thành viên, trạng thái sổ cái, dữ liệu nghiệp vụ, cấu hình nút và nhật ký. Đối phương có thể là người ngoài tấn công hạ tầng, người dùng bị chiếm tài khoản, quản trị viên lạm quyền hoặc một nhóm nút thông đồng.

Biện pháp bảo vệ phải bám vào mô hình đe dọa. Kênh truyền cần được mã hóa và xác thực; nút cần vá lỗi, phân đoạn mạng, giám sát và sao lưu; chính sách đồng thuận phải giới hạn quyền của một tổ chức; ứng dụng phải kiểm soát đầu vào, phiên đăng nhập và phân quyền. Việc nhiều nút lưu sổ cái không thay thế kế hoạch ứng phó sự cố: nếu các nút cùng phụ thuộc một hạ tầng, cùng dùng cấu hình sai hoặc cùng bị lộ khóa, lỗi có thể lan rộng trên toàn hệ thống.

1.6.2. Quản lý khóa, hợp đồng thông minh và nguồn dữ liệu ngoài

Khóa bí mật đại diện cho quyền ký giao dịch nên cần được quản lý suốt vòng đời: sinh bằng nguồn ngẫu nhiên an toàn, lưu trong kho được bảo vệ, giới hạn mục đích sử dụng, luân chuyển, sao lưu có kiểm soát và thu hồi khi nghi ngờ lộ [5]. Mô-đun bảo mật phần cứng, chữ ký nhiều bên hoặc mật mã ngưỡng có thể giảm rủi ro một khóa hay một cá nhân trở thành điểm lỗi đơn, nhưng không loại bỏ nhu cầu quản trị và phục hồi.

Hợp đồng thông minh cần kiểm tra quyền, dữ liệu đầu vào, chuyển trạng thái và các trường hợp đồng thời; phiên bản mới phải có quy trình rà soát, kiểm thử và phê duyệt [10]. Nguồn dữ liệu ngoài phải được xác định thẩm quyền, ký số khi phù hợp và có thời hạn hiệu lực. Dùng nhiều nguồn chỉ hữu ích khi các nguồn đủ độc lập; nếu tất cả cùng lấy dữ liệu từ một hệ thống gốc, biểu quyết không làm dữ liệu đáng tin hơn.

1.6.3. Quyền riêng tư và dữ liệu cá nhân

Số cái được nhân bản và khó sửa đặt ra xung đột với yêu cầu hạn chế mục đích sử dụng, chỉnh sửa hoặc xóa dữ liệu cá nhân. Đưa trực tiếp họ tên, ngày sinh, điểm số hay tệp văn bản lên chuỗi làm tăng phạm vi sao chép và khó kiểm soát thời hạn lưu giữ. Mã hóa không giải quyết hoàn toàn vấn đề vì bản mã tồn tại lâu dài và khóa có thể bị lộ trong tương lai.

Cách tiếp cận phù hợp là tối thiểu hóa dữ liệu trên chuỗi, giữ hồ sơ chi tiết ở kho ngoài chuỗi có phân quyền và chỉ ghi bằng chứng mật mã hoặc trạng thái cần thiết cho kiểm chứng [1], [9]. Tuy nhiên, giá trị băm không mặc nhiên ẩn danh: nếu dữ liệu gốc dễ đoán, kẻ tấn công có thể thử các khả năng và so sánh kết quả; bản ghi cũng có thể liên kết với sự kiện hoặc định danh khác. Việc xóa dữ liệu ngoài chuỗi không xóa bằng chứng đã ghi, vì vậy thiết kế phải đánh giá khả năng liên kết, căn cứ xử lý và trách nhiệm của từng bên.

1.6.4. Khả năng mở rộng và khả năng tương tác

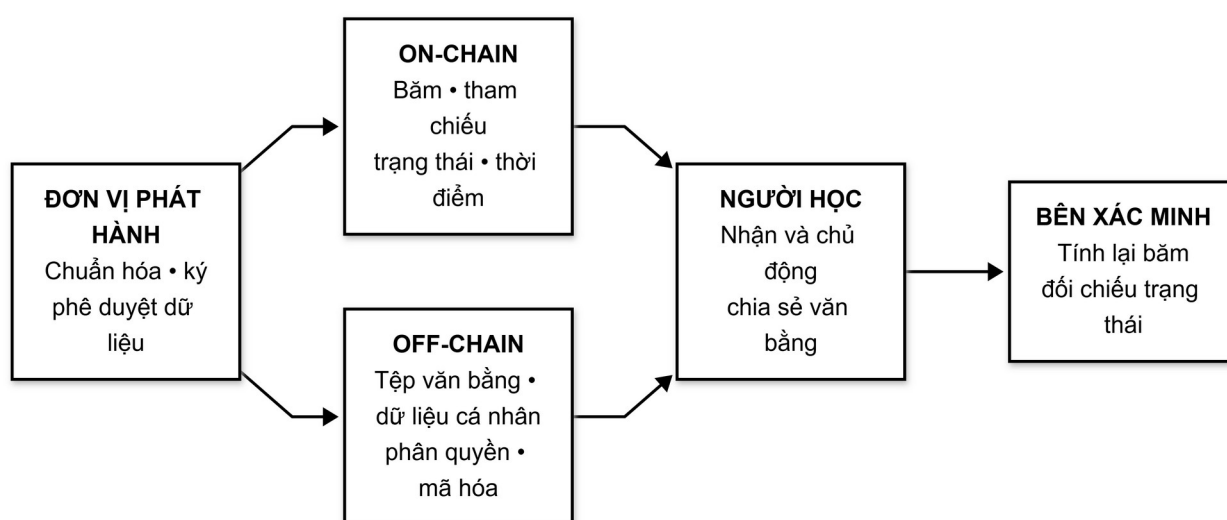
Khi nhiều nút cùng lưu và kiểm tra giao dịch, thông lượng, độ trễ, băng thông và dung lượng trở thành các giới hạn cần đo [10]. Có thể cải thiện bằng cách xử lý theo lô, tối ưu chính sách xác nhận, phân vùng dữ liệu hoặc đưa một phần xử lý ra ngoài chuỗi; mỗi cách tạo thêm giả định tin cậy và độ phức tạp. Các giải pháp lớp 2 chỉ nên được xem xét khi thật sự kế thừa an toàn từ chuỗi cơ sở, không nên đồng nhất mọi sidechain hoặc cầu nối với lớp 2.

Khả năng tương tác không chỉ là truyền thông điệp giữa hai mạng mà còn là thống nhất ý nghĩa dữ liệu, định danh tổ chức, phiên bản lược đồ và trạng thái nghiệp vụ. Cầu nối hoặc dịch vụ chuyển tiếp bổ sung điểm tin cậy mới và từng là mục tiêu của nhiều sự cố. Các chuẩn dữ liệu chứng thực như W3C Verifiable Credentials giúp thống nhất cách biểu diễn vai trò phát hành, chủ thể và bên xác minh, nhưng không tự giải quyết quản trị khóa, quyền truy cập hay tranh chấp pháp lý [9].

1.7. Định hướng ứng dụng trong quản lý văn bằng, chứng chỉ

1.7.1. Bài toán, tác nhân và yêu cầu kiểm chứng

Quản lý văn bằng, chứng chỉ liên quan ba vai trò chính: cơ sở đào tạo phát hành, người học nắm giữ hoặc chia sẻ và tổ chức bên ngoài xác minh [9]. Quy trình truyền thống thường dựa vào bản giấy, bản sao chứng thực hoặc yêu cầu xác nhận thủ công; khi dữ liệu nằm ở nhiều hệ thống, việc đối soát có thể chậm và khó phát hiện tài liệu đã bị chỉnh sửa. Một sổ bằng chứng dùng chung có thể hỗ trợ bên xác minh kiểm tra nguồn phát hành, tính toàn vẹn và trạng thái hiện thời mà giảm số bước liên hệ thủ công.



Hình 1.7. Mô hình khái niệm ứng dụng Blockchain trong quản lý văn bằng

Yêu cầu kiểm chứng không chỉ là so sánh mã băm. Hệ thống phải xác định đơn vị nào có thẩm quyền phát hành, khóa nào hợp lệ tại thời điểm ký, dữ liệu nào tạo thành nội dung văn bằng và trạng thái nào cho phép sử dụng. Kết quả không tìm thấy bằng chứng cũng không đủ để kết luận văn bằng giả, bởi hồ sơ cũ có thể chưa được số hóa hoặc nằm ngoài phạm vi hệ thống.

1.7.2. Mô hình dữ liệu khái niệm trên chuỗi và ngoài chuỗi

Ở mức khái niệm, sổ cái chỉ nên giữ lượng dữ liệu tối thiểu phục vụ kiểm chứng, chẳng hạn cam kết mật mã, tham chiếu đơn vị phát hành và trạng thái hiệu lực. Hồ sơ đầy đủ cùng dữ liệu cá nhân được quản lý ngoài chuỗi trong hệ thống có thẩm quyền, với mã hóa, phân quyền, nhật ký truy cập và sao lưu. Hai miền được liên kết bằng quy tắc tạo bằng chứng xác định để tài liệu bị thay đổi tạo kết quả kiểm tra khác [1], [9].

Cách phân tách này giảm việc sao chép dữ liệu nhạy cảm nhưng không làm rủi ro riêng tư biến mất. Bằng chứng trên chuỗi vẫn có thể bị liên kết hoặc dò đoán; kho ngoài chuỗi vẫn có thể bị mất, sửa hoặc ngừng phục vụ. Vì vậy, Chương 2 cần xác định mô hình dữ liệu, quy tắc chuẩn hóa, quyền truy cập và cơ chế xử lý khi dữ liệu nguồn sai mà không giả định Blockchain tự giải quyết các vấn đề đó.

1.7.3. Giá trị kỳ vọng, giới hạn và ranh giới trách nhiệm

Giá trị kỳ vọng là rút ngắn thời gian xác minh, tăng khả năng phát hiện tài liệu bị sửa và tạo lịch sử trạng thái dùng chung giữa các tổ chức. Công nghệ có thể hỗ trợ ghi nhận phát hành hoặc thay đổi hiệu lực, nhưng không tự quyết định người học đủ điều kiện tốt nghiệp, không chứng minh người xuất trình là chủ văn bằng và không đánh giá chất lượng đào tạo.

Cơ sở đào tạo vẫn chịu trách nhiệm về dữ liệu đầu vào và thẩm quyền phát hành; người vận hành chịu trách nhiệm bảo vệ khóa và hạ tầng; bên xác minh phải diễn giải kết quả đúng phạm vi. Nếu khóa phát hành bị lộ, bản ghi kỹ thuật hợp lệ vẫn có thể được tạo cho đến khi quyền bị thu hồi. Do đó, Blockchain phải được xem là công cụ hỗ trợ kiểm chứng, không phải nguồn phán quyết duy nhất.

1.8. Tổng kết Chương 1

1.8.1. Kết quả lý luận chính

Chương 1 đã trình bày Blockchain như một kiến trúc sổ cái phân tán kết hợp cấu trúc liên kết băm, chữ ký số, mạng ngang hàng và đồng thuận. Mức bảo đảm của hệ thống phụ thuộc đồng thời vào giả định mật mã, số nút lỗi, quản trị khóa, chất lượng ứng dụng và dữ liệu ngoài chuỗi; vì vậy không thể đồng nhất Blockchain với tính bất biến hay an toàn tuyệt đối [1], [10]. Phân tích cũng cho thấy việc lựa chọn mô hình công khai, riêng, cấp quyền hoặc không cấp quyền phải xuất phát từ ranh giới tin cậy và yêu cầu nghiệp vụ.

Đối với quản lý văn bằng, hướng tiếp cận phù hợp ở mức khái niệm là giữ hồ sơ cá nhân ngoài chuỗi và chỉ sử dụng sổ cái cho bằng chứng tối thiểu cùng lịch

sử trạng thái. Chương 2 sẽ chuyển các nguyên tắc này thành yêu cầu chức năng, yêu cầu an toàn, mô hình dữ liệu và luồng nghiệp vụ; đồng thời so sánh các phương án trước khi lựa chọn nền tảng, cấu trúc mạng và cơ chế triển khai cụ thể.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Bối cảnh số hóa văn bằng và nhu cầu xác minh

Chuyển đổi số trong giáo dục làm gia tăng việc phát hành, trao đổi và sử dụng văn bằng dưới dạng điện tử. Xu hướng này được thúc đẩy bởi nhu cầu học tập linh hoạt, tuyển dụng xuyên biên giới và yêu cầu cung cấp dịch vụ công trực tuyến. UNESCO xem chứng nhận số là một thành phần của hệ sinh thái giáo dục số và nhấn mạnh khả năng di chuyển, quyền kiểm soát dữ liệu của người học cùng cơ chế tin cậy giữa các tổ chức [1]. Khuyến nghị của Liên minh châu Âu về micro-credentials cũng đặt ra yêu cầu về tính minh bạch, khả năng kiểm chứng và khả năng mang theo của kết quả học tập [2]. Tại Việt Nam, định hướng phát triển giáo dục số tạo tiền đề cho hồ sơ học tập điện tử, nhưng đồng thời làm rõ hơn yêu cầu bảo vệ dữ liệu và xác minh nguồn gốc [3].

Trong quy trình truyền thống, đơn vị cấp thường lưu hồ sơ tập trung và cung cấp bản giấy, bản scan hoặc tệp PDF. Khi người học nộp hồ sơ cho nhà tuyển dụng hay cơ sở đào tạo khác, bên tiếp nhận phải kiểm tra thủ công qua email, điện thoại hoặc cổng tra cứu riêng. Quy trình này phụ thuộc giờ làm việc của đơn vị cấp, khó mở rộng khi số lượng yêu cầu tăng và không tạo ra một bằng chứng mật mã có thể kiểm tra độc lập. Bản sao điện tử cũng có thể bị sửa tên, chuyên ngành, xếp loại hoặc ngày cấp mà thay đổi không dễ nhận biết bằng quan sát thông thường.

Nhu cầu xác minh không chỉ là xác định tệp có tồn tại. Một cơ chế đầy đủ phải trả lời được các câu hỏi: ai là đơn vị phát hành; nội dung có còn nguyên vẹn; chứng thư có được neo vào một giao dịch hợp lệ; bằng chứng có thuộc đúng chứng thư đang kiểm tra; chứng thư có bị thu hồi; và kết quả không xác định do thiếu dữ liệu có bị phân biệt với kết quả không hợp lệ hay không. Ngoài ra, giải pháp phải hạn chế công khai dữ liệu cá nhân, không buộc mọi bên xác minh phải có tài khoản và vẫn cho phép người học tự lưu giữ, chuyển giao chứng thư.

Từ các yêu cầu trên, đề án xác định năm nhu cầu cốt lõi:

- Tính xác thực: nhận biết đúng đơn vị cấp và địa chỉ phát hành được công bố.
- Tính toàn vẹn: phát hiện mọi thay đổi đối với nội dung chứng thư sau khi phát hành.
- Khả năng kiểm chứng độc lập: bên thứ ba có thể kiểm tra chứng thư mà không cần truy cập cơ sở dữ liệu nội bộ của trường.
- Trạng thái hiệu lực: phân biệt chứng thư hợp lệ, không hợp lệ, đã thu hồi và chưa đủ dữ kiện kết luận.
- Quyền riêng tư và khả năng mang theo: dữ liệu chi tiết nằm trong chứng thư do người học nắm giữ; Blockchain chỉ lưu cam kết mật mã tối thiểu.

Blockcerts phù hợp với hướng tiếp cận này vì kết hợp chuẩn Verifiable Credentials/JSON-LD với Merkle proof và một giao dịch neo trên Blockchain [6], [9], [11], [14]. Tuy nhiên, công nghệ không thay thế trách nhiệm kiểm tra dữ liệu đầu vào hoặc thẩm quyền của đơn vị cấp. Vì vậy, hệ thống trong đồ án kết hợp Blockcerts với quy trình Maker–Checker, phân quyền RBAC, nhật ký kiểm toán và cơ chế thu hồi. Phần tiếp theo phân tích quy trình nghiệp vụ, tác nhân và phạm vi cụ thể của giải pháp.

2.2. Phân tích bài toán và phạm vi hệ thống

2.2.1. Quy trình nghiệp vụ hiện tại

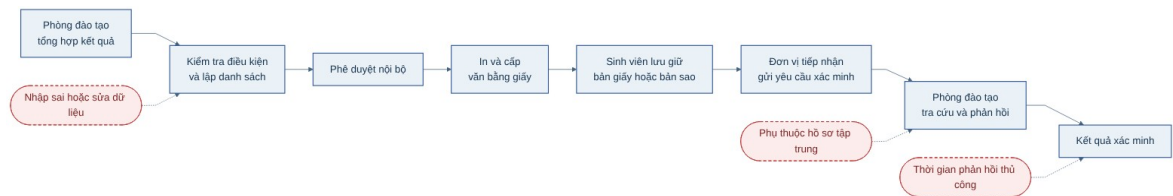
Trong phạm vi đồ án, quy trình hiện tại được mô hình hóa theo cách một cơ sở đào tạo tiếp nhận dữ liệu tốt nghiệp, kiểm tra điều kiện, phê duyệt danh sách, phát hành văn bằng và trả kết quả cho người học. Đây là mô hình nghiệp vụ khái quát phục vụ phân tích; không phải mô tả nguyên trạng của một đơn vị cụ thể. Dữ liệu đầu vào có thể được tổng hợp từ hệ thống quản lý đào tạo, bảng tính hoặc hồ sơ giấy. Sau khi đối chiếu, cán bộ có thẩm quyền phê duyệt danh sách và chuyển sang khâu tạo bản điện tử hoặc bản in.

Khi một bên thứ ba cần kiểm tra văn bằng, người học thường gửi bản sao hoặc ảnh chụp. Bên xác minh phải đánh giá dấu hiệu trên tài liệu, liên hệ nơi cấp hoặc tra cứu trên một cổng riêng nếu có. Chất lượng xác minh vì vậy phụ thuộc vào khả năng liên hệ, dữ liệu lưu trữ của đơn vị cấp và mức độ tin cậy của bản sao do người học cung cấp.

Bảng 2.1. Quy trình nghiệp vụ hiện tại và điểm yếu cần xử lý

Công đoạn	Bên tham gia	Dữ liệu vào	Dữ liệu ra	Điểm yếu cần xử lý
Tổng hợp hồ sơ	Cán bộ nghiệp vụ	Thông tin người học, chương trình, kết quả	Danh sách đề nghị cấp	Dễ sai khi nhập lại hoặc ghép dữ liệu từ nhiều nguồn
Kiểm tra và phê duyệt	Cán bộ kiểm tra, người có thẩm quyền	Danh sách đề nghị, hồ sơ đối chiếu	Danh sách được duyệt hoặc bị trả lại	Khó chứng minh ai thực hiện từng thao tác nếu nhật ký không đầy đủ
Phát hành	Đơn vị cấp	Danh sách đã duyệt, mẫu văn bằng	Văn bằng và bản điện tử	Bản sao điện tử có thể bị thay đổi sau khi phát hành
Bàn giao và lưu trữ	Đơn vị cấp, người học	Văn bằng đã phát hành	Bản người học nhận, hồ sơ lưu	Dữ liệu phân tán; việc khôi phục phụ thuộc kho lưu trữ nội bộ

Công đoạn	Bên tham gia	Dữ liệu vào	Dữ liệu ra	Điểm yếu cần xử lý
Xác minh	Người học, bên thứ ba, đơn vị cấp	Bản sao hoặc mã tra cứu	Kết luận xác minh	Phụ thuộc kênh liên hệ; khó kiểm tra ngoại tuyến hoặc độc lập
Thu hồi hoặc điều chỉnh	Đơn vị cấp	Quyết định thu hồi, định danh văn bản	Trạng thái thu hồi	Thông tin thu hồi có thể không đến kịp mọi bên đang giữ bản sao



Hình 2.1. Quy trình quản lý và xác minh văn bằng theo mô hình hiện tại

Hình 2.1 cho thấy nơi cấp giữ vai trò trung tâm ở cả khâu phát hành và xác minh. Bản sao do người học cung cấp không tự mang theo bằng chứng đủ để bên thứ ba kiểm tra tính toàn vẹn. Nếu cơ sở dữ liệu tra cứu tạm thời không sẵn sàng, quá trình xác minh có thể bị gián đoạn. Đây là vấn đề về khả năng kiểm chứng và truy vết, không đồng nghĩa mọi quy trình truyền thống đều thiếu giá trị pháp lý.

2.2.2. Các bất cập cần giải quyết

Từ quy trình trên, đồ án tập trung vào năm nhóm vấn đề. Thứ nhất, dữ liệu người học có thể bị nhập sai hoặc bị thay đổi trong quá trình chuyển giao giữa các khâu. Thứ hai, nếu người tạo yêu cầu đồng thời được tự phê duyệt và phát hành thì thiếu cơ chế kiểm soát chéo. Thứ ba, tệp điện tử thông thường không cung cấp bằng chứng mật mã để nhận biết nội dung đã bị sửa. Thứ tư, bên thứ ba khó xác minh độc lập khi chỉ có bản sao. Thứ năm, thông tin thu hồi phải được công bố theo cách có thể đối chiếu với chứng thư đã phát hành.

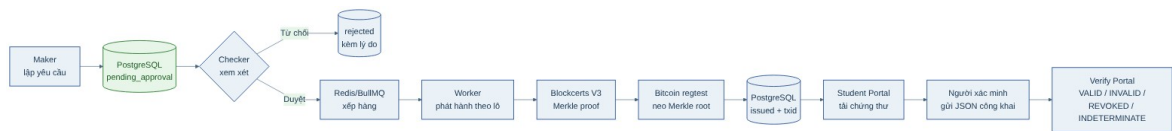
Blockchain không tự giải quyết độ chính xác của dữ liệu đầu vào. Nếu tên người học, chương trình hoặc ngày cấp được nhập sai nhưng vẫn được phê duyệt, việc neo dữ liệu chỉ làm cho sai sót đó khó bị chỉnh sửa âm thầm. Vì vậy, thiết kế phải kết hợp kiểm soát nghiệp vụ Maker–Checker, nhật ký thao tác, bằng chứng mật mã và cơ chế thu hồi.

2.2.3. Quy trình đề xuất

Quy trình đề xuất tách người lập yêu cầu (Maker) khỏi người phê duyệt (Checker). Maker nhập một hồ sơ hoặc một lô hồ sơ. Backend kiểm tra cấu trúc và

lưu trạng thái `pending_approval`. Checker xem danh sách, sau đó duyệt hoặc từ chối. Khi duyệt, hệ thống chuyển yêu cầu sang `queued` và đưa một công việc vào Redis/BullMQ. Worker xử lý bất đồng bộ, tạo chứng thư số Blockcerts V3, tạo cây Merkle và neo một Merkle root vào Bitcoin regtest bằng `OP_RETURN`. Mỗi chứng thư giữ target hash và đường dẫn Merkle riêng nhưng các chứng thư trong cùng lô có thể dùng chung giao dịch neo.

Người học đăng nhập Student Portal để xem và tải chứng thư thuộc tài khoản của mình. Người xác minh không cần tài khoản; họ gửi tệp JSON vào Verify Portal. Verifier kiểm tra cấu trúc, bằng chứng Merkle, giao dịch neo và trạng thái thu hồi, rồi trả về một trong bốn trạng thái `VALID`, `INVALID`, `REVOKED` hoặc `INDETERMINATE`.



Hình 2.2. Quy trình đề xuất với kiểm soát Maker–Checker và neo Bitcoin

So với quy trình hiện tại, thay đổi chính không phải là đưa toàn bộ văn bản lên Blockchain. PostgreSQL vẫn quản lý trạng thái nghiệp vụ; tệp chứng thư vẫn chứa dữ liệu cần xác minh; Bitcoin chỉ lưu cam kết mật mã dưới dạng Merkle root. Cách phân tách này hạn chế dữ liệu cá nhân xuất hiện trên chuỗi và cho phép nhiều chứng thư dùng chung một giao dịch neo.

2.2.4. Ranh giới, giả định và ràng buộc

Ranh giới hệ thống gồm ba frontend, backend NestJS, PostgreSQL, Redis/BullMQ, worker, bộ công cụ Blockcerts, verifier service và Bitcoin Core regtest. Hệ thống không quản lý quá trình xét tốt nghiệp, không thay thế hệ thống quản lý đào tạo và không xác nhận tính đúng của dữ liệu gốc trước khi Maker nhập. Đồ án cũng không xử lý giá trị pháp lý của văn bản điện tử, định danh quốc gia, chữ ký số chuyên dùng hoặc tích hợp với hệ thống thật của Học viện.

Các giả định nghiệp vụ gồm: tài khoản được cấp trước; Checker là bên có thẩm quyền phê duyệt và thu hồi; một yêu cầu chỉ được phát hành sau khi ở trạng thái `pending_approval`; mỗi tài khoản Student được định danh bằng `users.studentCode` và liên kết chứng thư qua `issued_certificates.studentId`; `recipientName` chỉ là dữ liệu hiển thị; khóa phát hành được quản lý ngoài cơ sở dữ liệu ứng dụng; thời gian của các container được đồng bộ đủ để ghi nhận nhật ký.

Các ràng buộc kỹ thuật gồm: tối đa 500 hồ sơ trong một yêu cầu lô; Bitcoin chạy ở regtest; queue hiện cấu hình một lần thử; dữ liệu thử nghiệm là dữ liệu tổng hợp có kiểm soát; việc xác minh phụ thuộc vào verifier service và Bitcoin RPC. Do đó, kết quả của đề án chỉ phản ánh nguyên mẫu trong môi trường thử nghiệm, không chứng minh mức sẵn sàng vận hành thực tế.

Bảng 2.2. Vai trò và giới hạn của Blockchain theo công đoạn

Công đoạn	Giá trị của Blockchain/Merkle	Giới hạn
Phê duyệt	Không trực tiếp giải quyết	Vấn cần Maker–Checker và RBAC
Phát hành	Neo cam kết mật mã của lô	Không chứng minh dữ liệu đầu vào đúng
Lưu trữ	Cho phép đối chiếu bằng chứng sau phát hành	Không thay thế sao lưu tệp chứng thư và PostgreSQL
Xác minh	Hỗ trợ kiểm tra tính toàn vẹn và giao dịch neo	Phụ thuộc parser, JSON-LD context, RPC và chính sách thu hồi
Thu hồi	Có thể ghi thêm dấu vết REVOKE: <certUid>	Cần quy tắc nhất quán giữa dấu vết on-chain và trạng thái DB

2.3. Thuật ngữ, tác nhân và quyền hạn

2.3.1. Quy ước thuật ngữ

Để tránh dùng lẫn đối tượng nghiệp vụ với đối tượng kỹ thuật, báo cáo sử dụng các thuật ngữ sau.

Bảng 2.3. Quy ước thuật ngữ trong đề án

Thuật ngữ	Nghĩa sử dụng trong đề án
Văn bản	Đối tượng nghiệp vụ ghi nhận kết quả đào tạo do đơn vị có thẩm quyền cấp
Chứng thư số Blockcerts	Tệp JSON biểu diễn số của văn bản, kèm bằng chứng phục vụ xác minh
Verifiable Credential	Mô hình dữ liệu chứng thực có thể xác minh theo W3C; là nền tảng khái niệm, không đồng nhất hoàn toàn với mọi trường của Blockcerts [9]
Certificate	Tên đối tượng kỹ thuật hoặc tệp chứng thư khi đề cập mã nguồn/API; trong diễn giải tiếng Việt ưu tiên “chứng thư số”
Issuer Profile	Tài liệu công bố thông tin và khóa công khai của đơn vị phát hành
Maker	Vai trò nghiệp vụ lập yêu cầu phát hành
Checker	Vai trò nghiệp vụ phê duyệt, từ chối, thu hồi và quản trị tài khoản trong nguyên mẫu

Thuật ngữ	Nghĩa sử dụng trong đồ án
Student	Vai trò người học, chỉ xem và tải chứng thư thuộc tài khoản của mình
Người xác minh	Tác nhân công khai sử dụng Verify Portal, không phải một vai trò đăng nhập
Worker	Thành phần kỹ thuật xử lý job; không phải tác nhân nghiệp vụ

Chứng thư Blockcerts được biểu diễn bằng JSON-LD; việc diễn giải trường @context tuân theo mô hình xử lý JSON-LD 1.1 [12]. Mô hình Data Integrity cung cấp khung khái niệm cho bằng chứng mật mã của Verifiable Credential [13]; trong hệ thống hiện tại, bằng chứng cụ thể là MerkleProof2019 theo revision đã chốt [14].

2.3.2. Phân tích tác nhân

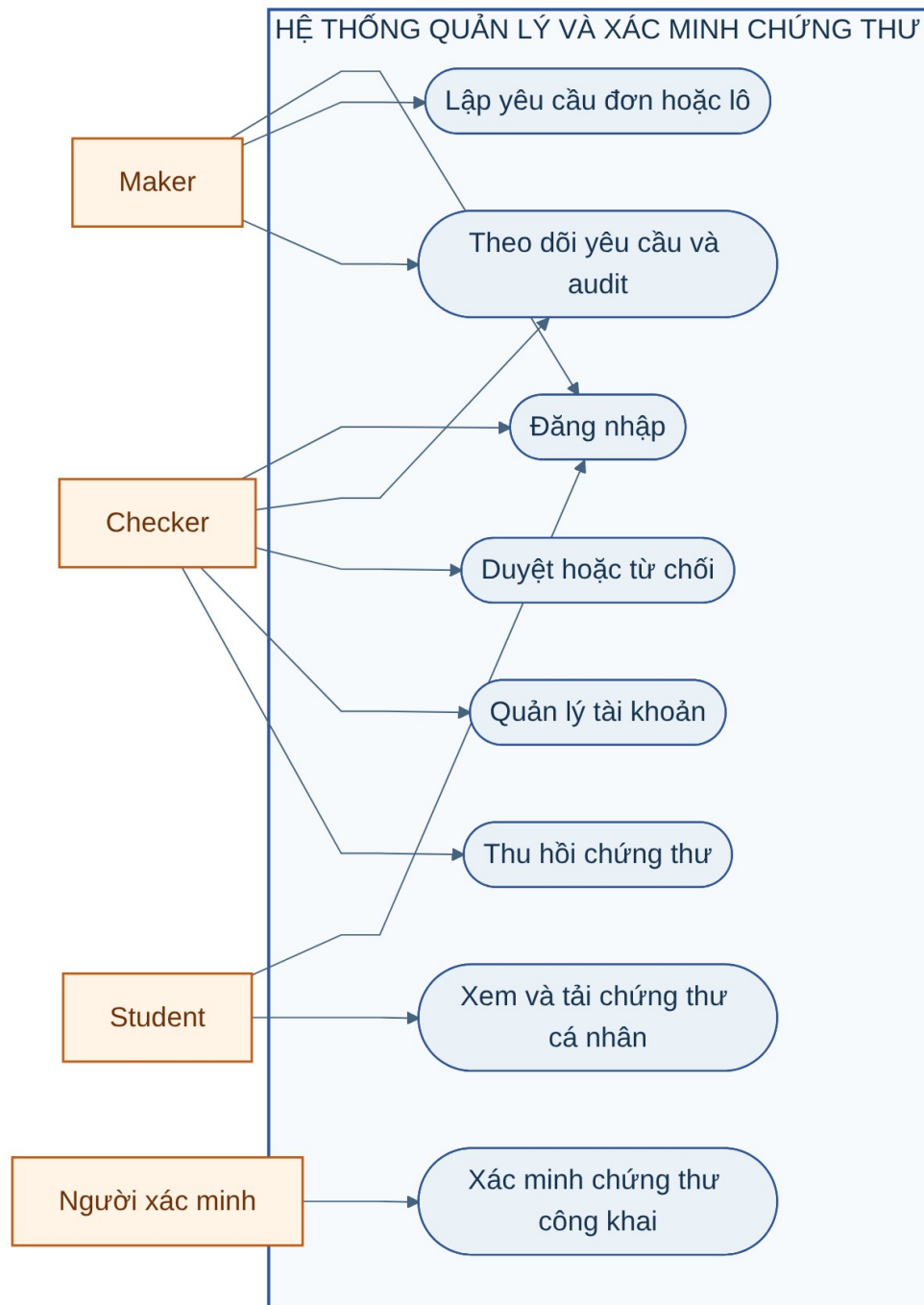
Bảng 2.4. Mục tiêu, quyền hạn và giới hạn của tác nhân

Tác nhân	Mục tiêu	Điều kiện tham gia	Dữ liệu được xem/thao tác	Hành động bị cấm hoặc giới hạn
Maker	Lập đúng yêu cầu phát hành	Có tài khoản mang vai trò maker	Tạo yêu cầu; theo thiết kế chỉ xem yêu cầu do mình tạo	Không duyệt, từ chối, thu hồi hoặc tạo tài khoản; API danh sách và chi tiết hiện chưa lọc theo requestedBy nên còn khoảng trống truy cập theo đối tượng
Checker	Kiểm soát và quyết định nghiệp vụ	Có tài khoản mang vai trò checker	Xem toàn bộ yêu cầu; duyệt/từ chối; thu hồi; xem audit; tạo và liệt kê tài khoản	Không được bỏ qua trạng thái; không thể duyệt yêu cầu đã rời pending_approval; chưa có khóa/mở tài khoản
Student	Nhận và quản lý chứng thư của mình	Có tài khoản Student; users.studentCode duy nhất và issued_certificates.studentId liên kết đúng tài khoản	Xem danh sách và tải JSON do backend trả kèm	Không xem chứng thư của Student khác; không tham gia phát hành/thu hồi
Người xác minh	Kiểm tra chứng thư do người giữ cung cấp	Không cần đăng nhập; có tệp JSON	Gửi chứng thư để kiểm tra và nhận kết luận	Không truy cập danh sách nội bộ hoặc sửa trạng thái chứng thư

Một tài khoản trong mô hình hiện tại chỉ lưu một giá trị role; mã nguồn không hỗ trợ đồng thời nhiều vai trò. Checker có thể tạo và liệt kê tài khoản, nhưng mã nguồn hiện chưa có chức năng khóa, mở khóa, sửa hoặc thu hồi tài khoản. Việc cấp tài khoản cũng chưa có quy trình phê duyệt hai bước. Đây là giới hạn quản trị cần được bổ sung nếu triển khai thật.

Worker, verifier service, Redis/BullMQ và Bitcoin Core là các thành phần kỹ thuật. Chúng được thể hiện trong sơ đồ kiến trúc và trình tự, không được coi là tác nhân nghiệp vụ trong bảng yêu cầu.

2.3.3. Mô hình use case tổng quát



Hình 2.3. Tác nhân và nhóm use case trong ranh giới hệ thống

Hình 2.3 đặt các chức năng bên trong đường biên hệ thống. Người xác minh chỉ tương tác với chức năng xác minh công khai. Các công việc tạo chứng thư, neo Bitcoin và lưu nhật ký là hành vi nội bộ được kích hoạt từ use case phê duyệt, không phải use case do Worker tự khởi tạo.

2.4. Yêu cầu chức năng và đặc tả use case

2.4.1. Danh sách yêu cầu chức năng

Bảng 2.5. Danh sách yêu cầu chức năng

Mã	Yêu cầu chức năng	Tác nhân khởi phát	Kết quả mong đợi
FR-01	Đăng nhập và nhận JWT	Maker, Checker, Student	Tài khoản tồn tại và đúng mật khẩu nhận token; nguyên mẫu chưa có khóa tài khoản
FR-02	Tạo và liệt kê tài khoản	Checker	Tài khoản mới được tạo, mật khẩu được băm và thao tác được ghi audit
FR-03	Tạo yêu cầu phát hành đơn hoặc lô	Maker	Hồ sơ hợp lệ được lưu ở <code>pending_approval</code>
FR-04	Phê duyệt yêu cầu	Checker	Toàn bộ yêu cầu hợp lệ chuyển <code>queued</code> , một job được enqueue
FR-05	Từ chối yêu cầu đang chờ	Checker	Yêu cầu chuyển <code>rejected</code> , lưu lý do và người xử lý
FR-06	Phát hành Blockcerts V3 và neo Bitcoin	Checker khởi phát; hệ thống xử lý	Job tạo đủ chứng thư, ánh xạ đúng hồ sơ và chuyển <code>issued</code>
FR-07	Xem, tải chứng thư của người học	Student	Trả bản ghi <code>issued/revoked</code> có <code>studentId/studentCode</code> khớp tài khoản; frontend tải JSON backend trả kèm
FR-08	Xác minh chứng thư công khai	Người xác minh	Trả <code>VALID</code> , <code>INVALID</code> , <code>REVOKED</code> hoặc <code>INDETERMINATE</code>
FR-09	Thu hồi chứng thư	Checker	Chỉ chứng thư <code>issued</code> được thu hồi và ghi nhận <code>txid</code> thu hồi
FR-10	Công bố Issuer Profile	Người xác minh	Trả hồ sơ issuer và khóa công khai dùng đối chiếu
FR-11	Công bố Revocation List	Người xác minh	Trả danh sách định danh đã thu hồi theo trạng thái DB

Mã	Yêu cầu chức năng	Tác nhân khởi phát	Kết quả mong đợi
FR-12	Tra cứu nhật ký nghiệp vụ	Maker, Checker	Trả danh sách audit có giới hạn số lượng; đây là quyền hiện có của nguyên mẫu
FR-13	Kiểm tra trạng thái tiến trình backend	Người vận hành/người xác minh	Endpoint /health phản hồi tiến trình đang hoạt động

2.4.2. UC-01 – Đăng nhập

Bảng 2.6. Đặc tả UC-01 – Đăng nhập

Thuộc tính	Đặc tả
Tác nhân chính	Maker, Checker hoặc Student
Mục tiêu	Nhận JWT để truy cập chức năng theo role
Tiền điều kiện	Tài khoản tồn tại và có mật khẩu đã bấm; nguyên mẫu chưa hỗ trợ trạng thái khóa tài khoản
Đầu vào	username, password
Luồng chính	Backend kiểm tra DTO; tìm tài khoản; so khớp bcrypt; phát JWT chứa sub, username, role và hạn 8 giờ
Luồng thay thế/ngoại lệ	Thiếu trường hoặc sai kiểu: từ chối validation; sai tên đăng nhập hoặc mật khẩu: trả cùng một lỗi xác thực
Hậu điều kiện	Không thay đổi dữ liệu nghiệp vụ; phía client lưu token cho phiên hiện tại
Phân quyền	Endpoint công khai; quyền của các request sau được xác định từ claim và dữ liệu người dùng
Tiêu chí chấp nhận	Đúng thông tin nhận token; token thiếu/sai/hết hạn không truy cập được endpoint bảo vệ

2.4.3. UC-02 – Tạo yêu cầu phát hành

Bảng 2.7. Đặc tả UC-02 – Tạo yêu cầu phát hành

Thuộc tính	Đặc tả
Tác nhân chính	Maker
Mục tiêu	Tạo một yêu cầu hoặc một lô yêu cầu để Checker xem xét
Tiền điều kiện	Maker đăng nhập; payload không rỗng; lô không quá 500 phần tử
Đầu vào	studentCode và pubkey bắt buộc; với lô là mảng recipients có studentCode riêng

Thuộc tính	Đặc tả
Luồng chính	Guard xác thực vai trò; DTO kiểm tra cấu trúc; backend sinh UUID, gán <code>requestedBy</code> , <code>batchId</code> nếu là lô; lưu <code>pending_approval</code> ; ghi audit
Luồng thay thế/ngoại lệ	Một phần tử không hợp lệ làm toàn request bị validation từ chối; payload quá 500 phần tử bị từ chối trước khi lưu
Hậu điều kiện	Chưa tạo chứng thư và chưa ghi Bitcoin; yêu cầu chờ Checker
Phân quyền	Thiết kế yêu cầu Maker chỉ tạo và xem bản ghi có <code>requestedBy</code> bằng username của mình; API hiện tại chưa thực thi bộ lọc đối tượng này
Tiêu chí chấp nhận	Số bản ghi lưu bằng số phần tử hợp lệ; mọi bản ghi có trạng thái ban đầu thống nhất

2.4.4. UC-03 – Phê duyệt hoặc từ chối

Bảng 2.8. Đặc tả UC-03 – Phê duyệt hoặc từ chối

Thuộc tính	Đặc tả
Tác nhân chính	Checker
Mục tiêu	Quyết định yêu cầu có được đưa vào hàng đợi phát hành hay không
Tiền điều kiện	Checker đăng nhập; danh sách <code>ids</code> có từ 1 đến 500 phần tử; mọi bản ghi tồn tại và ở <code>pending_approval</code>
Đầu vào	Danh sách <code>ids</code> ; khi từ chối có <code>reason</code>
Luồng phê duyệt	Backend loại id trùng; kiểm tra toàn bộ tập trước khi sửa; gán <code>approvedBy</code> , <code>batchId</code> ; chuyển <code>queued</code> ; enqueue một job; ghi audit
Luồng từ chối	Kiểm tra bản ghi và trạng thái; gán <code>rejectedBy</code> , lý do; chuyển <code>rejected</code> ; ghi audit
Ngoại lệ	Thiếu id, có bản ghi không tồn tại hoặc sai trạng thái; không cập nhật một phần; trả lỗi
Hậu điều kiện	Phê duyệt tạo một job cho tập id; từ chối không tạo job
Tiêu chí chấp nhận	Maker không gọi được endpoint; yêu cầu đã xử lý không thể được phê duyệt lại bằng luồng thông thường

2.4.5. UC-04 – Xử lý phát hành bất đồng bộ

Bảng 2.9. Đặc tả UC-04 – Xử lý phát hành bất đồng bộ

Thuộc tính	Đặc tả
Tác nhân chính	Hệ thống; được kích hoạt sau UC-03

Thuộc tính	Đặc tả
Mục tiêu	Tạo Blockcerts V3 cho toàn bộ yêu cầu trong job và neo một Merkle root
Tiền điều kiện	Job có 1–500 id; mọi bản ghi tồn tại, ở <code>queued</code> và có <code>pubkey</code>
Đầu vào	<code>ids</code> , <code>batchId</code>
Luồng chính	Worker chuyển các bản ghi sang <code>processing</code> ; tạo roster; cert-tools sinh JSON theo các chunk; cert-issuer ký và neo Merkle root; đào một block regtest; giải mã proof; ánh xạ nonce; lưu <code>certUid</code> , <code>txid</code> , <code>merkleRoot</code> ; chuyển <code>issued</code> ; ghi audit và manifest
Ngoại lệ	Thiếu hồ sơ, sai trạng thái, lỗi container/RPC, thiếu proof hoặc ánh xạ nonce sai: các bản ghi còn <code>processing</code> chuyển <code>failed</code> , lưu thông báo lỗi, job thất bại
Hậu điều kiện	Thành công: có đủ chứng thư và metadata neo; thất bại: không tuyên bố phát hành thành công
Tiêu chí chấp nhận	Số tệp đầu ra và số bản ghi <code>issued</code> bằng số đầu vào; các chứng thư cùng lô có cùng <code>txid/root</code> và proof riêng

Luồng phát hành dùng `batchId` làm idempotency key bền vững và `jobId` ổn định trên BullMQ. Trạng thái batch, transactional outbox, số lần thử, lock/lease và checkpoint giao dịch được lưu trong PostgreSQL/workspace. Nếu đã có `anchorTxid` hoặc `transaction intent`, Worker chỉ tra cứu, xác minh `OP_RETURN` và hoàn tất lại trạng thái; không tạo hoặc phát một giao dịch Bitcoin mới. Cách thiết kế này giới hạn retry, hỗ trợ reconciliation và đã được kiểm tra bằng lỗi giả lập trước broadcast và sau broadcast nhưng trước khi hoàn tất DB.

2.4.6. UC-05 – Xem và tải chứng thư của Student

Bảng 2.10. Đặc tả UC-05 – Xem và tải chứng thư của Student

Thuộc tính	Đặc tả
Tác nhân chính	Student
Mục tiêu	Nhận chứng thư liên kết với <code>studentId/studentCode</code> của tài khoản đã xác thực
Tiền điều kiện	Student đăng nhập; <code>users.id/studentCode</code> khớp <code>issued_certificates.studentId/studentCode</code> ; trạng thái <code>issued</code> hoặc <code>revoked</code>

Thuộc tính	Đặc tả
Đầu vào	Không có; backend lấy studentId/studentCode từ JWT và tài khoản đã xác thực
Luồng chính	Backend lọc theo studentId/studentCode và trả metadata cùng đối tượng chứng thư nếu đọc được tệp JSON; frontend cho phép tải đối tượng này xuống máy người dùng
Ngoại lệ	Nếu tài khoản không có studentCode hợp lệ thì từ chối/không trả chứng thư; nếu tệp không còn hoặc không đọc được thì trường certificate là null
Hậu điều kiện	Không thay đổi trạng thái chứng thư
Tiêu chí chấp nhận	Student A không nhận chứng thư của Student B theo studentId/studentCode; cả hai Student trùng tên nhưng khác mã phải đạt

2.4.7. UC-06 – Xác minh công khai

Bảng 2.11. Đặc tả UC-06 – Xác minh công khai

Thuộc tính	Đặc tả
Tác nhân chính	Người xác minh
Mục tiêu	Kiểm tra chứng thư JSON mà không cần tài khoản
Tiền điều kiện	Payload chứa đối tượng <code>certificate</code> ; kích thước request nằm trong giới hạn tiếp nhận
Đầu vào	Chứng thư Blockcerts dạng JSON
Luồng chính	Backend kiểm tra <code>credentialSubject.id</code> ; ghi tệp tạm; gọi verifier service; kiểm tra proof và anchor qua Bitcoin RPC; nếu hợp lệ thì tra trạng thái thu hồi theo <code>certUid</code> ; lưu verification log và audit
Kết quả	VALID: bằng chứng và anchor hợp lệ; INVALID: chứng thư/bằng chứng không hợp lệ; REVOKED: hợp lệ nhưng đã thu hồi; INDETERMINATE: chưa đủ điều kiện kết luận do lỗi phụ thuộc

Thuộc tính	Đặc tả
Ngoại lệ	Verifier/container/RPC lỗi: trả INDETERMINATE; lỗi ghi log không làm thay đổi kết luận xác minh
Tiêu chí chấp nhận	Không biến lỗi phụ thuộc thành INVALID; chứng thư đã thu hồi không trả VALID

Controller hiện tính biến trạng thái HTTP dự kiến nhưng chưa gắn biến đó vào response, vì vậy các request POST vẫn theo mã mặc định của NestJS. Báo cáo coi đây là điểm cần chỉnh ở hợp đồng API; không mô tả 422 như hành vi đã triển khai.

2.4.8. UC-07 – Thu hồi chứng thư

Bảng 2.12. Đặc tả UC-07 – Thu hồi chứng thư

Thuộc tính	Đặc tả
Tác nhân chính	Checker
Mục tiêu	Đánh dấu một chứng thư không còn hiệu lực và công bố dấu vết thu hồi
Tiền điều kiện	Bản ghi tồn tại, trạng thái issued, chưa có revocationTxid
Đầu vào	id nội bộ và reason
Luồng chính	Tạo giao dịch OP_RETURN chứa REVOKE:<certUid> trên regtest; đào block; cập nhật DB thành revoked; lưu revocationTxid, người, lý do, thời điểm; ghi audit
Ngoại lệ	Không có UTXO, RPC/ký thất bại hoặc trạng thái không phù hợp: không cập nhật revoked
Hậu điều kiện	Revocation List lấy trạng thái từ DB; giao dịch thu hồi là bằng chứng bổ sung
Tiêu chí chấp nhận	Maker/Student không thu hồi được; cùng chứng thư không bị thu hồi lần hai qua luồng chuẩn

Trong hiện trạng, DB là nguồn phục vụ quyết định nghiệp vụ và Revocation List; verifier không tự quét toàn bộ Blockchain để tìm chuỗi REVOKE:<certUid>. Nếu giao dịch thu hồi thành công nhưng lưu DB thất bại, hệ thống chưa có tiến trình tự động đối soát. Vì vậy, không thể coi hai nguồn luôn nhất quán.

2.4.9. UC-08 – Quản trị và công bố thông tin hỗ trợ

Checker tạo và liệt kê tài khoản qua /api/admin/users. Mã nguồn chưa có chức năng khóa/mở hoặc sửa tài khoản; thao tác tạo tài khoản được audit. Issuer Profile, Revocation List và /health là endpoint công khai. Endpoint health hiện chỉ chứng minh tiến trình backend phản hồi, chưa kiểm tra sâu PostgreSQL, Redis,

verifier service hoặc Bitcoin RPC. Audit cho phép Maker và Checker xem 200 bản ghi mới nhất; controller hiện chưa nhận tham số phân trang hoặc limit từ client.

2.5. Yêu cầu phi chức năng

Các yêu cầu phi chức năng được chia thành ba loại: ràng buộc đã cấu hình, mục tiêu thiết kế và chỉ tiêu cần kiểm chứng. Việc ghi một giá trị trong cấu hình không đồng nghĩa hệ thống đã đạt chỉ tiêu dưới tải. Những mục chưa có phép đo được đánh dấu là chưa đánh giá thay vì tự suy diễn.

Bảng 2.13. Yêu cầu phi chức năng và trạng thái kiểm chứng

Mã	Nhóm	Yêu cầu hoặc ràng buộc đo được	Căn cứ và trạng thái
NFR-01	Xác thực	JWT có hạn 8 giờ; JWT_SECRET tối thiểu 32 ký tự	Có trong <code>auth.module.ts</code> và <code>jwt.strategy.ts</code> ; chưa đánh giá cơ chế thu hồi token trước hạn
NFR-02	Mật khẩu	Mật khẩu lưu bằng bcrypt với cost factor 10	Có trong <code>auth.service.ts</code> và <code>admin.service.ts</code> ; phù hợp nguyên tắc không lưu mật khẩu rõ [17]
NFR-03	Phân quyền	Mọi endpoint nội bộ phải qua JWT và RolesGuard; mặc định từ chối khi thiếu quyền	Có kiểm tra role; Student được cô lập theo <code>studentId/studentCode</code> . Quyền xem danh sách của Maker cần tiếp tục giới hạn theo nhu cầu nghiệp vụ [16]
NFR-04	Giới hạn tần suất	Giá trị mặc định 120 request trong 60.000 ms trên backend	Có trong <code>app.module.ts</code> ; chưa benchmark hành vi khi vượt ngưỡng
NFR-05	Kích thước đầu vào	Nginx giới hạn body 10 MB; mỗi mảng tạo/duyệt tối đa 500 phần tử	Có trong Nginx và DTO; cần kiểm thử riêng JSON sát ngưỡng và JSON độc hại

Mã	Nhóm	Yêu cầu hoặc ràng buộc đo được	Căn cứ và trạng thái
NFR-06	Bất đồng bộ	API phê duyệt ghi trạng thái và outbox trong một transaction; job có retry/backoff hữu hạn, checkpoint và lịch sử trạng thái	Đã triển khai trong issuance.service.ts/dispatcher/processor với retry/backoff, outbox, checkpoint và reconciliation
NFR-07	Thời gian xử lý	Lệnh công cụ có timeout 30 phút; bước đào block có timeout 5 phút	Là ngưỡng kỹ thuật, không phải SLA; số liệu thực nghiệm trình bày ở Chương III
NFR-08	Toàn vẹn	Mỗi chứng thư phải có target hash/proof; Merkle root giải mã và txid phải là 64 ký tự hex	Worker kiểm tra trước khi ghi issued; bộ freeze đối chiếu lại artifact
NFR-09	Truy vết	Tạo, duyệt, từ chối, phát hành, thu hồi và quản trị tài khoản phải ghi audit; lần xác minh phải có verification log	Có hai entity log và các lời gọi tương ứng; chưa có chính sách chống sửa hoặc thời hạn lưu
NFR-10	Khả dụng	/health phải phản hồi khi backend sống	Hiện chỉ là liveness; chưa có readiness cho DB/Redis/RPC và chưa công bố SLA
NFR-11	Riêng tư	Không ghi dữ liệu cá nhân trực tiếp vào OP_RETURN; chỉ neo Merkle root hoặc định danh thu hồi	Phù hợp thiết kế hiện tại; tệp JSON và log vẫn cần kiểm soát truy cập
NFR-12	Khả năng kiểm chứng	Kết quả xác minh phải phân biệt INVALID với INDETERMINATE	Có bốn trạng thái nghiệp vụ; cần giữ phân biệt này trên UI và API
NFR-13	Phục hồi	Phải có quy trình sao lưu/khôi phục DB, workspace và khóa trước khi triển khai vận hành thực tế	Chưa triển khai và chưa kiểm thử RTO/RPO trong nguyên mẫu
NFR-14	Hiệu năng	Hệ thống phải xử lý lô đến 500 phần tử mà không làm API chờ toàn bộ phát hành	Giới hạn 500 đã có; khả năng chịu tải đồng thời chưa được kiểm chứng

Không đặt mục tiêu “API tạo lô dưới 2 giây” hay RTO/RPO cụ thể vì artifact hiện có chưa chứng minh các con số đó. Khi chuyển sang môi trường vận

hành, các chỉ tiêu phải được thỏa thuận theo hạ tầng, tải dự kiến và quy trình sao lưu, sau đó kiểm thử độc lập.

2.6. Mô hình đe dọa

2.6.1. Phương pháp đánh giá

Mô hình sử dụng STRIDE để nhóm đe dọa: giả mạo, sửa đổi, chối bỏ, lộ lọt thông tin, từ chối dịch vụ và nâng quyền. Khả năng xảy ra và mức ảnh hưởng được chấm từ 1 đến 3. Điểm rủi ro bằng tích hai giá trị: 1–2 là thấp, 3–4 là trung bình, 6–9 là cao. Đây là đánh giá sơ bộ phục vụ thiết kế nguyên mẫu, không thay thế kiểm thử xâm nhập.

Tài sản cần bảo vệ gồm tài khoản và JWT, dữ liệu yêu cầu, khóa phát hành, Issuer Profile, chứng thư JSON, trạng thái thu hồi, audit log, queue, workspace và kết nối Bitcoin RPC.

2.6.2. Ma trận tài sản–đe dọa–kiểm soát

Bảng 2.14. Ma trận tài sản–đe dọa–kiểm soát

ID	Tài sản và điểm tấn công	STRIDE	K/I/Điểm	Kiểm soát, phát hiện và phục hồi; rủi ro còn lại
T-01	Issuer Profile bị giả mạo qua endpoint hoặc file cấu hình	S/T	2/3/6 – Cao	Cố định nguồn cấu hình, HTTPS, đối chiếu khóa; hiện chưa có cơ chế ký riêng profile hoặc pin khóa phía client
T-02	Khóa công khai issuer bị thay thế	S/T/E	2/3/6 – Cao	Giới hạn quyền sửa file, quản lý thay đổi, audit; cần quy trình xoay khóa và công bố lịch sử khóa
T-03	Gửi lại request tạo/duyệt	T/R	2/2/4 – Trung bình	Kiểm tra trạng thái, batchId/jobId ổn định, transactional outbox và khóa idempotency bền vững
T-04	JWT bị đánh cắp	S/E/I	2/3/6 – Cao	Hạn 8 giờ, HTTPS, không ghi token vào log; chưa có refresh-token rotation, denylist hoặc MFA
T-05	Lạm dụng tài khoản Checker	E/R/T	2/3/6 – Cao	RBAC và audit; Checker hiện đồng thời quản trị tài khoản, duyệt và thu hồi nên mức tập trung quyền còn cao

ID	Tài sản và điểm tấn công	STRIDE	K/I/Điểm	Kiểm soát, phát hiện và phục hồi; rủi ro còn lại
T-06	IDOR hoặc truy cập trái quyền vào chứng thư	E/I	2/3/6 – Cao	Student được lọc bằng studentId/studentCode; quyền đối tượng của Maker/Checker được kiểm tra theo vai trò và vẫn cần rà soát khi mở rộng API
T-07	JSON quá lớn, cấu trúc sâu hoặc độc hại	D/T	2/2/4 – Trung bình	Nginx 10 MB, DTO, timeout container; chưa có giới hạn độ sâu JSON và quota theo người dùng
T-08	SSRF qua JSON-LD context hoặc URL ngoài	S/I/D	3/3/9 – Cao	safe-fetch áp HTTPS allowlist, kiểm tra DNS/IP ở mọi redirect, chặn private/link-local/loopback/metadata và giới hạn timeout/kích thước; 10 kiểm thử đạt [23]
T-09	Package/container bị cài cắm	T/E	2/3/6 – Cao	Lockfile, pin revision, checksum artifact, quét dependency; chưa có SBOM, ký image và quy trình cập nhật có phê duyệt
T-10	Audit log bị sửa hoặc xóa	T/R	2/2/4 – Trung bình	Maker và Checker đọc được qua API, DB kiểm soát truy cập; log chưa append-only và chưa neo checksum
T-11	Worker nhận job lặp	T/D	2/3/6 – Cao	State machine, batchId/jobId ổn định, outbox, lock/lease, checkpoint và reconciliation; fault injection trước/sau broadcast đã đạt

ID	Tài sản và điểm tấn công	STRIDE	K/I/Điểm	Kiểm soát, phát hiện và phục hồi; rủi ro còn lại
T-12	Bitcoin đã ghi nhưng DB cập nhật thất bại	T/R	2/3/6 – Cao	Lưu workspace và manifest hỗ trợ điều tra; chưa có tiến trình quét giao dịch để khôi phục trạng thái
T-13	Dữ liệu cá nhân rò từ log hoặc workspace	I	2/3/6 – Cao	Không đưa PII vào OP_RETURN, xóa file verify tạm; workspace phát hành và log vẫn cần phân quyền, retention và mã hóa backup

Các trường hợp T-08, T-11 và T-12 là rủi ro thiết kế quan trọng nhất. Kết quả dependency audit bằng không không loại trừ SSRF, lỗi logic hoặc tấn công chuỗi cung ứng. Tương tự, audit log hỗ trợ truy vết nhưng không tự bảo đảm tính bất biến khi người quản trị DB có quyền sửa dữ liệu.

2.7. Phân tích và lựa chọn kiến trúc

2.7.1. Blockchain công khai và Blockchain cấp quyền

Blockchain cấp quyền như Hyperledger Fabric hỗ trợ kiểm soát thành viên và chính sách tổ chức nhưng yêu cầu vận hành mạng consortium và mô hình quản trị riêng [8]. Blockchain công khai phù hợp hơn với mục tiêu để bên thứ ba kiểm tra anchor mà không cần tham gia mạng của cơ sở đào tạo. Tuy nhiên, đưa dữ liệu cá nhân trực tiếp lên mạng công khai là không phù hợp. Đề án vì vậy chọn cách chỉ neo cam kết mật mã.

Bảng 2.15. So sánh Blockchain công khai và Blockchain cấp quyền

Tiêu chí	Blockchain công khai	Blockchain cấp quyền	Lựa chọn cho đề án
Kiểm chứng bên ngoài	Bất kỳ bên nào có thể đọc dữ liệu chuỗi	Phụ thuộc quyền truy cập mạng	Ưu tiên khả năng kiểm chứng công khai
Quản trị	Theo quy tắc mạng công khai	Theo consortium	Nguyên mẫu không xây consortium
Riêng tư	Phải tránh đưa PII on-chain	Có thể kiểm soát thành viên	Chỉ neo Merkle root
Độ phức tạp thử nghiệm	Có thể mô phỏng bằng regtest	Phải dựng mạng tổ chức/nút	Chọn Bitcoin regtest
Phù hợp Blockcerts	Có công cụ và mô hình anchor Bitcoin	Cần tích hợp riêng	Chọn bộ công cụ Blockcerts hiện có [11], [14]

2.7.2. Bitcoin, Ethereum và Hyperledger Fabric

Bitcoin phù hợp vì mô hình Blockcerts đang dùng anchor dạng giao dịch Bitcoin và công cụ cert-issuer hỗ trợ quy trình này. Ethereum có khả năng lập trình smart contract nhưng đề án không cần logic on-chain; thêm hợp đồng thông minh sẽ tăng bề mặt tấn công và làm lệch mục tiêu xác minh Blockcerts. Fabric

phù hợp khi nhiều tổ chức đã có thỏa thuận quản trị mạng nhưng không cần thiết cho nguyên mẫu một đơn vị phát hành.

Bảng 2.16. So sánh nền tảng cho phạm vi đồ án

Tiêu chí	Bitcoin	Ethereum	Hyperledger Fabric
Cơ chế lưu dấu vết	OP_RETURN/giao dịch	Transaction hoặc smart contract	Ledger theo channel/chaincode
Blockcerts trong đồ án	Tương thích trực tiếp với công cụ đang dùng	Cần thay đổi bộ phát hành/xác minh	Cần tích hợp riêng
Khả năng lập trình	Hạn chế, đủ cho anchor	Cao	Cao theo chaincode
Môi trường thử nghiệm	Regtest chủ động đào block [18]	Mạng dev riêng	Mạng nhiều thành phần
Kết luận	Được chọn	Không cần smart contract	Không phù hợp phạm vi hiện tại

Regtest chỉ mô phỏng hành vi giao dịch trong mạng cục bộ; nó không phản ánh phí, độ trễ, biến động mempool, reorg hoặc mức tin cậy confirmation trên testnet/mainnet. Khi chuyển mạng, cần đổi cấu hình node/wallet, chính sách phí, số confirmation, quản lý khóa, giám sát giao dịch và xử lý reorg; đồng thời chạy lại toàn bộ kiểm thử tương thích.

2.7.3. Lưu toàn bộ dữ liệu và chỉ lưu cam kết mật mã

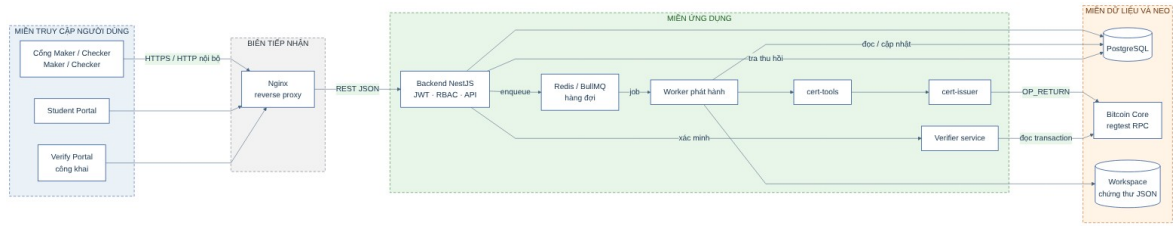
Lưu toàn bộ nội dung văn bản on-chain làm tăng rủi ro riêng tư và khó sửa sai dữ liệu. Thiết kế được chọn giữ nội dung trong chứng thư JSON và PostgreSQL/workspace, còn Blockchain chỉ chứa Merkle root. Với một lô, mỗi chứng thư được băm thành target hash, ghép vào cây Merkle và nhận một đường dẫn proof. Một giao dịch có thể neo root chung; vì vậy chi phí giao dịch không tăng tuyến tính theo số chứng thư trong lô, dù chi phí tạo tệp và proof vẫn tăng theo khối lượng.

2.7.4. Đồng bộ, bất đồng bộ, phát hành đơn và phát hành theo lô

Nếu API chờ cert-tools, cert-issuer và Bitcoin hoàn tất, request có thể kéo dài và khó phục hồi khi client ngắt kết nối. Queue bất đồng bộ tách tiếp nhận/phê duyệt khỏi xử lý nặng [21]. Phát hành theo lô tận dụng cây Merkle để dùng chung anchor, còn phát hành đơn là trường hợp lô có một phần tử. Đối lại, hệ thống phải quản lý trạng thái, job lập, retry và nhất quán DB–Blockchain. Nguyên mẫu mới giải quyết luồng chính và đánh dấu failed; chưa có retry tự động hay outbox.

2.8. Kiến trúc và các luồng xử lý

2.8.1. Kiến trúc thành phần và ranh giới tin cậy

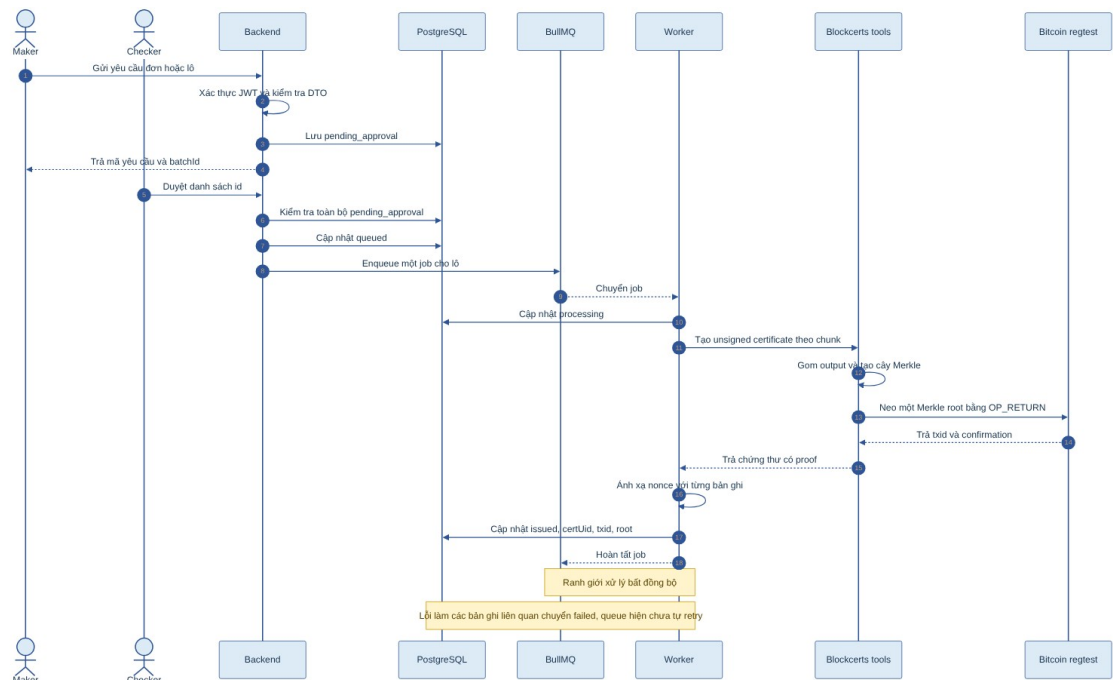


Hình 2.4. Kiến trúc thành phần và ranh giới tin cậy

Ba frontend có trách nhiệm khác nhau: Maker–Checker Portal phục vụ Maker/Checker; Student Portal phục vụ người học; Verify Portal phục vụ truy cập công khai. Nginx là biên tiếp nhận và giới hạn request. Backend thực hiện JWT, RBAC, validation và điều phối nghiệp vụ. PostgreSQL lưu trạng thái; Redis/BullMQ giữ job; Worker gọi cert-tools/cert-issuer; verifier service kiểm tra chứng thư; Bitcoin Core regtest cung cấp ledger thử nghiệm. Workspace lưu chứng thư và manifest, vì vậy phải được sao lưu cùng DB nếu cần khôi phục đầy đủ.

Ranh giới công khai nằm trước Nginx; ranh giới ứng dụng nằm giữa frontend và backend; ranh giới xử lý đặc quyền nằm ở Worker và các volume chứa khóa. JWT không được chuyển cho verifier service hoặc Bitcoin RPC. Cổng RPC và Redis không được công bố trực tiếp ra Internet trong kiến trúc mục tiêu.

2.8.2. Trình tự phát hành theo lô

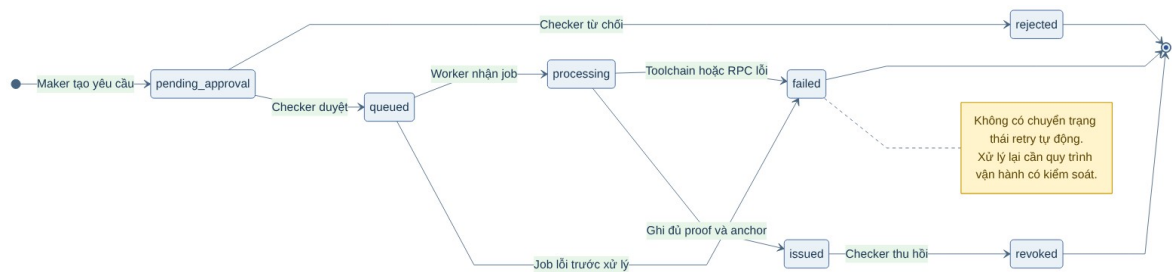


Hình 2.5. Trình tự phát hành chứng thư theo lô

Trình tự được chia thành ba pha. Pha nghiệp vụ gồm Maker tạo yêu cầu và Checker phê duyệt. Pha điều phối gồm backend chuyển trạng thái và enqueue một job. Pha phát hành do Worker thực hiện, gồm tạo JSON, Merkle proof, anchor, đào block regtest, ánh xạ output và lưu kết quả. Đường trả lời nét đứt biểu diễn phản hồi; đường liền biểu diễn lời gọi hoặc ghi dữ liệu. Ghi chú giữa queue và Worker đánh dấu ranh giới bất đồng bộ.

Hình dùng nhiều thông điệp kỹ thuật hơn số bước nghiệp vụ vì một bước như “phát hành lô” được triển khai bởi nhiều lời gọi nội bộ. Báo cáo không đồng nhất số thông điệp sequence với số bước nghiệp vụ.

2.8.3. Máy trạng thái yêu cầu phát hành



Hình 2.6. Máy trạng thái yêu cầu phát hành

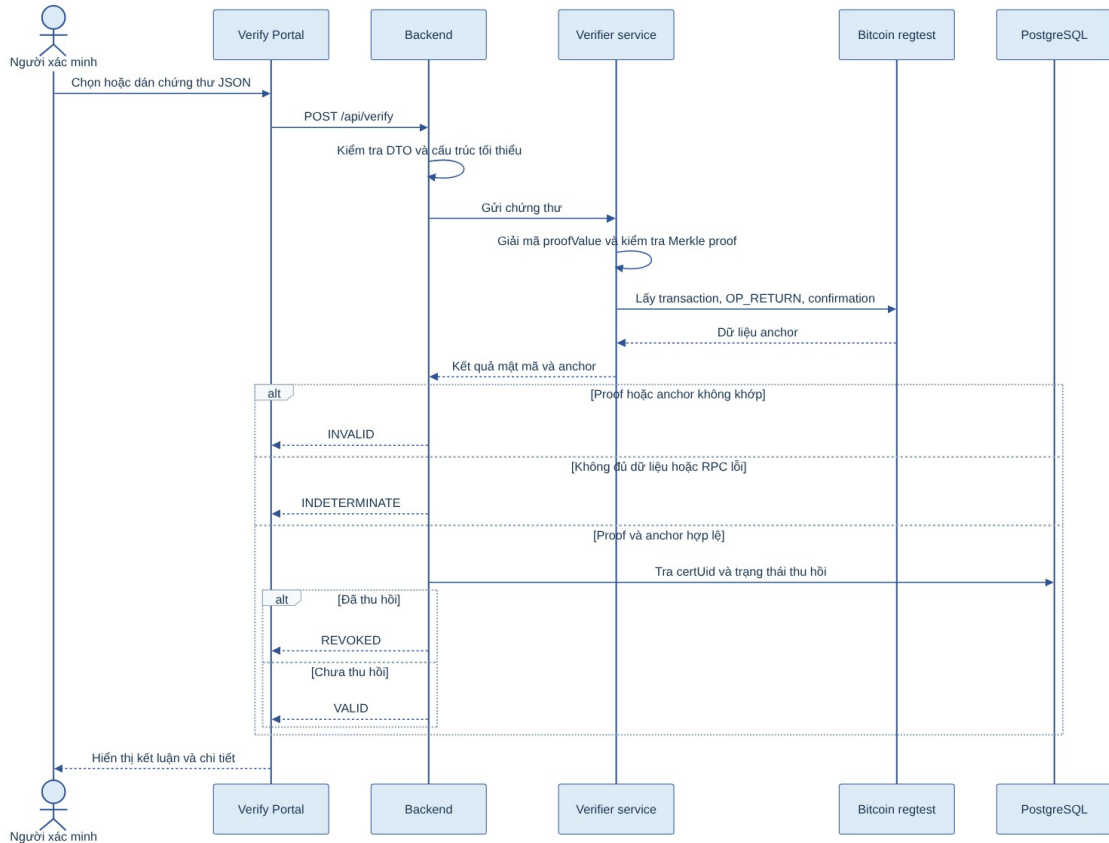
Trạng thái chuẩn trong mã nguồn là pending_approval, không phải pending_approve. Các chuyển tiếp hợp lệ được mô tả trong Bảng 2.17.

Bảng 2.17. Chuyển trạng thái yêu cầu phát hành

Trạng thái nguồn	Điều kiện/sự kiện	Trạng thái đích	Chủ thể
Khởi tạo	DTO hợp lệ và lưu DB thành công	pending_approval	Maker/backend
pending_approval	Checker duyệt toàn bộ id hợp lệ	queued	Checker/backend
pending_approval	Checker từ chối và lưu lý do	rejected	Checker/backend
queued	Worker nhận job, tiền điều kiện đặt	processing	Worker
processing	Có đủ JSON/proof, ánh xạ và lưu DB thành công	issued	Worker
processing	Phát sinh lỗi trong khối xử lý	failed	Worker
issued	Checker thu hồi và cập nhật DB thành công	revoked	Checker/backend

rejected, failed và revoked là trạng thái kết thúc trong luồng hiện tại. Chưa có endpoint retry từ failed; muốn xử lý lại phải có quy trình nghiệp vụ và cơ chế chống phát hành trùng trước khi bổ sung chuyển tiếp.

2.8.4. Trình tự xác minh công khai



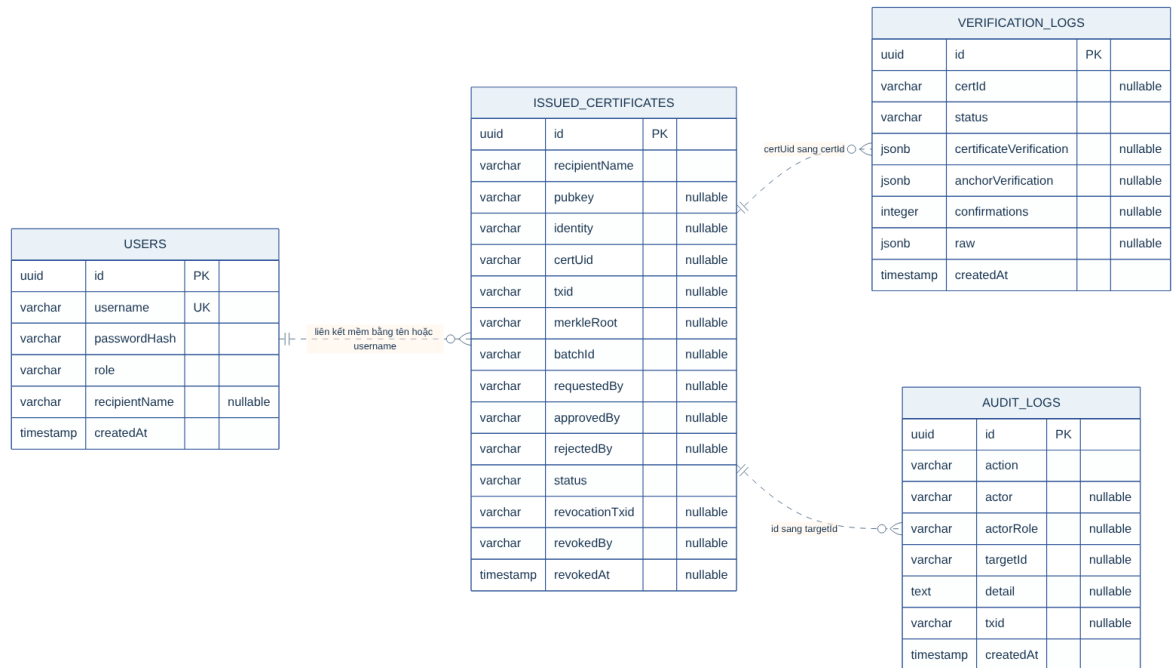
Hình 2.7. Trình tự xác minh công khai và các nhánh kết quả

Verifier service kiểm tra bằng chứng chứng thư và anchor. Backend sau đó dùng Revocation Service tra certUid trong PostgreSQL. Nếu bằng chứng sai, kết quả là INVALID; nếu hợp lệ và DB ghi đã thu hồi, kết quả là REVOKED; nếu phụ thuộc kỹ thuật không thể trả kết luận tin cậy, kết quả là INDETERMINATE. Trạng thái “chưa đủ confirmation” cần được biểu diễn là INDETERMINATE hoặc một kết quả chờ theo chính sách, không nên quy thành INVALID.

Việc không tải được Revocation List bên ngoài không xuất hiện trong luồng hiện tại vì verifier tra DB nội bộ qua service. Tuy nhiên, nếu Verify Portal độc lập khỏi backend trong tương lai, cần quy định rõ hành vi khi danh sách thu hồi không sẵn sàng.

2.9. Thiết kế dữ liệu

2.9.1. Mô hình dữ liệu đang triển khai



Hình 2.8. Mô hình dữ liệu vật lý hiện tại và liên kết logic

Mô hình hiện có bốn bảng. Các đường nét đứt trong Hình 2.8 là liên kết logic, không phải khóa ngoại vật lý. `issued_certificates.requestedBy`, `approvedBy`, `rejectedBy`, `revokedBy` lưu `username` nhưng entity không khai báo FK đến `users`. Tương tự, `audit_logs.targetId` và `verification_logs.certId` phục vụ truy vết nhưng không có FK đến chứng thư.

Bảng 2.18. Các bảng dữ liệu vật lý hiện tại

Bảng	Khóa/constraint chính	Vai trò	Hạn chế
users	UUID id; username unique; role; studentCode unique/nullable; recipientName là dữ liệu hiển thị	Tài khoản, role và định danh Student ổn định bằng studentCode	Role là chuỗi; chưa có trạng thái khóa, bảng quyền hoặc lịch sử thay đổi role
issued_certificates	UUID id; nhiều trường nullable; status	Vừa lưu yêu cầu vừa lưu chứng thư sau phát hành	Chưa có unique cho certUid; batch và request chưa tách bảng; không có FK actor

Bảng	Khóa/constraint chính	Vai trò	Hạn chế
verification_logs	UUID id; createdAt	Kết quả từng lần xác minh	Không FK; raw có thể chứa dữ liệu cần chính sách retention
audit_logs	UUID id; createdAt	Nhật ký hành động	Không append-only; không FK; chưa có chính sách xóa/lưu trữ

Quan hệ giữa lô và hồ sơ được thể hiện bằng chuỗi batchId; không có bảng issuance_batches. Điều này giúp nguyên mẫu đơn giản nhưng khó lưu trạng thái tổng thể, số lượng thành công/thất bại và lịch sử retry ở mức lô. Bảng issued_certificates đồng thời chứa dữ liệu trước và sau phát hành nên nhiều cột phải nullable, làm yếu khả năng ràng buộc ở DB.

2.9.2. Mô hình chuẩn hóa đề xuất cho giai đoạn sau

Nếu tiếp tục phát triển, nên tách issuance_batches, issuance_requests, certificates và revocations. issuance_batches quản lý trạng thái/job ở cấp lô; issuance_requests giữ dữ liệu Maker nhập và quyết định Checker; certificates chỉ được tạo khi phát hành thành công; revocations lưu mỗi quyết định thu hồi. Các bảng users, audit_logs, verification_logs tiếp tục tồn tại với FK phù hợp.

Đây là **phương án cải tiến**, chưa phải schema đang chạy. Báo cáo không dùng mô hình này để giải thích kết quả thực nghiệm hiện tại. Nếu áp dụng, cần migration dữ liệu, cập nhật entity/service, bổ sung unique/index và kiểm thử tương thích.

2.9.3. Ràng buộc, chỉ mục và chính sách dữ liệu

Các ràng buộc mục tiêu gồm: unique cho users.username và certificates.certUid; FK từ request đến Maker/Checker; FK từ certificate đến request và batch; index trên status, batchId, identity, createdAt; check constraint cho role và state; transaction khi đổi trạng thái và ghi audit. Với thao tác enqueue, mẫu transactional outbox phù hợp hơn việc lưu DB rồi gọi queue riêng rẽ.

Không nên xóa cứng chứng thư đã phát hành hoặc audit log qua API nghiệp vụ. Chính sách retention cần quy định riêng cho verification log, file tạm, workspace, backup và dữ liệu cá nhân. Nguyên mẫu hiện xóa file tạm xác minh trong finally, nhưng chưa có job dọn workspace phát hành hoặc thời hạn lưu log.

2.9.4. Nguồn sự thật đối với thu hồi

Trong code hiện tại, PostgreSQL là nguồn sự thật nghiệp vụ: Revocation List và kết quả REVOKED dựa vào revocationTxid trong bản ghi. Giao dịch Bitcoin chứa REVOKE:<certUid> là dấu vết bổ sung. Thứ tự xử lý là phát giao dịch trước, cập nhật DB sau; do đó tồn tại cửa sổ không nhất quán. Thiết kế cho môi trường vận hành thực tế cần một trạng thái trung gian như revocation_pending, lưu intent trước khi broadcast, sau đó worker xác nhận tx và hoàn tất; một tiến trình reconciliation phải dò lại các intent chưa hoàn thành.

2.10. Thiết kế API và phân quyền

2.10.1. Quy ước chung

API dùng tiền tố /api cho chức năng nghiệp vụ; /health không có tiền tố. Payload là JSON. Endpoint dành cho Student trả đối tượng chứng thư trong response để frontend tạo tệp tải xuống; backend hiện không có endpoint download riêng. DTO của NestJS thực hiện validation; JWT được truyền qua header Authorization theo lược đồ Bearer. Danh sách issuance hỗ trợ lọc status và batchId nhưng chưa phân trang; audit luôn trả tối đa 200 bản ghi. Vì vậy, phân trang và sắp xếp tùy chọn được ghi là yêu cầu cải tiến, không mô tả như chức năng sẵn có.

2.10.2. Đặc tả endpoint chính

Bảng 2.19. Đặc tả các endpoint chính

API và quyền	Đầu vào/validation chính	Kết quả và lỗi đáng chú ý	Ảnh xạ
POST /api/auth/login – công khai	username, password là chuỗi	Token JWT; sai tên đăng nhập hoặc mật khẩu: 401; validation: 400. Hiện POST thành công theo mặc định NestJS là 201	FR-01, UC-01
POST /api/auth/register – Checker	Username, password, role	Tạo user; trùng username: 409; sai quyền: 403	FR-02, UC-08

API và quyền	Đầu vào/validation chính	Kết quả và lỗi đáng chú ý	Ảnh xạ
GET/POST /api/admin/users – Checker	POST nhận username, password, role; Student bắt buộc có studentCode, recipientName là tên hiển thị	Danh sách/tài khoản mới không chứa passwordHash; trùng username: 409	FR-02, UC-08
POST /api/issue/request – Maker	student Code và địa chỉ P2PKH regtest pubkey	id, pending_approval; validation: 400	FR-03, UC-02
POST /api/issue/batch/request – Maker	recipients gồm 1–500 hồ sơ	batchId, count, danh sách id; toàn payload bị từ chối nếu DTO lỗi	FR-03, UC-02
POST /api/issue/batch/approve hoặc POST /api/issue/:id/approve – Checker	Lô: ids gồm 1–500 UUID; đơn: id trên URL	queued; lô trả thêm jobId, batchId; thiếu/sai trạng thái: 400/404	FR-04, UC-03
POST /api/issue/:id/reject – Checker	id trên URL; reason tùy chọn	rejected; sai trạng thái: 400; không thấy: 404	FR-05, UC-03
GET /api/issue?status=&batchId= và GET /api/issue/:id – Maker/Checker	Bộ lọc status, batchId tùy chọn; id trên URL	Hiện cả Maker và Checker đều có thể nhận bản ghi toàn hệ thống; đây là sai lệch so với mục tiêu Maker chỉ xem của mình	FR-03–06

API và quyền	Đầu vào/validation chính	Kết quả và lỗi đáng chú ý	Ảnh xạ
GET /api/issue/holder/certificates – Student	Không; backend dùng studentId/studentCode từ JWT và tài khoản	Danh sách issued/revoked; mỗi phần tử có thể kèm đối tượng certificate; không có endpoint tải riêng	FR-07, UC-05
POST /api/verify – công khai	{ certificate: object }	Bốn trạng thái nghiệp vụ; hiện controller chưa áp mã HTTP dự kiến nên cần chuẩn hóa contract	FR-08, UC-06
POST /api/revoke/:id – Checker; POST /api/revoke/check – công khai	Thu hồi: id trên URL, reason tùy chọn; kiểm tra: certUid	Thu hồi trả revoked và txid; sai trạng thái: 403; kiểm tra trả boolean và txid nếu có	FR-09, UC-07
GET /api/blockcerts/issuers/kma/profile.json – công khai	Không	Issuer Profile	FR-10
GET /api/blockcerts/issuers/kma/revocation-list.json – công khai	Không	revokedAssertions gồm id, lý do, thời điểm và txid thu hồi	FR-11
GET /api/audit – Maker/Checker	Không	200 audit log mới nhất; controller chưa hỗ trợ phân trang/limit	FR-12
GET /health – công khai	Không	{status:"ok", service:"backend"}; chỉ là liveness	FR-13

Các response lỗi hiện chủ yếu theo exception mặc định của NestJS và chưa có một envelope thống nhất. API dùng trong môi trường vận hành thực tế cần mã lỗi ổn định, requestId, thông điệp không lộ nội bộ, phân trang theo cursor hoặc page, giới hạn kích thước cụ thể ở cả Nginx và ứng dụng, cùng idempotency key cho thao tác tạo/duyệt.

2.10.3. Ma trận phân quyền

Bảng 2.20. Ma trận phân quyền chức năng

Chức năng	Maker	Checker	Student	Không đăng nhập
Đăng nhập	Có	Có	Có	Không
Tạo yêu cầu phát hành	Có	Không	Không	Không

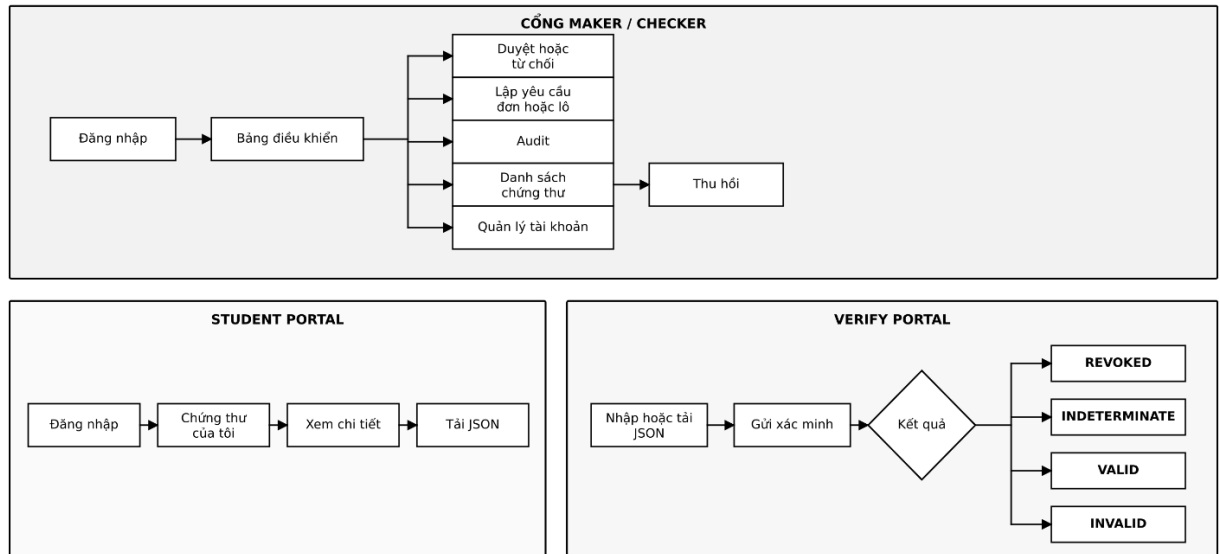
Chức năng	Maker	Checker	Student	Không đăng nhập
Xem yêu cầu	Mục tiêu: chỉ của mình; code hiện trả toàn bộ	Toàn bộ	Không	Không
Duyệt/từ chối	Không	Có	Không	Không
Quản trị tài khoản	Không	Có	Không	Không
Thu hồi	Không	Có	Không	Không
Xem/tải chứng thư	Không	Có qua danh sách nghiệp vụ	Theo studentId/studentCode của tài khoản	Không
Xác minh	Có thể dùng như công khai	Có thể dùng như công khai	Có thể dùng như công khai	Có
Xem audit	Có trong code hiện tại	Có	Không	Không
Issuer Profile/Revocation List/health	Có	Có	Có	Có

2.11. Thiết kế ba cổng giao diện

2.11.1. Nguyên tắc thiết kế

Giao diện phải làm rõ vai trò và trạng thái nghiệp vụ, không chỉ ẩn nút ở frontend mà còn dựa vào kiểm soát từ backend. Màu không được là tín hiệu duy nhất; nhãn trạng thái và biểu tượng phải có văn bản. Thông báo lỗi không hiển thị stack trace, đường dẫn máy chủ hoặc credential. Với tác vụ bất đồng bộ, giao diện phải phân biệt “đã tiếp nhận”, “đang xử lý”, “thành công” và “thất bại”.

2.11.2. Sơ đồ điều hướng



Hình 2.9. Sơ đồ điều hướng ba cổng giao diện

Maker–Checker Portal dùng cùng một ứng dụng nhưng menu thay đổi theo role. Maker thấy trang tổng quan, tạo yêu cầu đơn/lô và lịch sử của mình. Checker thấy hàng chờ phê duyệt, lịch sử toàn hệ thống, thu hồi, quản lý tài khoản và audit. Student Portal có trang đăng nhập, danh sách chứng thư, chi tiết và tải JSON. Verify Portal có trang tải/dán JSON và trang kết quả, không yêu cầu đăng nhập.

2.11.3. Đặc tả màn hình và trạng thái

Bảng 2.21. Đặc tả màn hình và trạng thái giao diện

Cổng/màn hình	Thành phần chính	Quy tắc và trạng thái lỗi
Maker/Checker – Đăng nhập	Username, password, nút đăng nhập	Sai thông tin hiển thị thông báo chung; không phân biệt username tồn tại hay không
Maker – Tạo yêu cầu đơn	Ba trường dữ liệu; nút gửi	Validate bắt buộc; sau gửi hiển thị id và <code>pending_approval</code>
Maker – Tạo lô	Nhập nhiều hồ sơ/roster; bộ đếm	Chặn rõ ràng và trên 500; báo rõ phần tử lỗi trước khi gửi
Checker – Hàng chờ	Bảng, chọn nhiều, chi tiết hồ sơ	Chỉ chọn <code>pending_approval</code> ; xác nhận trước khi duyệt/từ chối
Checker – Lịch sử/lô	Trạng thái, batchId, lỗi, txid	Không hiển thị <code>issued</code> khi worker chưa lưu đủ; lỗi có thông điệp an toàn
Checker – Thu hồi	Thông tin chứng thư, lý do	Chỉ bật với <code>issued</code> ; yêu cầu xác nhận vì không có thao tác hoàn tác trong nguyên mẫu

Cổng/màn hình	Thành phần chính	Quy tắc và trạng thái lỗi
Checker – Tài khoản/audit	Tạo, liệt kê user và xem log	Không trả passwordHash; chưa có khóa/mở/sửa tài khoản; Maker cũng đang được xem audit
Student – Danh sách	Chứng thư, trạng thái, ngày tạo	Chỉ dữ liệu của mình; revoked phải hiển thị nổi bật bằng chữ
Student – Chi tiết/tải	Metadata, txid, nút tải JSON	Nếu file thiếu, báo không thể tải thay vì trả đường dẫn máy chủ
Verify – Nhập chứng thư	Chọn file hoặc dán JSON	Từ chối file sai kiểu/quá lớn; không thực thi nội dung nhúng
Verify – Kết quả	Trạng thái, kiểm tra certificate, anchor, confirmation, thu hồi	VALID, INVALID, REVOKED, INDETERMINATE có giải thích riêng; không dùng màu làm tín hiệu duy nhất

Wireframe mức thấp được biểu diễn bằng sơ đồ điều hướng và đặc tả thành phần thay vì sao chép ảnh giao diện đã hiện thực ở Chương 3. Cách tách này giữ đúng trình tự: Chương 2 mô tả quyết định thiết kế, Chương 3 trình bày sản phẩm được xây dựng.

2.12. An toàn triển khai và vận hành

Backend phải chạy với `synchronize=false` và áp migration có kiểm soát [22]. Secret, WIF, mật khẩu RPC và JWT secret không được commit; file chứa khóa chỉ mount vào thành phần cần thiết với quyền tối thiểu. Redis, PostgreSQL và Bitcoin RPC chỉ mở trong mạng nội bộ. Nginx chấm dứt TLS ở môi trường thật, đặt giới hạn body, timeout và header bảo vệ phù hợp.

Đối với JSON-LD context hoặc URL do chứng thư tham chiếu, verifier áp allowlist HTTPS, phân giải DNS và kiểm tra địa chỉ đích ở mỗi lần truy cập/chuyển hướng; chặn loopback, private, link-local, metadata IPv4/IPv6, giới hạn thời gian và kích thước phản hồi theo hướng dẫn OWASP [23]. Bộ kiểm thử tự động bao phủ URL nội bộ, 127.0.0.1, 169.254.169.254, tên dịch vụ Docker, redirect và kết quả phân giải DNS. Rủi ro còn lại thuộc nhóm cấu hình allowlist sai hoặc thay đổi hạ tầng DNS, nên vẫn cần giám sát khi đưa ra môi trường bên ngoài.

Sao lưu phải bao gồm PostgreSQL, workspace chứng thư, manifest, cấu hình issuer và khóa theo quy trình tách biệt. Khôi phục cần được thử định kỳ; chỉ có file backup mà chưa restore thử không được coi là đạt. Cần giám sát độ sâu queue, job failed, dung lượng workspace, kết nối DB/Redis/RPC, giao dịch chưa đủ confirmation và sai lệch DB–Blockchain.

Audit log không được chứa mật khẩu, JWT, WIF, connection string hoặc toàn bộ chứng thư nếu không cần thiết. Khi vận hành thật, nên chuyển log sang

kho append-only hoặc hệ thống tập trung với phân quyền và retention rõ ràng. Checker không nên đồng thời có toàn quyền quản trị tài khoản và quyền can thiệp hạ tầng.

2.13. Tiêu chí chấp nhận và ma trận truy vết

2.13.1. Tiêu chí chấp nhận

Bảng 2.22. Tiêu chí chấp nhận

Mã	Tiêu chí	Yêu cầu liên quan
AC-01	Tài khoản hợp lệ đăng nhập được; sai mật khẩu, token sai, hết hạn hoặc thiếu bị từ chối	FR-01
AC-02	Chỉ Checker tạo/liệt kê tài khoản; response không chứa <code>passwordHash</code> ; ghi rõ chưa có khóa/mở tài khoản	FR-02
AC-03	Maker tạo yêu cầu đơn/lô đến 500 phần tử; payload rỗng, sai kiểu hoặc vượt giới hạn bị từ chối	FR-03
AC-04	Chỉ Checker duyệt/từ chối; toàn bộ id phải tồn tại và ở <code>pending_approval</code> ; không cập nhật một phần	FR-04, FR-05
AC-05	Một job lô tạo đủ chứng thư, proof riêng và một anchor chung; lỗi không bị báo thành công	FR-06
AC-06	Student chỉ nhận chứng thư có <code>studentId/studentCode</code> khớp tài khoản; frontend tải JSON backend trả về	FR-07
AC-07	Verifier phân biệt đúng <code>VALID</code> , <code>INVALID</code> , <code>REVOKED</code> , <code>INDETERMINATE</code> ; lỗi RPC không bị quy thành chứng thư giả	FR-08
AC-08	Chỉ Checker thu hồi chứng thư <code>issued</code> ; lần thu hồi được ghi DB, txid và audit	FR-09
AC-09	Issuer Profile công bố đúng định danh/khóa cấu hình và không lộ khóa bí mật	FR-10
AC-10	Revocation List chứa chứng thư đã thu hồi và không chứa chứng thư còn hiệu lực	FR-11

Mã	Tiêu chí	Yêu cầu liên quan
AC-11	Các thao tác nghiệp vụ trọng yếu sinh audit; Maker và Checker đọc được audit theo quyền hiện có của nguyên mẫu	FR-12
AC-12	/health phản hồi liveness; tài liệu không diễn giải đây là health sâu của toàn hệ thống	FR-13
AC-13	Request vượt body limit, JSON không hợp lệ và truy cập trái quyền bị từ chối an toàn	NFR-03-05
AC-14	Artifact sau khi chốt khớp checksum công bố; không suy diễn checksum thành bằng chứng hệ thống đã sẵn sàng vận hành thực tế	NFR-08

2.13.2. Ma trận FR–use case–thành phần–API–dữ liệu–kiểm thử

Bảng 2.23. Ma trận truy vết yêu cầu–thiết kế–kiểm thử

F R	U s e c a s e	Thành phần/API	Dữ liệu chính	Bảng chứng kiểm thử hiện có hoặc khoảng trống
FR-01	UC-01	AuthModule; /api/auth/login	users	E2E backend và kiểm tra RBAC; chưa kiểm thử brute-force chuyên sâu
FR-02	UC-08	mô-đun quản trị tài khoản; /api/admin/users	users, audit_logs	E2E quyền Checker; chưa có chức năng khóa/mở/sửa tài khoản
FR-03	UC-02	IssuanceModule; request/request-batch	issued_certificates	Kịch bản phát hành đơn/lô và validation giới hạn
FR-04	UC-03	approve; BullMQ	issued_certificates, queue	E2E Maker–Checker; chưa kiểm thử cạnh tranh hai Checker
FR-05	UC-03	reject	issued_certificates, audit_logs	Có luồng chức năng; cần ca tự động riêng cho lý do/từ chối lặp

FR	Use case	Thành phần/API	Dữ liệu chính	Bảng chứng kiểm thử hiện có hoặc khoảng trống
FR - 06	UC -04	Worker, cert-tools, cert-issuer, Bitcoin	DB, workspace, manifest, tx	B12 kiểm tra lô và 1.830 proof; bản cập nhật kiểm thử crash, retry, idempotency và reconciliation
FR - 07	UC -05	/api/issue/holder/certificates	DB và file JSON	Đã lọc theo studentId/studentCode; ca hai Student trùng tên và Student isolation đạt; tiếp tục mở rộng kiểm thử IDOR
FR - 08	UC -06	VerificationModule, verifier service	verification_logs, Bitcoin RPC	Ba nhóm kịch bản đạt 6/6, 9/9, 8/8; chưa có penetration test
FR - 09	UC -07	RevocationModule, Bitcoin RPC	trạng thái, revocationTxid, audit	Có kịch bản thu hồi-xác minh; chưa kiểm thử DB lỗi sau broadcast
FR - 10	UC -08	/api/blockcerts/issuers/kma/profile.json	cấu hình issuer	Kiểm tra thủ công/artifact; chưa có kiểm thử xoay khóa
FR - 11	UC -08	/api/blockcerts/issuers/kma/revocation-list.json	issued_certificates	Đối chiếu kịch bản thu hồi; chưa kiểm thử phân trang/dữ liệu lớn

F R	U s e c a s e	Thành phần/API	Dữ liệu chính	Bảng chứng kiểm thử hiện có hoặc khoảng trống
FR - 12	UC -08	AuditModule; /api/audit	audit_logs	Maker và Checker đọc được theo code; chưa có kiểm thử chống sửa log
FR - 13	UC -08	HealthController	Không	Smoke test liveness; chưa có readiness DB/Redis/RPC

Ma trận cho thấy artifact hiện tại đủ chứng minh các luồng chính trong phạm vi regtest. Bản cập nhật đã bổ sung kiểm thử idempotency, transactional outbox, worker crash, retry/reconciliation và SSRF. Các phạm vi chưa được chứng minh đầy đủ vẫn gồm tải đồng thời dài hạn, mất kết nối RPC kéo dài, sao lưu/khôi phục toàn hệ thống, reorg/mainnet, xoay khóa và kiểm thử xâm nhập độc lập.

2.14. Kết luận Chương 2

Chương 2 đã đi từ quy trình nghiệp vụ hiện tại đến quy trình đề xuất, xác định thuật ngữ, bốn tác nhân, mười ba yêu cầu chức năng, yêu cầu phi chức năng đo được, mô hình đe dọa, lựa chọn kiến trúc, luồng bất đồng bộ, mô hình dữ liệu, API, giao diện và ma trận truy vết. Thiết kế sử dụng ba role đăng nhập là Maker, Checker và Student; Người xác minh là tác nhân công khai không có role. Worker chỉ là thành phần nội bộ.

Giải pháp chọn Blockcerts V3 và Bitcoin regtest, giữ dữ liệu nghiệp vụ trong PostgreSQL/workspace và chỉ neo Merkle root. Bản hiện thực đã bổ sung studentId/studentCode, state machine, transactional outbox, checkpoint, retry/reconciliation cho phát hành và lớp safe-fetch chống SSRF. Các cơ chế này nâng độ bền của nguyên mẫu nhưng không biến PostgreSQL và Bitcoin thành một giao dịch ACID chung, không khắc phục dữ liệu đầu vào sai và chưa giải quyết đầy đủ quản trị khóa hoặc khôi phục thảm họa. Chương 3 trình bày cách hiện thực, triển khai và bằng chứng trong phạm vi đó.

CHƯƠNG 3. XÂY DỰNG, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

3.1. Môi trường và công nghệ triển khai

3.1.1. Môi trường thực nghiệm

Bản triển khai được tái lập trên máy ảo Ubuntu 22.04.5 LTS, kernel 6.8.0-138-generic, 4 vCPU Intel Xeon Gold 6130, khoảng 19 GiB RAM và phân vùng hệ thống 100 GB. Mã nguồn được chốt tại hai commit af9ae0c (Student identity, state machine/outbox/reconciliation) và 8d20eb2 (phòng chống SSRF). Số liệu benchmark lịch sử vẫn lấy từ bộ artifact đóng băng b12-frozen-20260912-r2; còn bằng chứng triển khai và kịch bản thao tác trong mục hướng dẫn tái lập được chạy lại trên máy mới. Bitcoin chạy regtest, cho phép chủ động tạo block mà không dùng tiền thật [18].

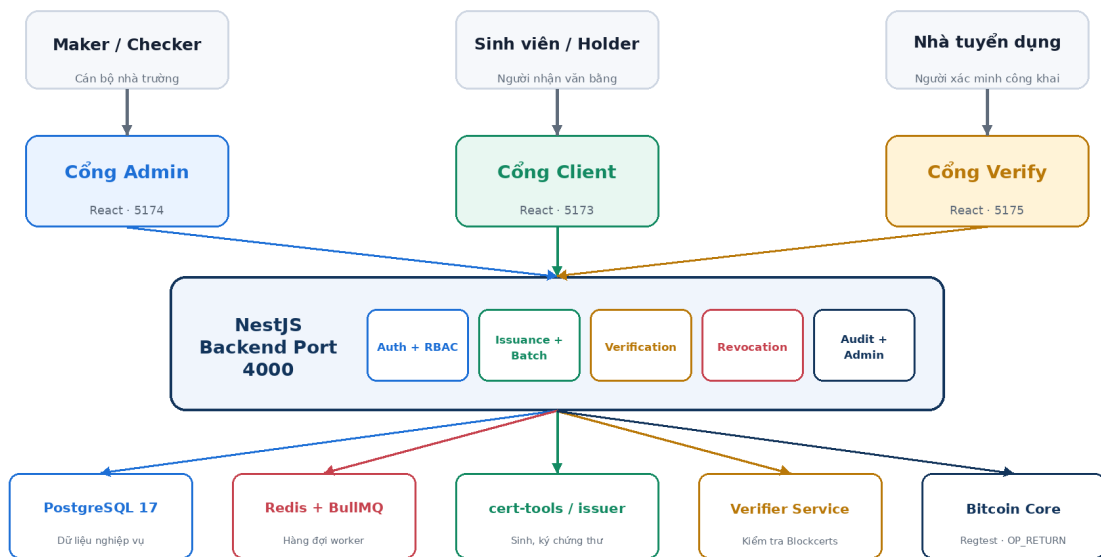
Bảng 3.1. Thành phần phần mềm

Thành phần	Phiên bản/dòng phiên bản	Vai trò
Node.js	22	Runtime backend/frontend build
NestJS	11	REST API, JWT, RBAC và nghiệp vụ
PostgreSQL	17-bookworm	Dữ liệu nghiệp vụ và log
Redis	7.4-bookworm	Nền lưu trữ BullMQ
BullMQ	5	Hàng đợi phát hành
Bitcoin Core	31.1	Số cái regtest và RPC
cert-tools	3.0.2	Sinh chứng thư unsigned từ roster
cert-issuer	3.13.1	Tạo Merkle proof và neo Bitcoin
React/Vite	19/7	Ba giao diện web
Nginx	Bản hệ thống	Reverse proxy và base path

Repository cung cấp Compose, migration, .env.example, script triển khai/health check và script thực nghiệm. PostgreSQL, Redis và Bitcoin RPC chỉ bind loopback. WIF, mật khẩu, JWT secret, RPC credential và tài khoản thử nghiệm nằm ngoài Git.

KIẾN TRÚC HỆ THỐNG HIỆN TẠI

Ba cổng web độc lập, backend mô-đun và hạ tầng Blockcerts



Hình 3.1. Kiến trúc triển khai

3.1.2. Tổ chức triển khai

PostgreSQL, Redis và Bitcoin Core chạy bằng Docker với restart policy. Backend, worker và Nginx được quản lý bằng systemd user service. Nginx phục vụ ba base path và chuyển tiếp API. Health check kiểm tra backend, DB, Redis và Bitcoin tùy mức ready. Kiến trúc này phù hợp pilot một máy; chưa phải kiến trúc HA nhiều node.

3.2. Hướng dẫn tái lập triển khai từ Ubuntu mới

Mục này là quy trình thao tác có thể lặp lại từ một máy Ubuntu mới. Lệnh và phần output minh họa được lấy từ lần chạy ngày 21/09/2026 trên máy triển khai nêu ở mục 3.1; ảnh giao diện được chụp từ chính hệ thống tại cổng 8088. Các giá trị bí mật, WIF, mật khẩu, JWT secret và RPC credential không được đưa vào báo cáo, được biểu diễn bằng [REDACTED].

3.2.1. Chuẩn bị máy Ubuntu và kiểm tra tài nguyên

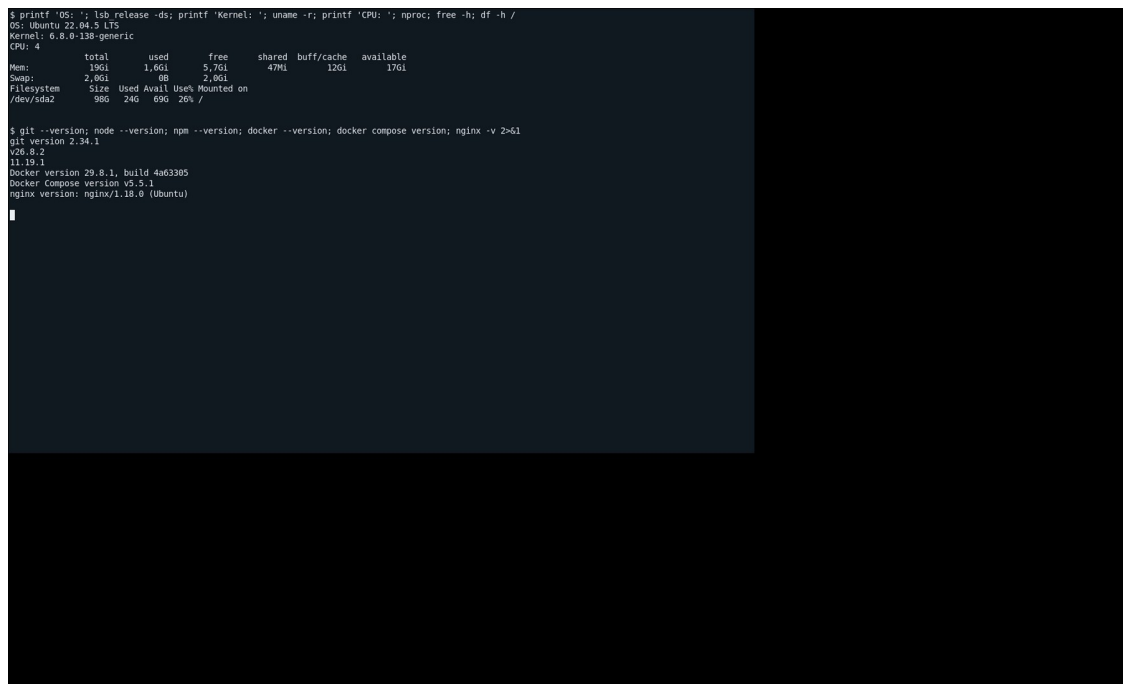
Bước 1 – cập nhật hệ thống và cài các công cụ nền. Nếu Docker đã được cài từ kho chính thức thì không cài đề; chỉ kiểm tra phiên bản trước khi tiếp tục.

```

sudo apt-get update
sudo apt-get install -y git curl ca-certificates nginx
uname -a
nproc
free -h
df -h /
docker --version
docker compose version
node --version
npm --version
nginx -v

```

Output quan trọng trên máy tái lập: Ubuntu 22.04.5 LTS, kernel 6.8.0-138-generic, 4 CPU; Docker 29.8.1; Docker Compose v5.5.1; Node.js v26.8.2; npm 11.11.0; Nginx 1.18.0. Nginx hệ thống tiếp tục giữ cổng 80, còn bản đồ án dùng cổng 8088.



```

$ printf 'OS: '; lsb_release -ds; printf 'Kernel: '; uname -r; printf 'CPU: '; nproc; free -h; df -h /
OS: Ubuntu 22.04.5 LTS
Kernel: 6.8.0-138-generic
CPU: 4
              total        used        free      shared  buff/cache   available
Mem:            19Gi       1.6Gi       5.7Gi         47Mi        12Gi        17Gi
Swap:           2.8Gi          0B        2.8Gi
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        98G   24G   69G   25% /

$ git --version; node --version; npm --version; docker --version; docker compose version; nginx -v 2>&1
git version 2.34.1
v26.8.2
11.11.0
Docker version 29.8.1, build 4963385
Docker Compose version v5.5.1
nginx version: nginx/1.18.0 (Ubuntu)

```

Hình 3.2. Kiểm tra hệ điều hành, tài nguyên và phiên bản công cụ trên máy mới

3.2.2. Lấy mã nguồn và cố định phiên bản

Bước 2 – clone repository, chuyển vào nhánh main và kiểm tra đúng commit. Việc cố định SHA giúp người đọc chạy đúng phiên bản đã được mô tả, thay vì một phiên bản mới hơn có thể thay đổi hành vi.

```

mkdir -p ~/DATN_work
cd ~/DATN_work
git clone https://github.com/ebkk24/DATN-AT190226.git datn-blockcerts
cd datn-blockcerts
git checkout main
git pull --ff-only origin main
git rev-parse HEAD
git status --branch --short
8d20eb263bccbf1cfff3acbb8f8e82c45d5c27588
## main...origin/main

```

Hai commit chức năng cần có là af9ae0c – “feat: harden student identity and issuance workflow” và 8d20eb2 – “security: prevent SSRF in certificate verifier”. Không dùng force checkout nếu thư mục đang có thay đổi chưa lưu.

3.2.3. Tạo cấu hình runtime và bảo vệ bí mật

Bước 3 – tạo tệp cấu hình từ mẫu, sinh các secret riêng cho máy và giới hạn quyền đọc. Không sao chép .env hoặc khóa riêng từ máy khác; không chụp nội dung các giá trị bí mật.

```
cp .env.example .env
chmod 600 .env
openssl rand -hex 32      # dùng làm JWT_SECRET
openssl rand -base64 32   # dùng cho mật khẩu dịch vụ
nano .env
```

Các biến tối thiểu phải được điền gồm POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, REDIS_PASSWORD, BITCOIN_RPC_USER, BITCOIN_RPC_PASSWORD, JWT_SECRET, PUBLIC_BASE_URL và CORS_ORIGINS. Giá trị nhạy cảm trong output được ghi [REDACTED]. Khóa phát hành đặt tại storage/credentials/pk_issuer.txt, quyền 600; thư mục này, .env, experiments/private và backend/.batch-work phải ở trạng thái ignored trước khi chạy.

```
git check-ignore -v .env storage/credentials/pk_issuer.txt
experiments/private/current-accounts.json backend/.batch-work
stat -c '%a %n' .env storage/credentials/pk_issuer.txt
# Kết quả yêu cầu: các đường dẫn đều được ignore; hai tệp bí mật có mode 600.
```

3.2.4. Khởi động hạ tầng và áp dụng migration

Bước 4 – khởi động PostgreSQL, Redis và Bitcoin Core regtest, chờ health check rồi áp dụng toàn bộ migration.

```
docker compose up -d postgres redis bitcoin-core
docker compose ps
cd backend
npm ci
npm run migration:run
npm run migration:show
```

Output thực tế cho thấy ba container datn-postgres, datn-redis và datn-bitcoin-core đều Up/healthy. migration:show đánh dấu [X] cho StudentCertificateIdentity, IssuanceIdempotency và ReconcileLegacyIssuanceBatches; không còn migration pending.

```
$ docker compose up -d postgres redis bitcoin-core && docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED    STATUS    PORTS
datn-bitcoin-core   bitcoind/bitcoin:31.1 /entrypoint.sh -reg-  bitcoin-core  14 hours ago    Up 14 hours (healthy)    8332-8333/tcp, 18332-18333/t
cp, 18444/tcp, 18332-18333/tcp, 127.0.0.1:18443->18443/tcp
datn-postgres       postgres:17-bookworm "docker-entrypoint.s..." postgres     14 hours ago    Up 14 hours (healthy)    127.0.0.1:5432->5432/tcp
datn-redis          redis:7.4-bookworm   "docker-entrypoint.s..." redis        14 hours ago    Up 14 hours (healthy)    127.0.0.1:6379->6379/tcp
Container datn-bitcoin-core Running
Container datn-redis Running
Container datn-postgres Running
[exit=0; duration=0.4s]

> backend@0.0.1 build
> nest build

o Injected env (22) from ../env // tip: M override existing ( override: true )
[X] 1 B12BaselineAndBatch1091909000000
[X] 2 StudentCertificateIdentity1789900000000
[X] 4 IssuanceIdempotency1790000000000
[X] 5 ReconcileLegacyIssuanceBatches1790000001000
```

Hình 3.3. Trạng thái hạ tầng và các migration đã áp dụng

3.2.5. Cài phụ thuộc, kiểm thử và build

Bước 5 – dùng npm ci tại từng dự án có package-lock.json để cài đúng dependency đã khóa; sau đó chạy unit test, kiểm thử SSRF và build các ứng dụng.

```
cd ~/DATN_work/datn-blockcerts/backend
npm ci
npm test -- --runInBand
npm run build

cd ../apps/verifier-service
npm ci
npm test

cd ../../frontend-admin && npm ci && npm run build
cd ../frontend-client && npm ci && npm run build
cd ../frontend-verify && npm ci && npm run build
```

Kết quả chạy lại: backend đạt 8/8 test suite và 21/21 test; verifier đạt 10/10 kiểm thử SSRF; backend và ba frontend build thành công. Một lỗi test hoặc build phải được xử lý trước khi tiếp tục, không bỏ qua bằng cờ cường bức.

```

> backend@0.0.1 test
> jest --runInBand

[Nest] 650392 - 09/21/2026, 8:41:08 AM   WARN [IssuanceProcessor] Bỏ qua duplicate job đang được xử lý: batch-1
PASS src/issuance/issuance.dispatcher.spec.ts
PASS src/issuance/issuance.processor.spec.ts
PASS src/issuance/issue.dto.spec.ts
PASS src/issuance/issuance.service.spec.ts
PASS src/verification/verification.service.spec.ts
PASS src/issuance/issuance.service.spec.ts
PASS src/app.controller.spec.ts
PASS src/auth/roles.guard.spec.ts

Test Suites: 8 passed, 8 total
Tests:       21 passed, 21 total
Snapshots:   0 total
Time:        6.962 s
Ran all test suites.

✓ chỉ cho phép HTTPS và không cho URL chứa credential (0.922895ms)
✓ chấp hostname nếu DNS trả về một địa chỉ private hoặc địa chỉ qua proxy (0.511457ms)
✓ chuẩn hóa và chặn IPv4 dạng số nguyên, hex và octal (0.529135ms)
✓ ghim kết nối vào IP công khai đã kiểm tra (38.953834ms)
✓ kiểm tra lại URL sau redirect và chặn redirect vào localhost (0.714417ms)
✓ giới hạn số redirect (0.789183ms)
✓ từ chối phản hồi vượt giới hạn kích thước (1.377637ms)
✓ từ chối redirect và trạng thái HTTP không hợp lệ mà không treo tiến trình (2.464162ms)
  1 tests 10
    1 suites 0
      1 pass 10
      1 fail 0
      1 cancelled 0
      1 skipped 0
      1 todo 0
      1 duration ms 167.04448

```

Hình 3.4. Kết quả unit test backend và kiểm thử SSRF của verifier

3.2.6. Khởi tạo Blockcerts regtest và chạy preflight

Bước 6 – tạo/kiểm tra watch-only wallet regtest và chạy preflight của cert-issuer. Script chỉ dùng dữ liệu mẫu, xác nhận image, cấu hình issuer, giao dịch neo và proof trước khi triển khai ứng dụng.

```

cd ~/DATN_work/datn-blockcerts
./scripts/setup-regtest-watch-wallet.sh
./scripts/preflight-cert-issuer.sh

```

Output thực tế kết thúc với PRECHECKS PASSED; chứng thư mẫu được cấp, receipt chứa đúng chain=bitcoinRegtest và Merkle root khớp OP_RETURN.txid trong ảnh là giao dịch regtest, không mang giá trị tài chính.

```

$ ./scripts/preflight-cert-issuer.sh
[1/7] Kiểm tra Compose và dịch vụ
NAME          IMAGE          COMMAND          SERVICE          CREATED          STATUS          PORTS
datn-bitcoin-core  bitcoin/bitcoin:31.1  "entrypoint.sh -reg-"  bitcoin-core      14 hours ago     Up 14 hours (healthy)  8332-8333/tcp, 18332-18333/t
cp, 18444/tcp, 38332-38333/tcp, 127.0.0.1:18443->18443/tcp
datn-postgres    postgres:17-bookworm  "docker-entrypoint.s-"  postgres          14 hours ago     Up 14 hours (healthy)  127.0.0.1:5432->5432/tcp, 127.0.0.1:6379->6379/tcp
datn-redis       redis:7.4-bookworm    "docker-entrypoint.s-"  redis             14 hours ago     Up 14 hours (healthy)
[2/7] Kiểm tra quyền khóa
[3/7] Kiểm tra phiên bản và chứng thư chưa neo
cert-tools 3.0.2 cert-schema 3.10.0 unsigned certificates/fdfb8618-abad-4c51-9b3e-6f5477c889d6.json
[4/7] Kiểm tra khóa khớp địa chỉ phát hành
Khóa và địa chỉ khớp nhau
[5/7] Kiểm tra UTXO qua đường RPC của cert-issuer
UTXO: 123 x xác nhận: 1
[6/7] Kiểm tra pipeline Merkle/proof bằng mockchain, không broadcast
Mock proof hợp lệ về pipeline
[7/7] Kiểm tra tạo và ký giao dịch Bitcoin thật nhưng chặn broadcast
Ký giao dịch và kiểm tra OP_RETURN thành công; không broadcast.
TIỀN KIỂM TRA THÀNH CÔNG – chưa có giao dịch nào được broadcast.
Container datn-postgres Running
Container datn-redis Running

```

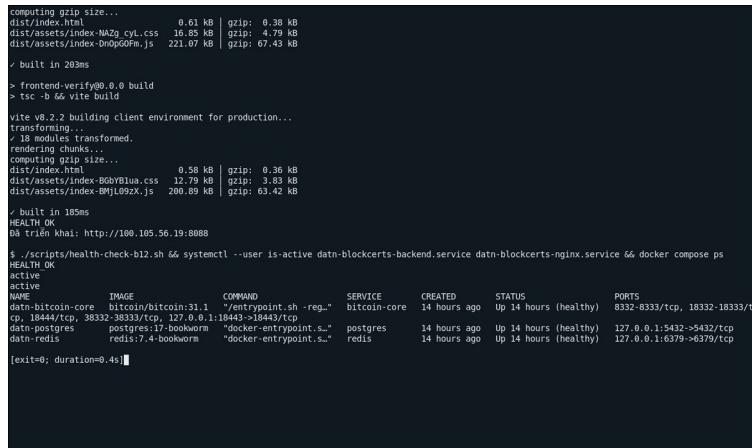
Hình 3.5. Preflight Blockcerts và giao dịch neo trên Bitcoin regtest

3.2.7. Triển khai backend, frontend và kiểm tra dịch vụ

Bước 7 – chạy script triển khai. Script build/copy ba frontend vào /srv/datn-blockcerts, cài unit systemd user cho backend/Nginx và khởi động dịch vụ ở cổng 8088.

```
cd ~/DATN_work/datn-blockcerts
./scripts/deploy-b12.sh
./scripts/health-check-b12.sh
systemctl --user is-active datn-blockcerts-backend.service datn-blockcerts-nginx.service
curl -fsS http://127.0.0.1:8088/health
```

Output thực tế: DEPLOY_OK, HEALTH_OK, hai unit đều active; API trả status=ok, database=up, redis=up và bitcoin=up. Có thể truy cập /admin/, /student/ và /verify/ qua địa chỉ LAN hoặc Tailscale của máy. Cổng 80 của Nginx hệ thống không bị dừng hoặc thay đổi.



```
computing gzip size...
dist/index.html      0.61 kB | gzip: 0.38 kB
dist/assets/index-NAZg.cyl.css 16.85 kB | gzip: 4.79 kB
dist/assets/index-DnQpG0Rn.js 221.07 kB | gzip: 67.43 kB

/ built in 203ms

> frontend-verify00.0.0 build
> tsc -b 66 vite build
vite v8.2.2 building client environment for production...
transforming...
18 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html      0.58 kB | gzip: 0.36 kB
dist/assets/index-B0bYBlua.css 12.79 kB | gzip: 3.83 kB
dist/assets/index-BMJL092x.js 200.89 kB | gzip: 63.42 kB

/ built in 185ms
HEALTH OK
Bà triển khai: http://180.105.56.19:8088

$ ./scripts/health-check-b12.sh && systemctl --user is-active datn-blockcerts-backend.service datn-blockcerts-nginx.service && docker compose ps
HEALTH OK
active
active
NAME                IMAGE                COMMAND                SERVICE    CREATED    STATUS    PORTS
datn-bitcoin-core   bitcoin/bitcoin:31.1 "entrypoint.sh -reg-" bitcoin-core 14 hours ago Up 14 hours (healthy)
tcp, 18444/tcp, 38332-38333/tcp, 127.0.0.1:18443->18443/tcp
datn-postgres       postgres:17-bookworm "docker-entrypoint.s..." postgres    14 hours ago Up 14 hours (healthy)
127.0.0.1:5432->5432/tcp
datn-redis          redis:7.4-bookworm  "docker-entrypoint.s..." redis       14 hours ago Up 14 hours (healthy)
127.0.0.1:6379->6379/tcp

[exit=0; duration=0.4s]
```

Hình 3.6. Triển khai hoàn tất và health check toàn hệ thống

3.2.8. Cơ chế khôi phục Worker cần kiểm tra sau triển khai

Sau khi Checker duyệt, backend không chỉ enqueue một job. Transaction PostgreSQL ghi đồng thời trạng thái yêu cầu, issuance_batches và issuance_outbox. Dispatcher đọc outbox và tạo BullMQ job với jobId ổn định. Worker nhận lock/lease, sử dụng thư mục backend/.batch-work/<batchId>, lưu transaction intent trước broadcast và chỉ đánh dấu hoàn tất sau khi đối chiếu txid, OP_RETURN cùng Merkle root.

Quy tắc an toàn khi retry là: nếu batch đã có anchorTxid hoặc checkpoint chứa raw transaction/txid dự kiến thì không tạo giao dịch mới. Worker tra cứu giao dịch trong wallet, sau đó fallback sang raw transaction; nếu tìm thấy và dữ liệu neo khớp thì tiếp tục hoàn tất DB. Nếu trạng thái RPC chưa rõ, job dừng để reconciliation thay vì broadcast mù quáng. PostgreSQL và Bitcoin không được mô tả như một giao dịch ACID chung vì giao dịch Bitcoin đã phát không thể rollback.

```
cd ~/DATN_work/datn-blockcerts/backend
npm test -- --runInBand issuance.processor.spec.ts issuance.dispatcher.spec.ts
# Kết quả của toàn bộ test backend tại lần xác minh: 8 suites, 21 tests đều PASS.
```

3.3. Hiện thực backend và cơ sở dữ liệu

3.3.1. Các module

Backend gồm AuthModule, mô-đun quản trị tài khoản, IssuanceModule, VerificationModule, RevocationModule, AuditModule, IssuerModule và health controller. Controller gắn AuthGuard rồi RolesGuard tại route cần bảo vệ; service thực thi nghiệp vụ; entity mô tả dữ liệu; processor nhận job BullMQ. NestJS được chọn vì hỗ trợ module, dependency injection, guard và validation rõ ràng [19].

AuthModule đăng nhập, so khớp bcrypt và ký JWT. mô-đun quản trị tài khoản cho Checker list/tạo tài khoản nhưng không trả passwordHash. IssuanceModule quản lý phiếu và worker. VerificationModule điều phối verifier service và lưu log. RevocationModule tạo transaction thu hồi. IssuerModule công bố profile cùng revocation list.

3.3.2. Migration và transaction

TypeORM chạy synchronize=false; schema thay đổi qua migration [22]. Bốn bảng chính là users, issued_certificates, verification_logs và audit_logs. PostgreSQL transaction bảo đảm nhóm thay đổi DB được commit hoặc rollback như một đơn vị [20]. txid không có unique constraint vì nhiều chứng thư trong một batch dùng chung transaction.

Bảng `issued_certificates` lưu yêu cầu và kết quả, liên kết người học bằng `studentId/studentCode`. Hai bảng `issuance_batches` và `issuance_outbox` lưu `state`, `machine`, `transaction intent`, số lần thử và ý định `enqueue` bền vững. `batchId` là `idempotency key`; Worker cập nhật chứng thư theo chunk để tránh một câu lệnh quá lớn.

3.3.3. Audit và verification log

Audit được gắn vào đăng nhập, đăng ký, lập phiếu, duyệt, từ chối, phát hành, thu hồi và xác minh. Nếu riêng thao tác ghi audit lỗi, service không làm thất bại nghiệp vụ chính; lựa chọn này ưu tiên tính sẵn sàng nhưng cần cơ chế cảnh báo để không bỏ sót log âm thầm. Verification log lưu kết quả chi tiết từ verifier, adapter Bitcoin, confirmation và trạng thái cuối.

3.4. Hiện thực quy trình phát hành

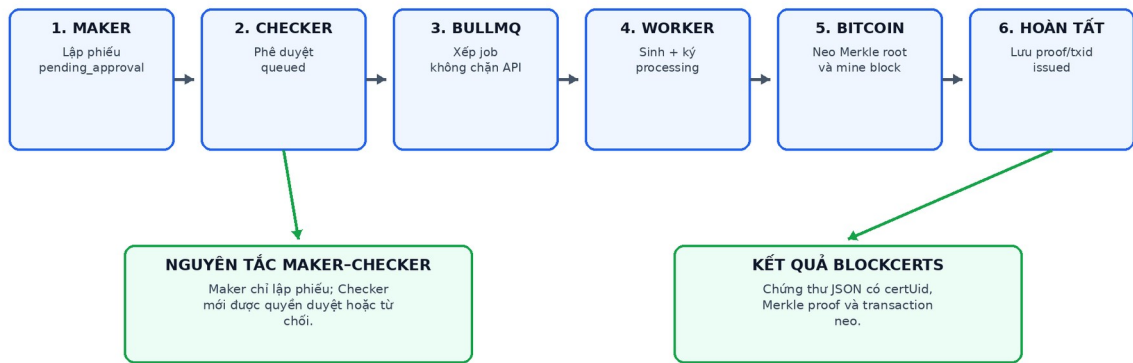
3.4.1. Phát hành đơn và Maker–Checker

Phiếu phát hành mới tách hồ sơ Student nguồn khỏi dữ liệu riêng của văn bằng. Maker chỉ tra cứu hồ sơ theo `studentCode`; backend gắn chủ sở hữu bằng `studentId` và chụp snapshot họ tên, ngày sinh, email, niên khóa cùng dữ liệu văn bằng. Các trường nghiệp vụ gồm tên văn bằng, ngành/chuyên ngành, trình độ, xếp loại, năm tốt nghiệp, ngày cấp, số hiệu duy nhất toàn hệ thống, hình thức đào tạo và địa chỉ nhận.

Migration mới bổ sung cột nullable để chứng thư lịch sử vẫn đọc và xác minh được. Với phiếu mới, DTO và backend kiểm tra độ dài, ngày ISO hợp lệ, miền năm, địa chỉ P2PKH regtest và các danh mục cố định. Checker, Records, Holder, Verify và `credentialSubject` cùng sử dụng snapshot này; việc chỉnh hồ sơ Student về sau không làm thay đổi chứng thư đã cấp.

Maker gọi `/api/issue/request`. Backend lấy username từ JWT, không tin `requestedBy` trong body, kiểm tra `studentCode` rồi lưu `pending_approval`. Checker gọi `approve/reject`. Khi duyệt, transaction PostgreSQL đồng thời chuyển yêu cầu sang `queued`, tạo/cập nhật batch và ghi outbox. Dispatcher dùng `jobId` ổn định để enqueue BullMQ [21]. Worker nhận lease, chuyển state, tạo roster, gọi toolchain và cập nhật `issued` hoặc `failed`; retry bị giới hạn và tiếp tục từ checkpoint thay vì phát lại mù quáng.

LUỒNG PHÁT HÀNH MỘT VĂN BẰNG



Hình 3.7. Luồng phát hành một văn bằng

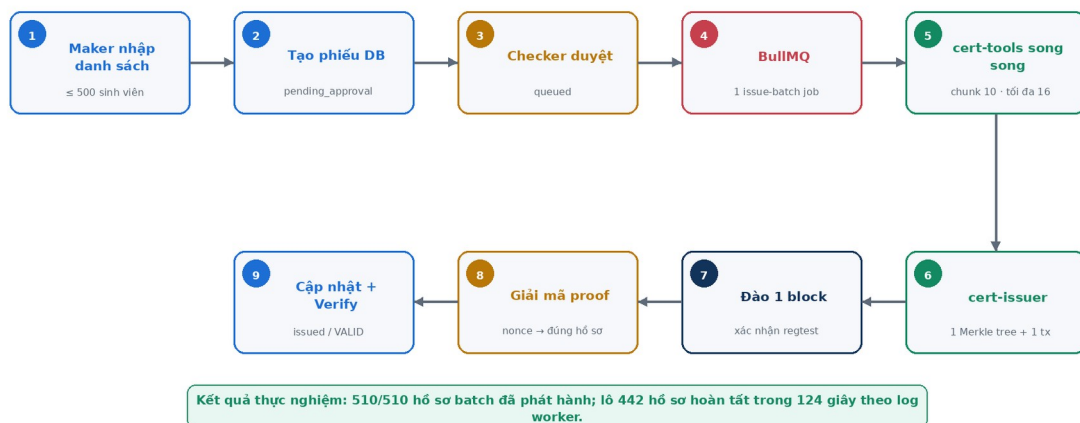
3.4.2. Tối ưu phát hành theo lô

Phương án tuần tự ban đầu lập cert-tools → cert-issuer → mine cho từng hồ sơ, làm tăng chi phí khởi động container và số transaction. Một thử nghiệm roster 462 dòng chạy quá 5 phút mà chưa có output nên được hủy có kiểm soát, queue pause và bản ghi reset. Kết quả này không được dùng trong benchmark chính thức nhưng giúp xác định nút thắt.

Thiết kế cuối chia roster thành chunk 10 hồ sơ; tối đa 16 chunk tạo unsigned certificate song song, mỗi chunk dùng workspace riêng. Sau khi hoàn tất, worker gom output, đặt batch_size bằng số hồ sơ và gọi cert-issuer một lần. Tool tạo một Merkle tree và một transaction cho toàn batch. Mỗi output mang nonce để ánh xạ đúng bản ghi, không phụ thuộc tên hoặc thứ tự file.

LUỒNG CẤP VĂN BẰNG THEO LÔ

Tối đa 500 hồ sơ — chia nhỏ bước sinh JSON, gom một Merkle anchor



Hình 3.8. Luồng phát hành theo lô

Workspace xác định theo batchId có roster, unsigned certificate, blockchain certificate, issuer work, manifest và checkpoint/anchor-intent.json. Trước broadcast, helper lưu raw transaction cùng txid dự kiến; sau khi RPC trả về, checkpoint ghi nhận broadcastReturned. Khi khởi động lại, Worker tra wallet/raw transaction, kiểm tra OP_RETURN chứa đúng Merkle root rồi hoàn tất DB. Dữ liệu runtime bị loại khỏi Git. Mỗi job tối đa 500 hồ sơ và mục tiêu là một transaction cho một batch.

3.4.3. Cấu trúc chứng thư đầu ra

Chứng thư thực tế dùng context VC v1, Data Integrity v2 và Blockcerts v3.1; type gồm VerifiableCredential và BlockcertsCredential. Proof có type=DataIntegrityProof, cryptosuite=merkle-proof-2019, proofPurpose=assertionMethod, thời gian, verification method và proofValue. Cấu trúc được đối chiếu với VC, JSON-LD và Blockcerts [9], [11]–[14].

Nhiều chứng thư cùng txid là hành vi đúng của Merkle batching. Mỗi chứng thư có certificate ID, certUid, target hash và đường dẫn Merkle riêng; root chung mới được ghi trong OP_RETURN. SHA-256 là hàm băm nền theo FIPS 180-4 [4].

3.5. Hiện thực xác minh và thu hồi

3.5.1. Xác minh công khai

Verify portal gửi JSON đến POST /api/verify. Backend từ chối semantic không phù hợp, chuyển dữ liệu tới verifier service, nhận kết quả kiểm tra credential và adapter regtest. Adapter giải mã proofValue, lấy anchor, gọi RPC để đọc transaction, kiểm tra địa chỉ phát hành, OP_RETURN, Merkle root và ít nhất một confirmation. Sau đó backend tra thu hồi và lưu verification log/audit.

Kết quả VALID chỉ xuất hiện khi bằng chứng chứng thư và anchor đạt, đồng thời không bị thu hồi. INVALID phản ánh sai lệch được xác định; REVOKED phản ánh chứng thư hợp lệ nhưng mất hiệu lực; INDETERMINATE phản ánh không đủ dữ kiện để kết luận. Phân loại này quan trọng vì lỗi proof/tx không truy cập được không nên tự động bị diễn giải giống sửa payload.

3.5.2. Thu hồi

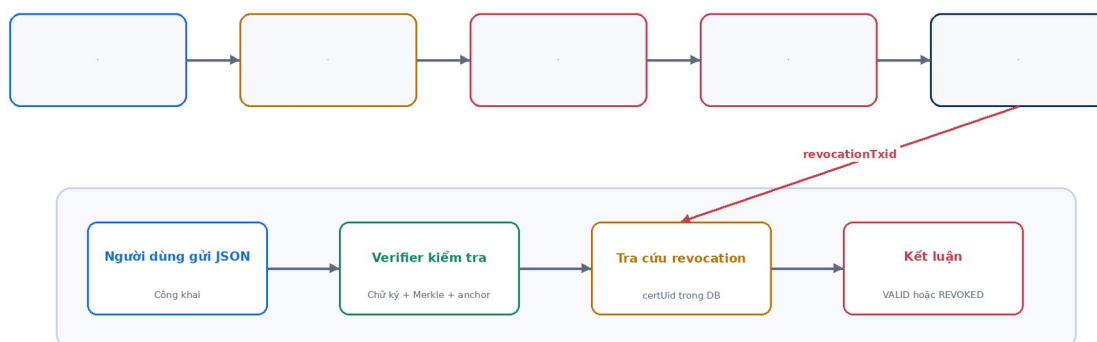
Chỉ Checker gọi được POST /api/revoke/:id. Service tạo REVOKE:<certUid>, mã hóa hex, tạo raw transaction với OP_RETURN, ký bằng

signrawtransactionwithkey, broadcast và tạo một block. Việc ký trực tiếp bằng khóa được dùng vì ví regtest là watch-only. Khóa không nằm trong mã nguồn.

Sau khi giao dịch được xác nhận, PostgreSQL cập nhật revoked, transaction, actor, lý do và thời gian. Revocation List đọc các bản ghi này. Verify trước hết kiểm tra credential/anchor, sau đó trả REVOKED nếu DB ghi nhận thu hồi. Hình 3.9 mô tả luồng hai lớp.

LUỒNG THU HỒI VÀ XÁC MINH

Bảng chứng OP_RETURN trên regtest kết hợp trạng thái nghiệp vụ trong PostgreSQL



Hình 3.9. Luồng thu hồi và xác minh

3.6. Hiện thực ba frontend

Cổng quản trị hiển thị dashboard, lập phiếu đơn/lô, duyệt nhiều phiếu, danh sách văn bằng, thu hồi, người dùng và audit. Menu được lọc theo Maker/Checker. Cổng Holder dùng tài khoản Student; backend lấy studentId/studentCode từ JWT để truy vấn đúng chứng thư, không ánh xạ bằng tên hiển thị. Holder cho xem/tải JSON và văn hiển thị bản ghi revoked. Cổng Verify không yêu cầu đăng nhập.

Một lỗi route đã được sửa bằng cách đặt holder/certificates, batch/request, batch/approve và revoke/check trước route tham số :id. Một lỗi khác xảy ra khi RolesGuard toàn cục chạy trước JWT strategy; giải pháp là gắn AuthGuard('jwt') và RolesGuard cục bộ đúng thứ tự. Các lỗi này được giữ trong nhật ký phát triển để giải thích quyết định thiết kế.

3.7. Gia cố an toàn, định danh và độ bền Worker

Mật khẩu được băm bằng bcrypt với cost 10; đối với hệ thống mới, Argon2id vẫn là lựa chọn ưu tiên [17]. JWT tuân theo RFC 7519 [15]. RBAC từ chối Student ở cổng quản trị, actor lấy từ JWT; Helmet, throttling, CORS allowlist, giới hạn body và Nginx tạo lớp bảo vệ biên. Verifier dùng safe-fetch: chỉ chấp nhận HTTPS/hostname cho phép, phân giải và kiểm tra IP trước kết nối, kiểm tra

lại sau mỗi redirect, chặn loopback/private/link-local/metadata IPv4 và IPv6, đặt timeout và giới hạn kích thước phản hồi [23]. Mười kiểm thử SSRF và các kiểm tra trực tiếp trong container đều đạt; kết quả này chứng minh các trường hợp đã kiểm thử, không thay thế rà soát cấu hình khi triển khai thật.

Bản freeze R1 bị vô hiệu vì Nginx không có quyền ghi thư mục tạm cho request body lớn. Sau khi sửa quyền, preflight được chạy lại và đóng băng R2. Việc loại R1 khỏi thống kê nhưng lưu riêng bằng chứng lỗi giúp tránh lựa chọn dữ liệu có lợi và tăng khả năng truy vết.

3.8. Phương pháp kiểm thử

3.8.1. Mục tiêu và nguyên tắc

Kiểm thử trả lời ba câu hỏi: chứng thư bị sửa có bị phát hiện và Maker có bị chặn tự duyệt hay không; Merkle batching có phát hành đủ 10, 100 và 500 chứng thư với một transaction/lô hay không; thu hồi có đúng quyền, được xác nhận và phản ánh nhất quán ở Verify, Revocation List cùng audit hay không.

Bộ dữ liệu là dữ liệu giả lập, không dùng hồ sơ cá nhân thật. Mỗi kích thước hiệu năng lặp ba lần. Thời gian đo từ lúc Checker duyệt đến khi toàn bộ bản ghi issued. Các tệp bằng chứng gồm JSON thô, CSV, manifest, biểu đồ, trạng thái sau thử nghiệm và checksum SHA-256. Lần chạy cấu hình sai được tách khỏi tập chính thức.

Thời gian mỗi chứng thư và throughput được tính:

$$T_{\text{cert}} = T_{\text{batch}} / N$$

$$\text{Throughput} = N / T_{\text{batch}}$$

Trong đó T_{batch} là thời gian từ duyệt đến phát hành hoàn tất, N là kích thước batch. Độ lệch chuẩn báo cáo là độ lệch chuẩn mẫu của ba lần lặp.

3.8.2. Kiểm thử tự động và chất lượng mã

Backend có 5 test suite với 8/8 test đạt: controller cơ bản; ba trường hợp RolesGuard; DTO phát hành; Issuer Profile/Revocation List; và semantic credentialSubject.id. Cả ba frontend build và lint thành công. npm audit của backend và ba frontend không phát hiện lỗi hồng tại thời điểm freeze. Kết quả audit dependency không đồng nghĩa hệ thống không có lỗi hồng logic hoặc cấu hình.

3.9. Kịch bản 1 – Quy trình cấp văn bằng đầu cuối

Mục tiêu: kiểm tra xuyên suốt việc Maker tra cứu hồ sơ Student, nhập dữ liệu nghiệp vụ, Checker duyệt hoặc từ chối, Worker phát hành, Student nhận chứng thư và đối chiếu JSON Blockcerts đầu ra.

Maker đăng nhập, tra cứu Student theo studentCode và chỉ được đọc mã sinh viên, họ tên, ngày sinh, email, niên khóa từ hồ sơ nguồn.

Maker nhập tên văn bằng, ngành/chuyên ngành, trình độ, xếp loại, năm tốt nghiệp, ngày cấp, số hiệu văn bằng, hình thức đào tạo và địa chỉ nhận. Backend từ chối thiếu trường, sai ngày, giá trị ngoài danh mục và số hiệu trùng toàn hệ thống.

Luồng đơn và luồng lô cùng dùng một mô hình dữ liệu; lô giới hạn 500 dòng, kiểm tra từng phần tử và chỉ ghi khi toàn bộ lô hợp lệ.

Checker nhìn thấy toàn bộ snapshot trước khi duyệt/từ chối; Worker tạo credential, cây Merkle và neo Merkle root bằng OP_RETURN trên Bitcoin regtest.

Student chỉ đọc chứng thư gắn với studentId của tài khoản; dữ liệu nghiệp vụ từ phiếu được đối chiếu với credentialSubject trong JSON đầu ra.

Kết quả thực tế: unit test backend đạt 25/25; E2E mới hoàn tất Maker–Checker–Worker–Holder–Verify, chứng thư chuyển sang VALID trước khi thu hồi. Snapshot hồ sơ và dữ liệu văn bằng không thay đổi khi đọc lại từ Records, Holder và Verify.

KMA

Blockcerts

Cổng quản trị

KHÔNG GIAN LÀM VIỆC

Tổng quan

Lập phiếu

Lập theo lớp

Nhật ký

b12_maker_2026092012...

Maker

Đăng xuất

MAKER

Lập phiếu cấp văn bằng

Trà cứu hồ sơ Student nguồn, sau đó nhập dữ liệu riêng của văn bằng.

Chưa ghi blockchain

Mã sinh viên

B12-20260920123633

Tra cứu hồ sơ

HỒ SƠ STUDENT – CHỈ ĐỌC

Mã sinh viên

B12-20260920123633

Họ và tên

Thực nghiệm B12 Holder 20260920123633

Ngày sinh

2004-01-15

Email

b12.student@example.edu.vn

Niên khóa

2022 - 2027

Tên văn bằng

Bằng tốt nghiệp đại học

Ngành/chuyên ngành

An Toàn Thông Tin

Trình độ đào tạo

Đại học

Xếp loại tốt nghiệp

Giỏi

Năm tốt nghiệp

2027

Ngày cấp

06/30/2027

Số hiệu văn bằng

KMA-UI-1789975876569

Hình thức đào tạo

Chính quy

Địa chỉ nhận / public key

n4mRaTGzVkl9YfccFzpEqThrk2u2DoA9Pj

Lập phiếu

Hình 3.10. Maker đối chiếu hồ sơ Student chỉ đọc và nhập đầy đủ dữ liệu văn bằng

KMA

Blockcerts

Cổng quản trị

KHÔNG GIAN LÀM VIỆC

Tổng quan

Lập phiếu

Lập theo lớp

Nhật ký

b12_maker_2026092012...

Maker

Đăng xuất

MAKER

Lập phiếu cấp văn bằng

Trà cứu hồ sơ Student nguồn, sau đó nhập dữ liệu riêng của văn bằng.

Chưa ghi blockchain

Mã sinh viên

Vi dụ: AT190226

Trà cứu hồ sơ

Tên văn bằng

Bằng tốt nghiệp đại học

Ngành/chuyên ngành

An Toàn Thông Tin

Trình độ đào tạo

Đại học

Xếp loại tốt nghiệp

Giỏi

Năm tốt nghiệp

2026

Ngày cấp

09/21/2026

Số hiệu văn bằng

KMA-2027-0001

Hình thức đào tạo

Chính quy

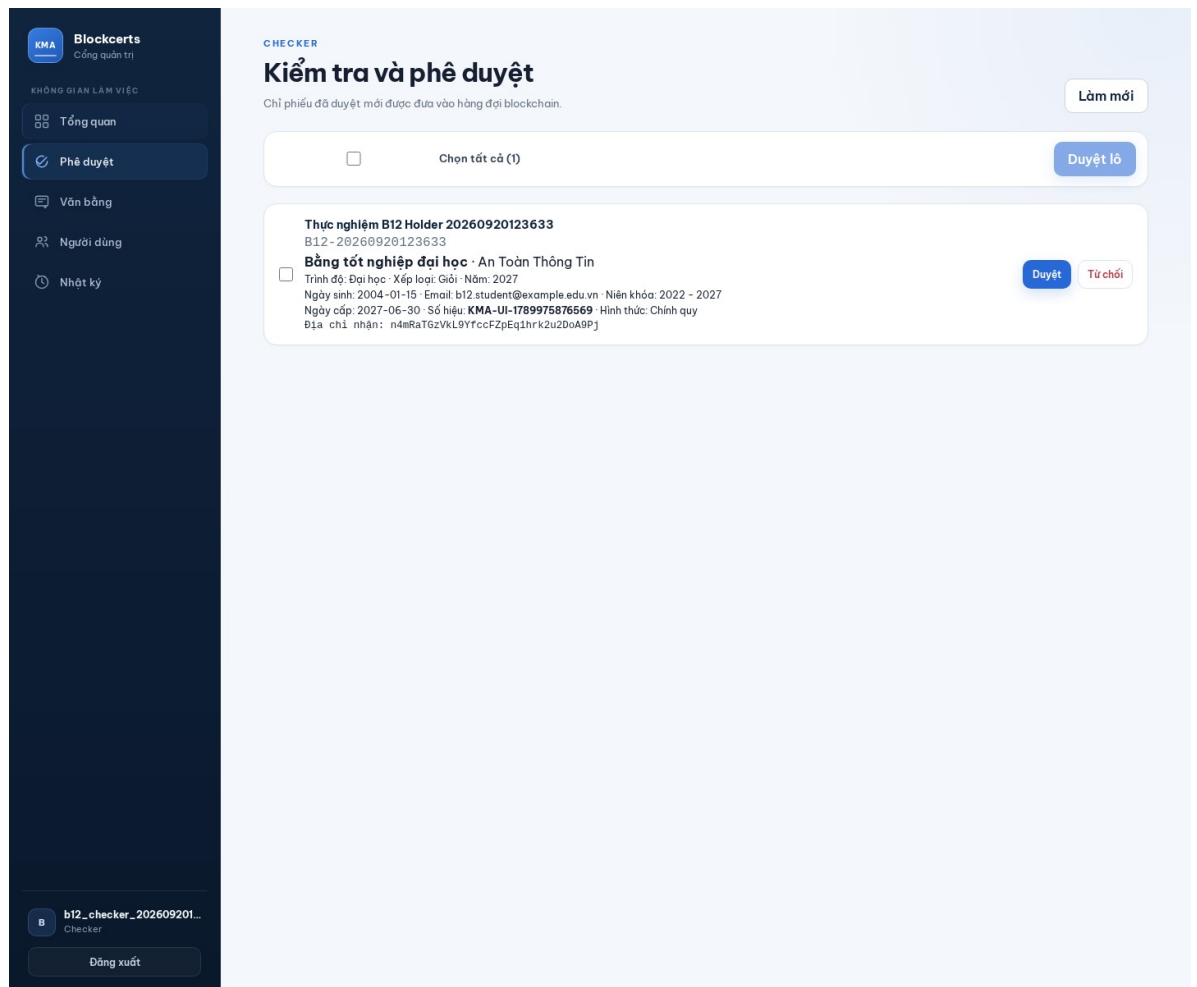
Địa chỉ nhận / public key

Địa chỉ Bitcoin P2PKH regtest

Lập phiếu

Đã lập phiếu dId30e47 – đang chờ Checker duyệt.

Hình 3.11. Phiếu chứa dữ liệu nghiệp vụ được tạo thành công và chờ duyệt



Hình 3.12. Checker xem đầy đủ snapshot hồ sơ và thông tin văn bản

KMA

Blockcerts

Cổng quản trị

KHÔNG GIAN LÀM VIỆC

Tổng quan

Phê duyệt

Văn bằng

Người dùng

Nhật ký

b12_checker_202609201...

Checker

Đăng xuất

VĂN BẰNG

Danh sách phát hành

Theo dõi trạng thái, giao dịch neo và lịch sử thu hồi.

1/25

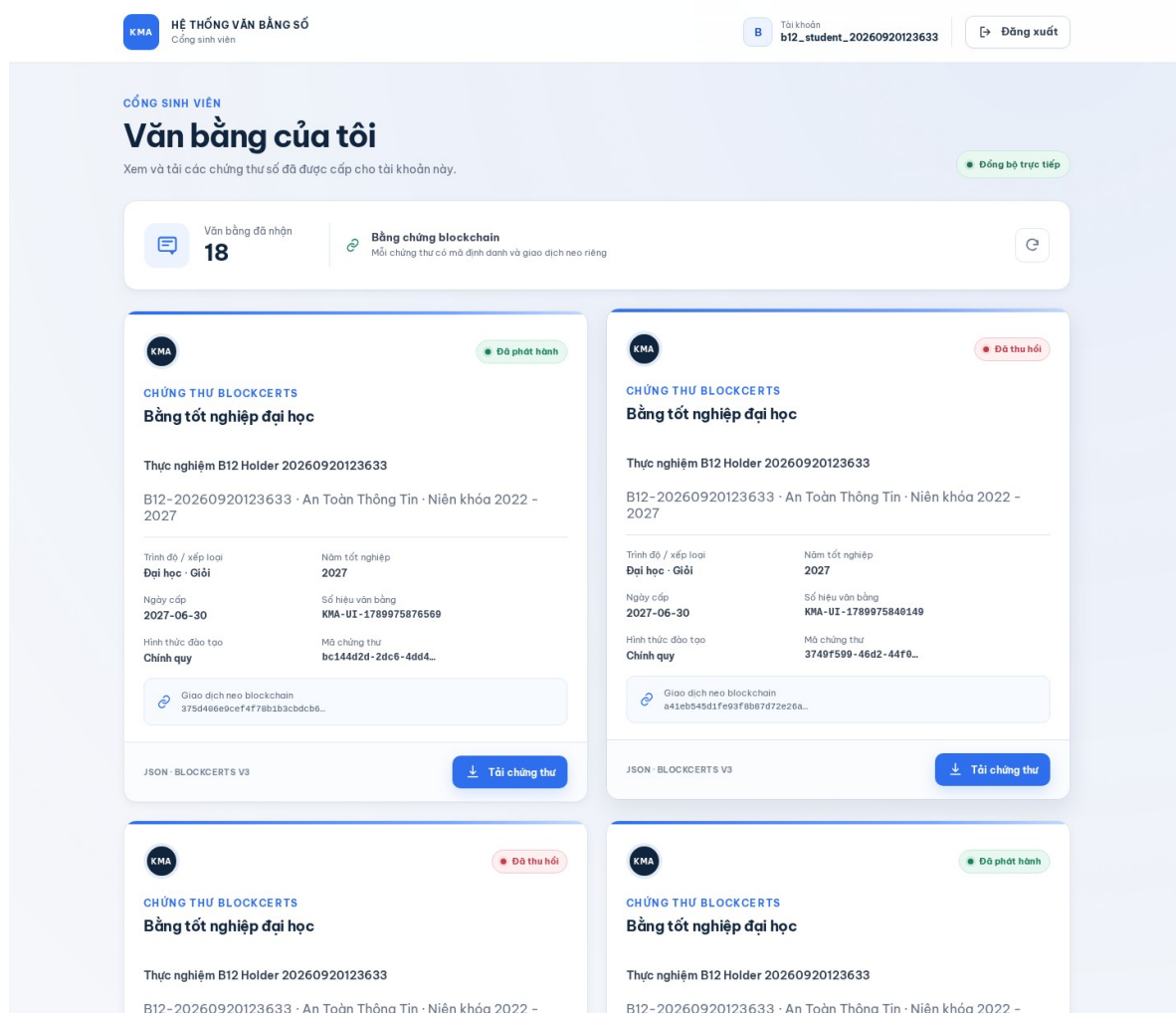
KMA-UI-1789975876569

Tất cả trạng thái

Làm mới

NGƯỜI NHẬN	VĂN BẰNG	TRẠNG THÁI	NGƯỜI DUYỆT	TXID PHÁT HÀNH	THỜI GIAN	THAO TÁC
Thực nghiệm B12 Holder 20260920123633 B12-28260920123633 2022 - 2027	Bằng tốt nghiệp đại học An Toàn Thông Tin - Giải KMA-UI- 1789975876569	Đã cấp	b12_checker_20260920123633	375d406e9cef...	07:31:18 21/9/2026	Thu hồi

Hình 3.13. Records hiển thị văn bằng đã được Worker phát hành



Hình 3.14. Holder xem thông tin văn bằng thuộc đúng tài khoản Student

3.10. Kịch bản 2 – Xác minh, toàn vẹn và kiểm soát an toàn

Mục tiêu: xác minh tính hợp lệ, khả năng phát hiện sửa đổi và thu hồi, đồng thời kiểm tra RBAC, cách ly Student, audit trail, verification log, chống SSRF và yêu cầu không công bố bí mật hoặc PII lên blockchain.

Credential gốc được verifier trả VALID; sửa tên, tên văn bằng, số hiệu, proofValue hoặc transaction id phải cho INVALID hoặc INDETERMINATE, không được nhận nhầm là hợp lệ.

Sau khi Checker thu hồi, verifier trả REVOKED; Maker không tự duyệt hoặc thu hồi, Student không gọi được API quản trị, hai Student trùng họ tên nhưng khác mã không đọc chứng thư của nhau.

Verifier chặn localhost, private IP, link-local/metadata, service nội bộ, HTTP không mã hóa, redirect và DNS không an toàn. Bộ safe-fetch đạt 10/10 ca.

Audit ghi request, approve, issue, revoke; blockchain chỉ chứa Merkle root và marker thu hồi trong OP_RETURN, không chứa hồ sơ Student, credential đầy đủ, secret hay khóa riêng.

Kết quả E2E thực tế: VALID → phát hiện bản sửa → REVOKED; mã phản hồi RBAC cho hành vi sai quyền đều là 403; phép thử hai Student trùng họ tên xác nhận cách ly theo studentId. Bảng kết quả cũ và toàn bộ ca kiểm thử toàn vẹn được giữ nguyên bên dưới.

KMAHỆ THỐNG VĂN BẰNG SỐXác minh công khai

Không cần đồng nhập

XÁC MINH CÔNG KHAI

Kiểm tra văn bằng số

Đối chiếu tính toàn vẹn, trạng thái phát hành và bằng chứng blockchain của chứng thư Blockcerts V3.

CÁCH THỰC HIỆN

Ba bước xác minh

1Chon chứng thư

Tải lên tệp JSON do người nhận cung cấp.

2Kiểm tra nội dung

Có thể xem lại hoặc dán trực tiếp JSON vào biểu mẫu.

3Nhận kết quả

Hệ thống đối chiếu Merkle proof và trạng thái thu hồi.

Xác minh độc lập

Không cần tài khoản và không làm thay đổi nội dung chứng thư.

CHỨNG THƯ BLOCKCERTS

Nội dung cần xác minh

bc144d2d-2dc6-4dd4-ba59-03b56256ba9c

Tệp đã sẵn sàng để xác minh

Chọn tệp

hoặc dán nội dung

Nội dung JSON

```
{  "@context": [    "https://www.w3.org/2018/credentials/v1",    {      "alumniOf": "https://schema.org/alumniOf",      "studentCode": "https://schema.org/identifier",      "dateOfBirth": "https://schema.org/birthDate",      "email": "https://schema.org/email",      "cohort": "https://actvn.edu.vn/credentials/terms#cohort",      "degreeName": "https://schema.org/educationalCredentialAwarded",      "major": "https://actvn.edu.vn/credentials/terms#major",      "educationLevel": "https://schema.org/educationalLevel"    }  ]}
```

2.781 ký tựĐã liệu được gửi qua kết nối hiện tại

Xác minh văn bằng

KẾT QUẢ XÁC MINH

Văn bằng hợp lệ

Chữ ký, Merkle proof và giao dịch neo blockchain đã được xác thực.

Người nhận

Thực nghiệm B12 Holder 20260920123633

Mã sinh viên

B12-20260920123633

Ngày sinh

2004-01-15

Email

b12.student@example.edu.vn

Niên khóa

2022 - 2027

Tên văn bằng

Bằng tốt nghiệp đại học

Ngành

An Toàn Thông Tin

Trình độ / xếp loại

Đại học - Giỏi

Năm tốt nghiệp

2027

Số hiệu văn bằng

KMA-UI-1789975876569

Hình thức đào tạo

Chính quy

Ngày cấp

2027-06-30

Mô chứng thư

bc144d2d-2dc6-4dd4-ba59-03b56256ba9c

Chi tiết kỹ thuật

Toàn vẹn chứng thư

Kiểm tra nội dung JSON

Merkle proof

Đối chiếu bằng chứng

Trạng thái hiệu lực

Kiểm tra dữ liệu thu hồi

© 2026 Học viện Kỹ thuật Mật mã

Blockcerts V3 - Bitcoin blockchain

Hình 3.15. Verify xác nhận chứng thư hợp lệ và hiển thị dữ liệu nghiệp vụ

KMA

HỆ THỐNG VĂN BẰNG SỐ

Xác minh công khai

Không cần đăng nhập

X

XÁC MINH CÔNG KHAI

Kiểm tra văn bằng số

Đối chiếu tính toàn vẹn, trạng thái phát hành và bằng chứng blockchain của chứng thư Blockcerts V3.

CÁCH THỰC HIỆN

Ba bước xác minh

1

Chọn chứng thư

Tải lên tệp JSON do người nhận cung cấp.

2

Kiểm tra nội dung

Có thể xem lại hoặc dán trực tiếp JSON vào biểu mẫu.

3

Nhận kết quả

Hệ thống đối chiếu Merkle proof và trạng thái thu hồi.

Xác minh độc lập

Không cần tài khoản và không làm thay đổi nội dung chứng thư.

CHỨNG THƯ BLOCKCERTS

Làm lại

Nội dung cần xác minh

↑

tampered-bc144d2d-2dc6-4dd4-ba59-03b56256ba9c.json

Chọn tệp

Tệp đã sẵn sàng để xác minh

hoặc dán nội dung

Nội dung JSON

```
{
  "@context": [
    "https://www.w3.org/2018/credentials/v1",
    {
      "alumniOf": "https://schema.org/alumniOf",
      "studentCode": "https://schema.org/identifier",
      "dateOfBirth": "https://schema.org/birthDate",
      "email": "https://schema.org/email",
      "cohort": "https://actvn.edu.vn/credentials/terms#cohort",
      "degreeName": "https://schema.org/educationalCredentialAwarded",
      "major": "https://actvn.edu.vn/credentials/terms#major",
      "educationLevel": "https://schema.org/educationalLevel",
    }
  ]
}
```

2.770 ký tự

Dữ liệu được gửi qua kết nối hiện tại

Xác minh văn bằng

KẾT QUẢ XÁC MINH

Không thể xác thực

Nội dung chứng thư không hợp lệ hoặc bằng chứng blockchain không khớp.

Người nhận

Thực nghiệm B12 Holder 20260920123633

Mô sinh viên

B12-20260920123633

Ngày sinh

2004-01-15

Email

b12.student@example.edu.vn

Niên khóa

2022 - 2027

Tên văn bằng

Bằng tốt nghiệp đại học

Ngành

An Toàn Thông Tin

Trình độ / xếp loại

Đại học - Giỏi

Năm tốt nghiệp

2027

Số hiệu văn bằng

DA-B1-SUA

Hình thức đào tạo

Chính quy

Ngày cấp

2027-06-30

Mô chứng thư

bc144d2d-2dc6-4dd4-ba59-03b56256ba9c

► Chi tiết kỹ thuật

Toàn vẹn chứng thư

Kiểm tra nội dung JSON

Merkle proof

Đối chiếu bằng chứng

Trạng thái hiệu lực

Kiểm tra dữ liệu thu hồi

© 2026 Học viện Kỹ thuật Mật mã

Blockcerts V3 - Bitcoin blockchain

Hình 3.16. Verify phát hiện số hiệu văn bằng trong JSON đã bị sửa

KMA

Blockcerts

Cổng quản trị

KHÔNG GIAN LÀM VIỆC

Tổng quan

Phê duyệt

Văn bằng

Người dùng

Nhật ký

b12_checker_202609201...

Checker

Đăng xuất

VĂN BẰNG

Danh sách phát hành

Theo dõi trạng thái, giao dịch neo và lịch sử thu hồi.

1/25

KMA-UI-1789975876569

Tất cả trạng thái

Làm mới

Đã thu hồi. Txid: e9139fd06733555c02a73e5c27093b22c226ab2ff1fab7e52fc6f5d3c4df6f7

NGƯỜI NHẬN	VĂN BẰNG	TRẠNG THÁI	NGƯỜI DUYỆT	TXID PHÁT HÀNH	THỜI GIAN	THAO TÁC
Thực nghiệm B12 Holder 20260920123633 B12-20260920123633 2022 - 2027	Bằng tốt nghiệp đại học An Toàn Thông Tin Giới KMA-UI- 1789975876569	Đã thu hồi	b12_checker_20260920123633	375d406e9cef...	07:31:18 21/9/2026	Đã ghi OP_RETURN

Hình 3.17. Checker thu hồi văn bằng và ghi nhận giao dịch thu hồi

KMA

HỆ THỐNG VĂN BẰNG SỐ

Xác minh công khai

Không cần đăng nhập

XÁC MINH CÔNG KHAI

Kiểm tra văn bằng số

Đối chiếu tính toàn vẹn, trạng thái phát hành và bằng chứng blockchain của chứng thư Blockcerts V3.

CÁCH THỰC HIỆN

Ba bước xác minh

1

Chọn chứng thư

Tải lên tệp JSON do người nhận cung cấp.

2

Kiểm tra nội dung

Có thể xem lại hoặc dán trực tiếp JSON vào biểu mẫu.

3

Nhận kết quả

Hệ thống đối chiếu Merkle proof và trạng thái thu hồi.

Xác minh độc lập

Không cần tài khoản và không làm thay đổi nội dung chứng thư.

CHỨNG THƯ BLOCKCERTS

Nội dung cần xác minh

bc144d2d-2dc6-4dd4-ba59-03b56256ba9c.json

Tệp đã sẵn sàng để xác minh

Chọn tệp

hoặc dán nội dung

Nội dung JSON

```
{
  "@context": [
    "https://www.w3.org/2018/credentials/v1",
    {
      "alumniOf": "https://schema.org/alumniOf",
      "studentCode": "https://schema.org/identifier",
      "dateOfBirth": "https://schema.org/birthDate",
      "email": "https://schema.org/email",
      "cohort": "https://actvn.edu.vn/credentials/terms#cohort",
      "degreeName": "https://schema.org/educationalCredentialAwarded",
      "major": "https://actvn.edu.vn/credentials/terms#major",
      "educationLevel": "https://schema.org/educationalLevel",
    }
  ]
}
```

2.781 ký tự

Dữ liệu được gửi qua kết nối hiện tại

Xác minh văn bằng

KẾT QUẢ XÁC MINH

Văn bằng đã bị thu hồi

Chứng thư từng được phát hành nhưng hiện không còn hiệu lực.

Người nhận

Thực nghiệm B12 Holder 20260920123633

Mã sinh viên

B12-20260920123633

Ngày sinh

2004-01-15

Email

b12.student@example.edu.vn

Niên khóa

2022 - 2027

Tên văn bằng

Bằng tốt nghiệp đại học

Ngành

An Toàn Thông Tin

Trình độ / xếp loại

Đại học - Giỏi

Năm tốt nghiệp

2027

Số hiệu văn bằng

KMA-UI-1789975676569

Hình thức đào tạo

Chính quy

Ngày cấp

2027-06-30

Mã chứng thư

bc144d2d-2dc6-4dd4-ba59-03b56256ba9c

Chứng thư này đã bị thu hồi

Chi tiết kỹ thuật

Toàn vẹn chứng thư

Kiểm tra nội dung JSON

Merkle proof

Đối chiếu bằng chứng

Trạng thái hiệu lực

Kiểm tra dữ liệu thu hồi

© 2026 Học viện Kỹ thuật Mật mã

Blockcerts V3 · Bitcoin blockchain

Hình 3.18. Verify trả trạng thái đã thu hồi cho chứng thư gốc

Kịch bản bắt đầu từ chứng thư gốc hợp lệ, lần lượt sửa tên người nhận, tên văn bằng, proofValue, transaction id và thử để Maker tự duyệt. Sáu trên sáu ca đạt.

Bảng 3.2. Kết quả Kịch bản 1

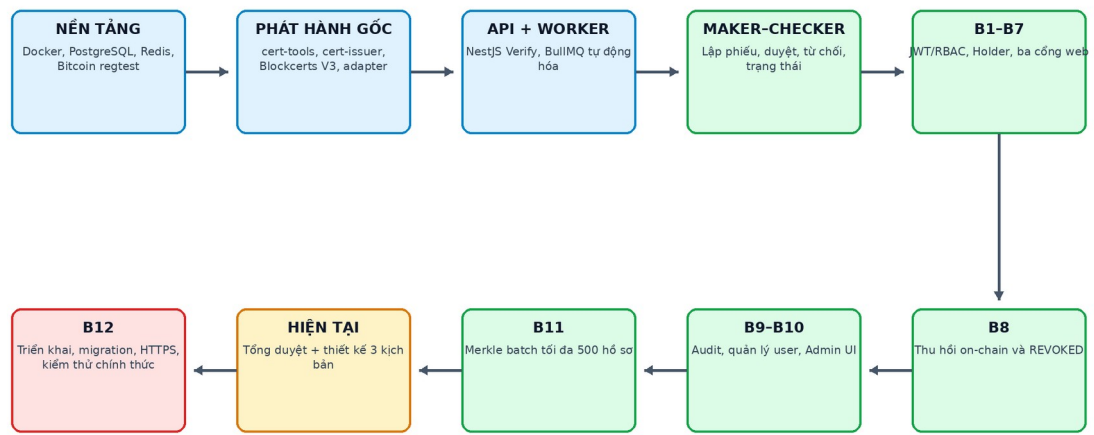
Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S1-01	Chứng thư gốc	VALID	VALID	Đạt
S1-02	Sửa tên người nhận	Không VALID	INVALID	Đạt
S1-03	Sửa tên văn bằng	Không VALID	INVALID	Đạt

76

Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S1-04	Sửa proofValue	Không VALID	INDETERMINATE	Đạt
S1-05	Sửa txid trong proof	Không VALID	INDETERMINATE	Đạt
S1-06	Maker tự duyệt	HTTP 403	HTTP 403	Đạt

Hai trường hợp sửa nội dung trả INVALID; hai trường hợp phá bằng chứng/anchor trả INDETERMINATE. Cả bốn đều không bị nhận là VALID. Ca S1-06 cho thấy guard Maker–Checker đã chặn đúng hành vi tự duyệt trong phạm vi API và cấu hình được kiểm thử, không chỉ ở giao diện.

TIẾN TRÌNH XÂY DỰNG CHƯƠNG 3



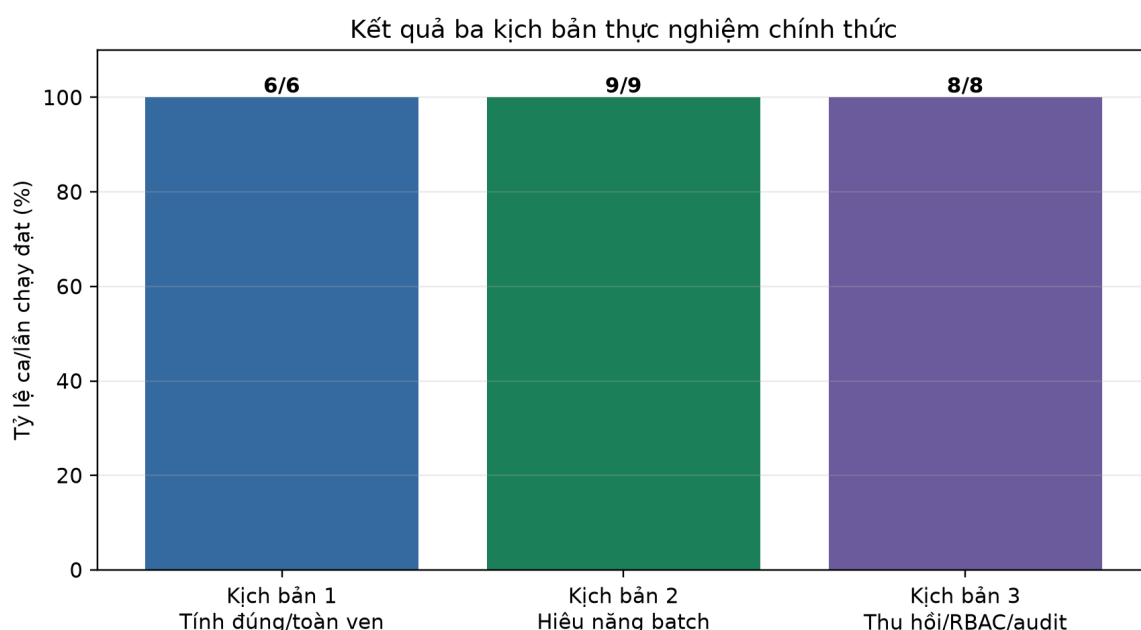
Màu xanh: đã hoàn thành • Màu vàng: đang tổng duyệt • Màu đỏ: chưa triển khai

Hình 3.19. Tiến trình thực nghiệm Chương 3

Bảng 3.3. Kết quả Kịch bản 3

Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S3-01	Trước thu hồi	VALID	VALID	Đạt
S3-02	Student thu hồi	HTTP 403	HTTP 403	Đạt
S3-03	Maker thu hồi	HTTP 403	HTTP 403	Đạt
S3-04	Checker thu hồi	HTTP 201	HTTP 201	Đạt
S3-05	Sau thu hồi	REVOKED	REVOKED	Đạt
S3-06	Tx thu hồi xác nhận	≥ 1	1	Đạt
S3-07	Có trong Revocation List	Có	Có	Đạt
S3-08	Audit đủ role/action	4 action	revoke/issue/approve/request	Đạt

Tám trên tám ca đạt. Trong phạm vi môi trường R2 và các ca đã thiết kế, kết quả ghi nhận chỉ Checker thực hiện được thao tác thu hồi; transaction có xác nhận; Verify, Revocation List và audit trả kết quả nhất quán.



Hình 3.20. Tổng hợp tỷ lệ đạt của ba kịch bản

3.11. Kịch bản 3 – Hiệu năng, phát hành theo lô và khả năng phục hồi

Mục tiêu: giữ toàn bộ benchmark Merkle batching cũ, đồng thời kiểm tra phát hành theo lô có dữ liệu nghiệp vụ và khả năng phục hồi khi Redis, Worker hoặc Bitcoin RPC gặp lỗi.

Giữ nguyên toàn bộ 9 lượt chạy benchmark và 1.830/1.830 proof đã kiểm tra, cùng artifact, checksum, thời gian và số liệu tài nguyên công bố trong các bảng bên dưới.

Mỗi batchId ánh xạ một transaction intent và BullMQ jobId ổn định; transactional outbox giữ yêu cầu khi enqueue Redis lỗi và dispatcher dùng bounded retry với exponential backoff.

Fault injection trước broadcast và sau broadcast/trước DB finalization được khôi phục từ checkpoint. Khi đã có anchorTxid hoặc transaction intent, retry không tạo giao dịch Bitcoin thứ hai.

Reconciliation tra wallet/raw transaction và đối chiếu Merkle root với OP_RETURN trước khi hoàn tất DB; PostgreSQL và Bitcoin không bị mô tả sai thành một giao dịch ACID chung.

Kết quả regression: năm kiểm thử idempotency/reconciliation của IssuanceProcessor và ba kiểm thử outbox/dispatcher đều đạt; health check sau deploy trả HEALTH_OK. Số liệu benchmark và kiểm toán artifact cũ được giữ nguyên đầy đủ trong các tiêu mục sau.

3.11.1. Kết quả từng lần

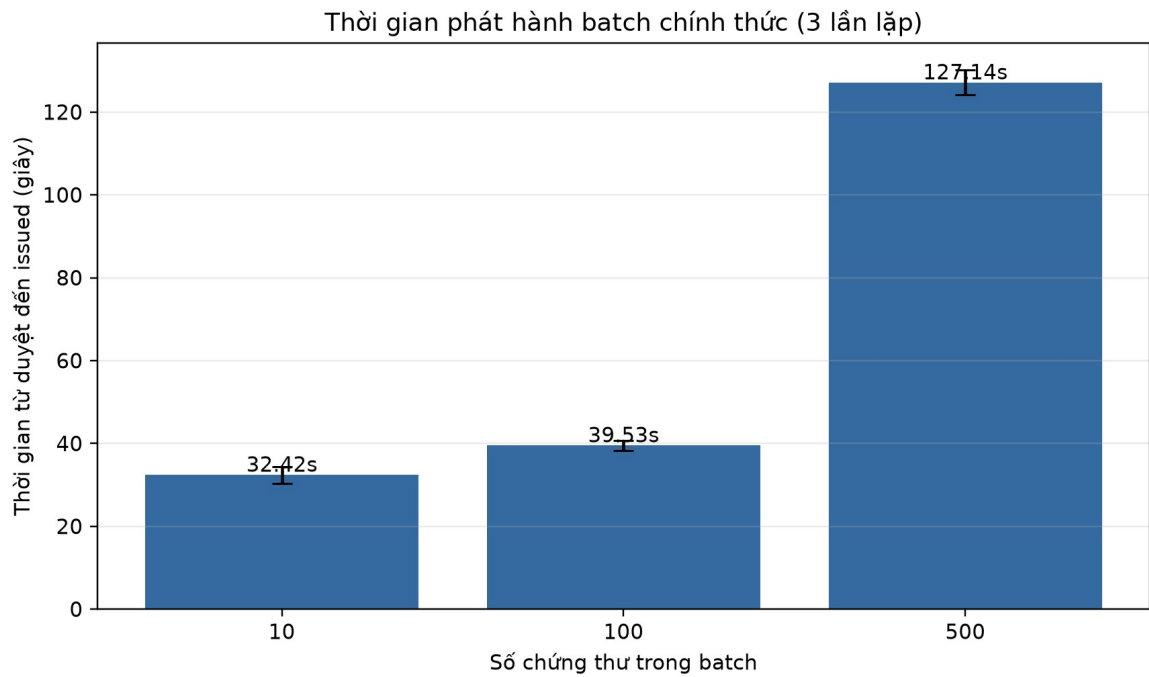
Bảng 3.4. Chín lần chạy hiệu năng chính thức

Kích thước	Lần	Duyệt → issued (giây)	Giây/chứng thư	Chứng thư/giây	Thành công	Transaction
10	1	34,450	3,4450	0,2903	10/10	1
10	2	32,406	3,2406	0,3086	10/10	1
10	3	30,412	3,0412	0,3288	10/10	1
100	1	38,794	0,3879	2,5777	100/100	1
100	2	38,841	0,3884	2,5746	100/100	1
100	3	40,965	0,4097	2,4411	100/100	1
500	1	129,673	0,2593	3,8559	500/500	1
500	2	123,850	0,2477	4,0371	500/500	1
500	3	127,889	0,2558	3,9096	500/500	1

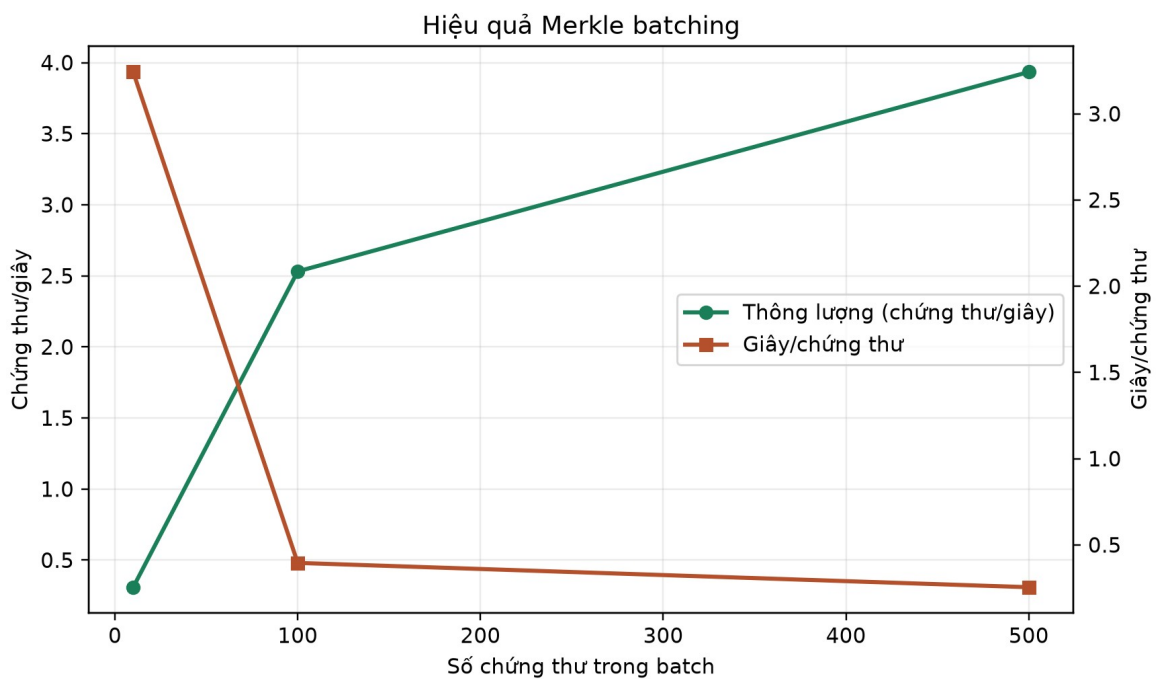
3.11.2. Giá trị tổng hợp

Bảng 3.5. Tổng hợp theo kích thước batch

Batch	Số lần	Tổng chứng thư	Thành công	Thời gian TB ± SD (giây)	Giây/chứng thư TB	Throughput TB (chứng thư/giây)	Tx/batch
10	3	30	100%	32,423 ± 2,019	3,2423	0,3092	1,00
100	3	300	100%	39,533 ± 1,240	0,3953	2,5311	1,00
500	3	1.500	100%	127,137 ± 2,983	0,2543	3,9342	1,00



Hình 3.21. Thời gian phát hành theo kích thước batch



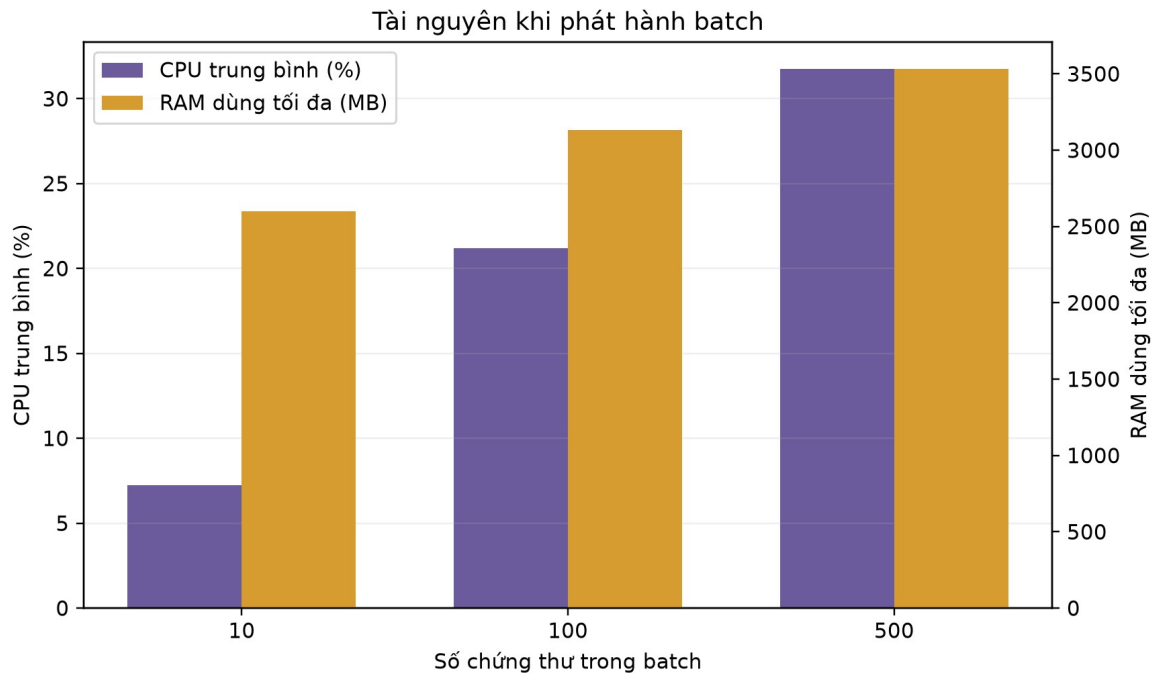
Hình 3.22. Thông lượng và thời gian trên mỗi chứng thư

Khi batch tăng từ 10 lên 500, kích thước tăng 50 lần nhưng thời gian trung bình tăng khoảng 3,92 lần. Throughput tăng khoảng 12,72 lần; thời gian trung bình mỗi chứng thư giảm khoảng 92,16%. Kết quả phản ánh lợi ích phân bổ chi phí cố định và batching trong môi trường thử nghiệm, không phải cam kết hiệu năng trong môi trường vận hành thực tế.

3.11.3. Tài nguyên

Bảng 3.6. Tài nguyên hệ thống

Batch	CPU TB	CPU cực đại	Bộ nhớ dùng TB (MB)	Bộ nhớ cực đại TB (MB)	Backend RSS TB (MB)	Backend RSS cực đại TB (MB)
10	7,24%	15,98%	2.561,1	2.599,4	125,3	126,1
100	21,20%	78,63%	2.877,6	3.130,1	134,5	138,1
500	31,74%	82,30%	3.144,5	3.531,1	159,7	163,8



Hình 3.23. Mức sử dụng tài nguyên

CPU và bộ nhớ tăng theo kích thước batch nhưng nằm trong giới hạn máy thử nghiệm. CPU cực đại khoảng 82,30% ở nhóm 500; bộ nhớ dùng cực đại trung bình khoảng 3.531,1 MB. Số liệu chỉ đại diện cấu hình 16 lõi và concurrency 16 đã nêu.

3.11.4. Kiểm toán artifact

Script kiểm toán ghi nhận 9/9 lần chạy đạt, 1.830/1.830 chứng thư được phát hành, 1.830 certificate ID duy nhất, 1.830/1.830 Merkle proof hợp lệ, 9 batch tương ứng 9 transaction và các transaction có confirmation. SHA256SUMS_PUBLIC kiểm tra 25/25 artifact được công bố; hai artifact nhạy cảm trong manifest nội bộ không nằm trong snapshot công khai.

Bảng 3.7. Kết quả kiểm toán

Chi số	Kết quả
Lần chạy	9/9 đạt
Chứng thư	1.830/1.830
Certificate ID duy nhất	1.830
Merkle proof hợp lệ	1.830/1.830
Batch/transaction	9/9
Artifact checksum công khai	25/25

Tổng 1.830 là tổng lượt/chứng thư của chín lần chạy: $3 \times 10 + 3 \times 100 + 3 \times 500$. Không có việc cộng 1.830 chứng thư rồi nhân thêm ba lần; từng lần chạy sinh tập ID riêng.

3.12. Đánh giá kết quả

3.12.1. Mức đáp ứng

Bảng 3.8. Đối chiếu yêu cầu và bằng chứng

Yêu cầu	Bằng chứng	Kết quả
Maker không tự duyệt	S1-06	Đạt
Phát hiện sửa chứng thư/bằng chứng	S1-02 đến S1-05	Đạt
Phát hành 10/100/500	9 lần chạy, 1.830 chứng thư	Đạt
Một transaction mỗi batch	9 batch/9 transaction	Đạt
Chỉ Checker thu hồi	S3-02 đến S3-04	Đạt
Verify nhận biết thu hồi	S3-05 đến S3-07	Đạt
Audit actor/action	S3-08	Đạt
Build/test/health	8/8 test; 3 frontend; HEALTH_OK	Đạt

3.12.2. Hạn chế

Thứ nhất, regtest không phản ánh phí, độ trễ và kinh tế an ninh của mainnet. Thứ hai, thử nghiệm chạy trên một máy và ba lần lặp mỗi kích thước; chưa đại diện tải dài hạn hoặc hệ thống phân tán nhiều node. Liên kết Student–chứng thư đã chuyển sang studentId/studentCode và có kiểm thử hai sinh viên trùng tên, nhưng chưa tích hợp nguồn định danh chính thức của Học viện. Khóa vẫn chưa ở HSM/KMS; hệ thống chưa có MFA, HTTPS công khai, secret manager và quy trình xoay khóa hoàn chỉnh.

Thứ ba, Revocation List vẫn dựa vào DB và chưa có indexer phục hồi đầy đủ từ blockchain. Thứ tư, audit DB có thể bị quản trị viên hạ tầng sửa. Thứ năm,

fault injection đã bao phủ các điểm trước/sau broadcast của luồng phát hành nhưng chưa mô phỏng mọi lỗi RPC kéo dài, reorg hoặc mất toàn bộ volume. Thứ sáu, safe-fetch đã chặn các trường hợp SSRF chính trong 10 kiểm thử; tuy vậy allowlist, DNS và redirect vẫn cần được rà soát khi hạ tầng thật thay đổi [23]. Thứ bảy, khả năng tương tác với verifier Blockcerts bên ngoài regtest cần tiếp tục đánh giá.

3.12.3. Hướng phát triển

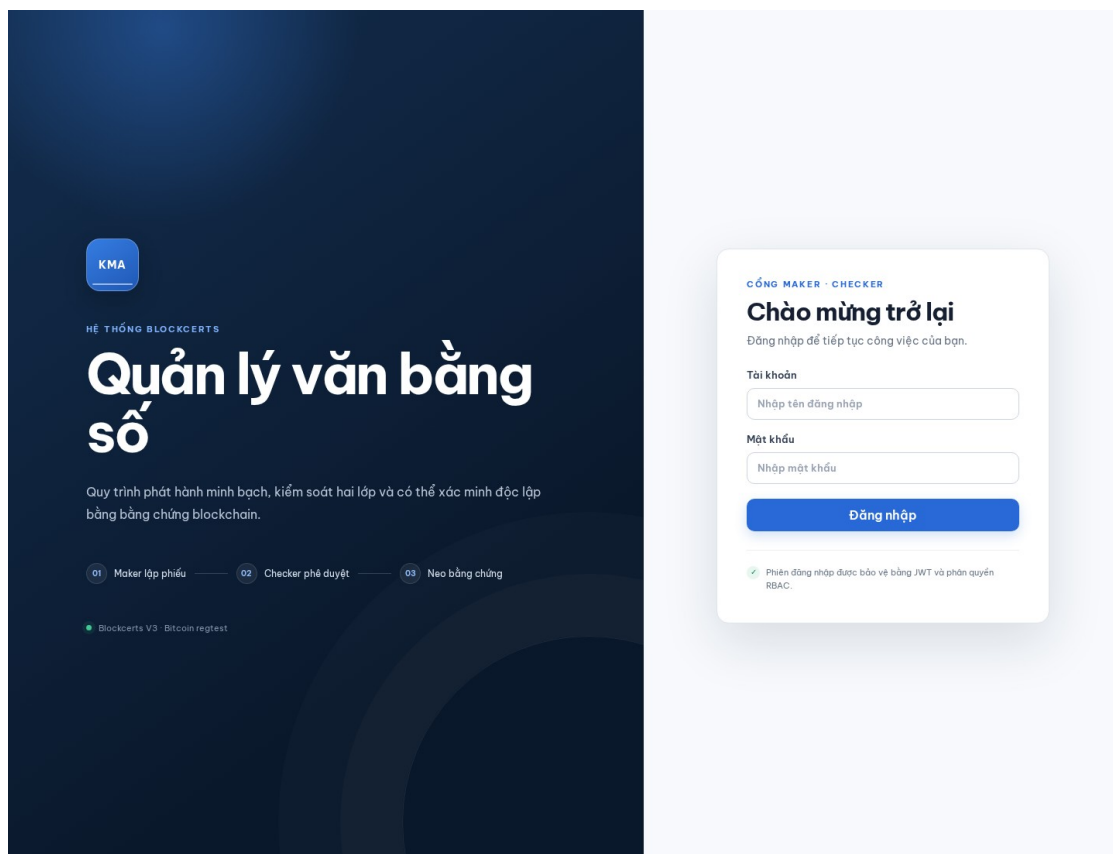
Giai đoạn tiếp theo cần chuẩn hóa riêng bảng request/certificate, xây indexer thu hồi, đưa khóa vào HSM/KMS, thêm MFA, HTTPS/HSTS, sao lưu/khôi phục, giám sát và kiểm thử xâm nhập. Cơ chế outbox, idempotency, checkpoint/reconciliation và safe-fetch hiện tại cần được quan sát dưới tải dài hạn và diễn tập mất Redis, mất RPC, hỏng volume. Trước mainnet cần kiểm thử testnet, xác định phí, confirmation policy, quyền riêng tư và trách nhiệm pháp lý. Có thể bổ sung verifier độc lập phía người dùng để giảm phụ thuộc dịch vụ trung tâm.

3.13. Quy trình thao tác chi tiết để tái hiện ba kịch bản

3.13.1. Bước quy trình 1 – Maker tạo yêu cầu cấp chứng thư

Điều kiện: Checker đã tạo một tài khoản Maker và một tài khoản Student có studentCode duy nhất. Không sử dụng recipientName làm khóa định danh.

Bước 1. Mở `http://<IP_MÁY>:8088/admin/`. Nhập tài khoản Maker và mật khẩu, sau đó chọn “Đăng nhập”. Nếu xác thực đúng, menu chỉ hiển thị chức năng Maker được phép dùng; Maker không nhìn thấy nút phê duyệt hoặc thu hồi.



Hình 3.24. Màn hình đăng nhập cổng quản trị

Bước 2. Chọn “Lập phiếu”, nhập Mã sinh viên đã tồn tại và Địa chỉ nhận/public key hợp lệ của regtest. Kiểm tra lại dữ liệu rồi nhấn “Lập phiếu”.

Hình 3.25. Maker nhập mã sinh viên và địa chỉ nhận của chứng thư

Bước 3. Hệ thống trả thông báo đã lập phiếu và trạng thái pending_approval. Ghi lại mã yêu cầu để đối chiếu audit; Maker không được tự chuyển yêu cầu sang queued.

Blockcerts
Công quản trị

KHÔNG GIAN LÀM VIỆC

- Tổng quan
- Lập phiếu**
- Lập theo lô
- Nhật ký

MAKER

Lập phiếu cấp văn bằng

Họ tên được lấy từ hồ sơ Student theo mã sinh viên, không nhập thủ công.

Mã sinh viên
Ví dụ: AT180001

Địa chỉ nhận / public key
myFN5aFINpvPeyedlipicfsvD2QGVU3D5M

Lập phiếu

Đã lập phiếu b7652a26 - đang chờ Checker duyệt.

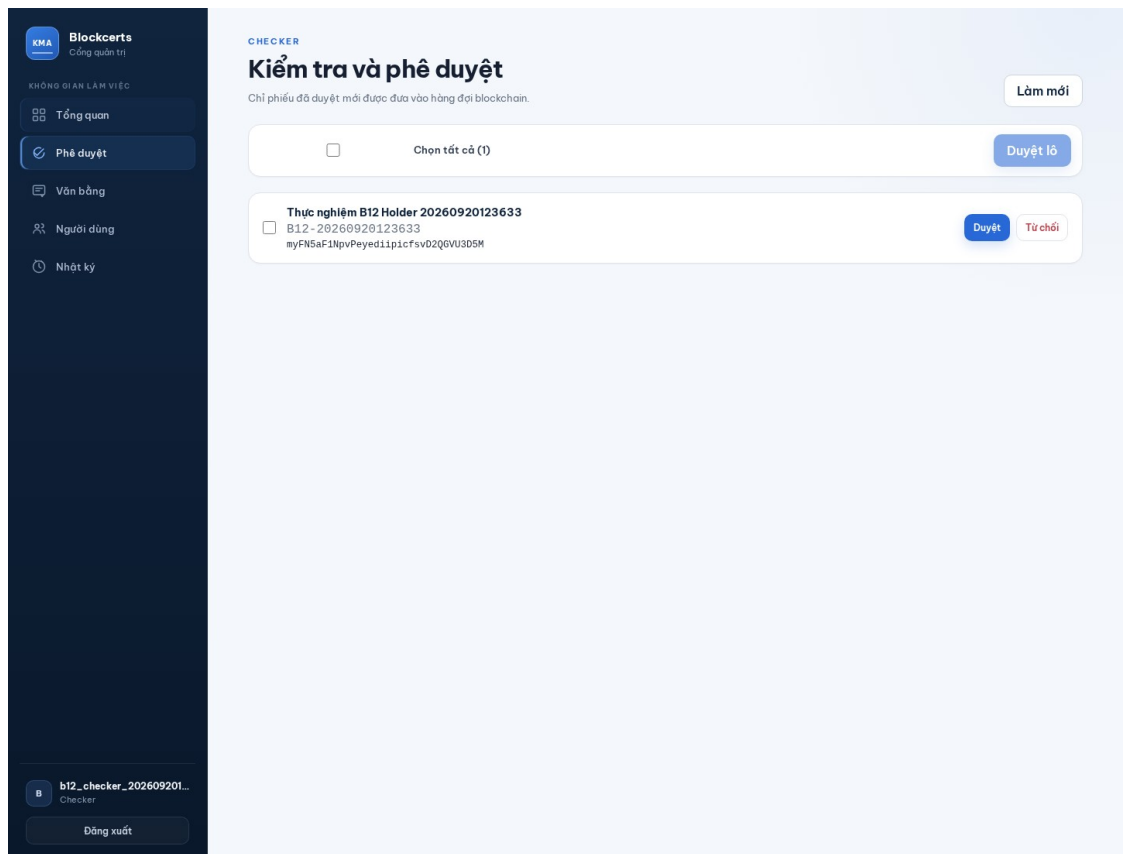
b12_maker_2026092012...
Maker

Đăng xuất

Hình 3.26. Yêu cầu mới được tạo và chờ Checker phê duyệt

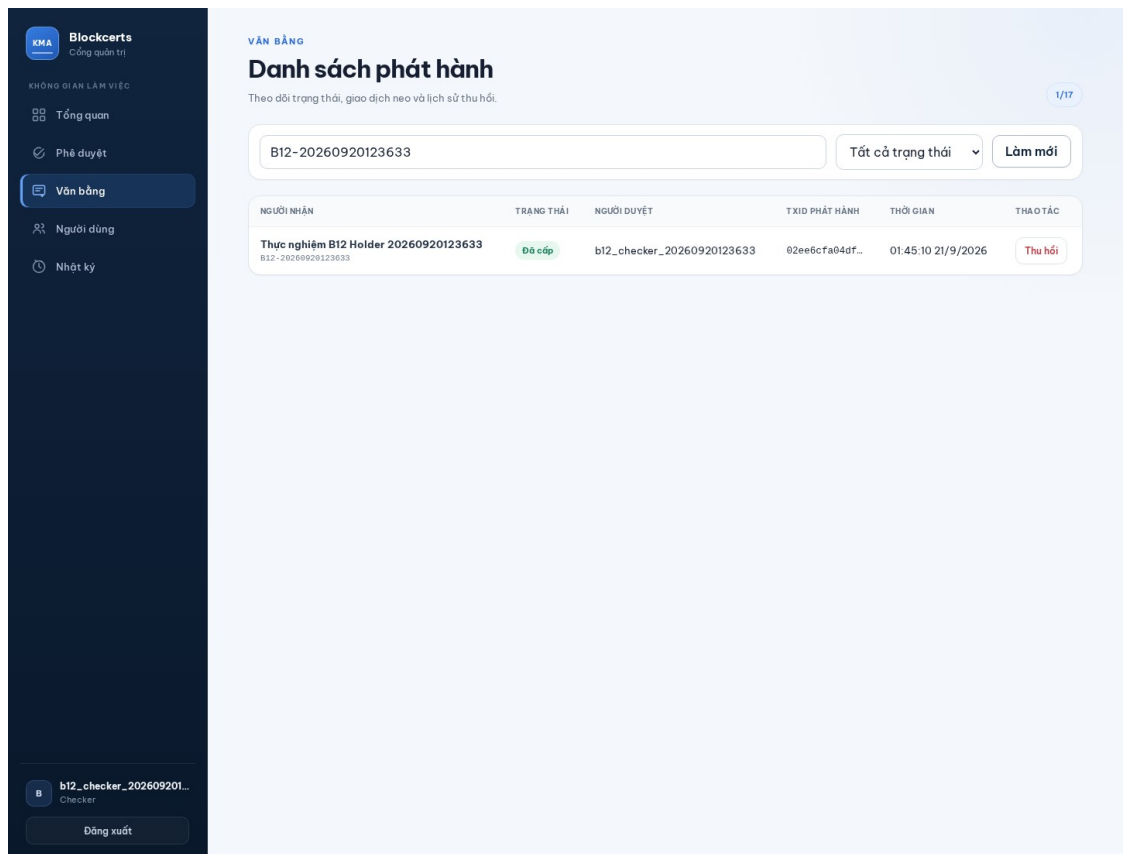
3.13.2. Bước quy trình 2 – Checker phê duyệt và Worker phát hành

Bước 1. Đăng xuất Maker, đăng nhập bằng Checker, chọn “Phê duyệt”. Tìm đúng bản ghi bằng studentCode và đối chiếu người lập, địa chỉ nhận, thời điểm tạo. Nhấn “Duyệt”; nếu dữ liệu sai thì chọn “Từ chối” và nhập lý do.



Hình 3.27. Checker kiểm tra yêu cầu đang chờ phê duyệt

Bước 2. Sau khi duyệt, trạng thái chuyển queued/processing. Dispatcher và Worker chạy bất đồng bộ; không nhấn duyệt lặp lại. Chọn “Văn bằng” và “Làm mới” cho đến khi trạng thái là “Đã cấp”. Bản chạy làm bằng chứng tạo batch single-b7652a26-4d46-40b8-b584-09f9de4dd4f3 và txid 02ee6cfa04df26532d52c14e9976d9b4dba226465b924cd4352dcde540dc1a58.

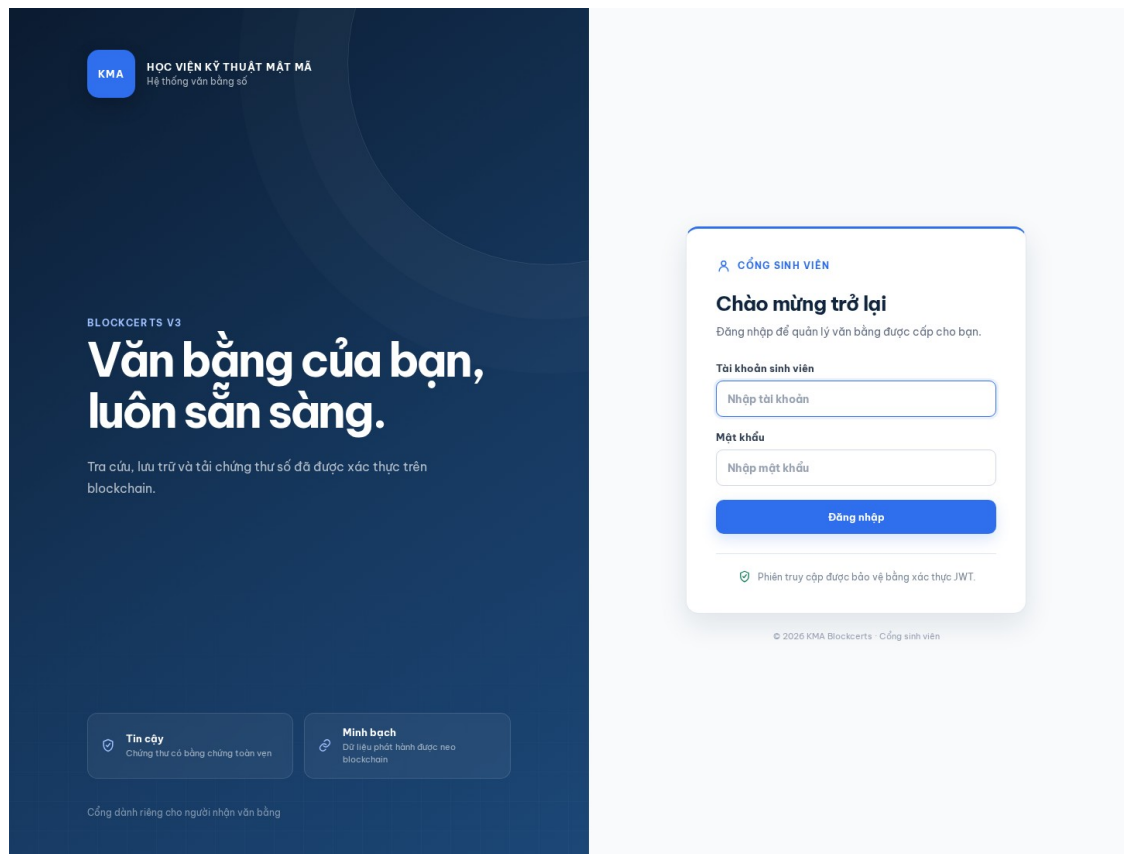


Hình 3.28. Chứng thư chuyển sang trạng thái đã cấp sau khi Worker xử lý

Nếu trạng thái failed, trước tiên xem `journalctl --user -u datn-blockcerts-backend.service`, bản ghi `issuance_batches/outbox` và `checkpoint` theo `batchId`. Chỉ dùng chức năng `retry` sau khi xác định nguyên nhân; tuyệt đối không xóa `checkpoint` hoặc broadcast lại thủ công.

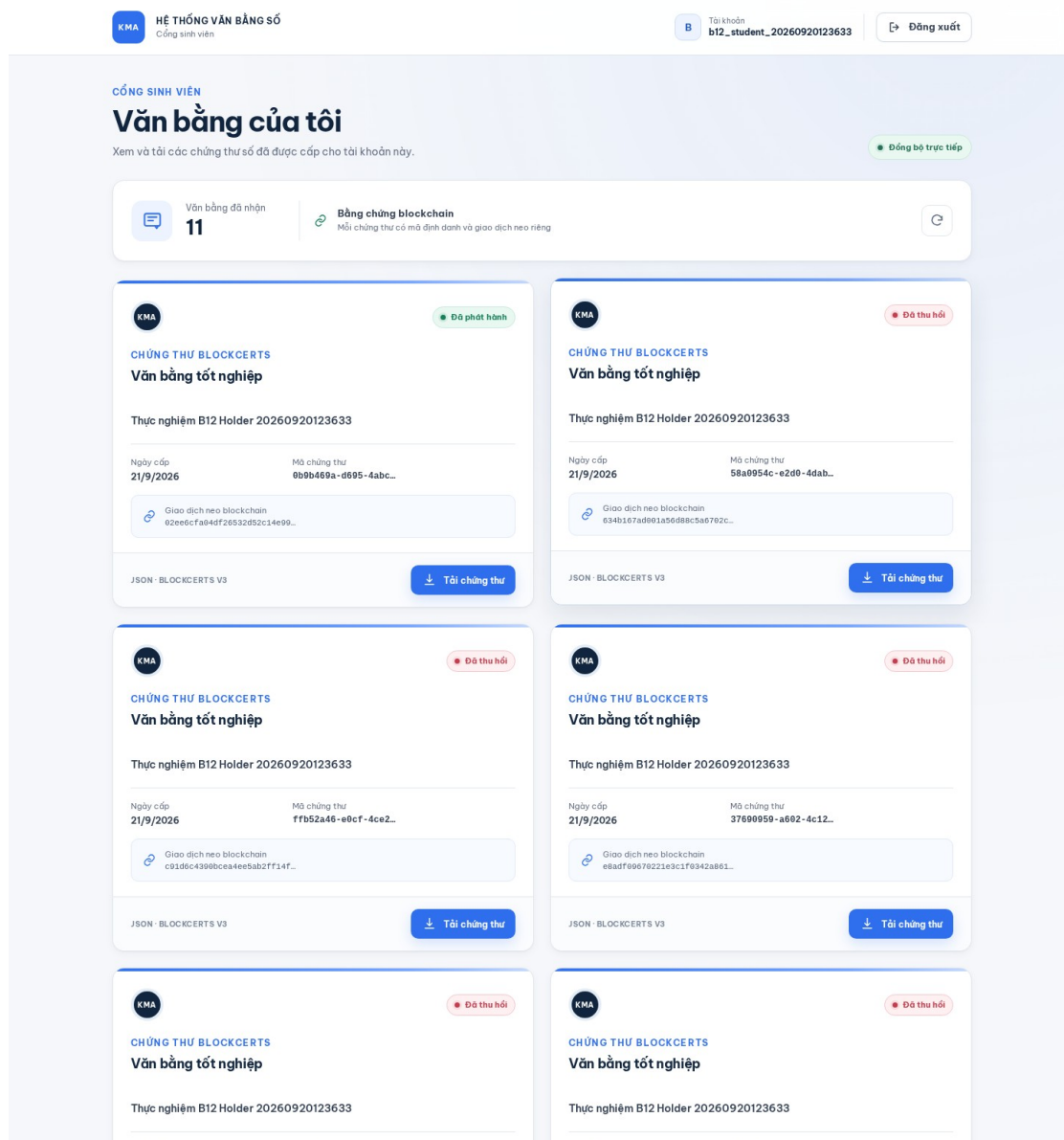
3.13.3. Bước quy trình 3 – Student xem và tải chứng thư

Bước 1. Mở `http://<IP_MÁY>:8088/student/`, đăng nhập bằng tài khoản Student. Backend lấy `studentId/studentCode` trong JWT và chỉ trả chứng thư thuộc đúng người dùng; hai sinh viên trùng tên nhưng khác mã vẫn bị tách biệt.



Hình 3.29. Màn hình đăng nhập cổng Student

Bước 2. Tại “Văn bằng của tôi”, kiểm tra tên, mã sinh viên, trạng thái, certUid và txid; chọn “Tải JSON”. Tập JSON này là đầu vào cho cổng xác minh công khai. Student vẫn nhìn thấy chứng thư đã thu hồi nhưng trạng thái phải được hiển thị rõ.



Hình 3.30. Student xem danh sách và tải chứng thư thuộc tài khoản

3.13.4. Bước quy trình 4 – Xác minh công khai chứng thư hợp lệ

Bước 1. Không cần đăng nhập, mở http://<IP_MÁY>:8088/verify/. Chọn tệp JSON vừa tải rồi nhấn “Xác minh văn bằng”. Verifier kiểm tra cấu trúc VC/Blockcerts, Merkle proof, giao dịch neo regtest, địa chỉ phát hành, confirmation và trạng thái thu hồi.

KMA

HỆ THỐNG VĂN BẰNG SỐ

Xác minh công khai

Không cần đăng nhập

XÁC MINH CÔNG KHAI

Kiểm tra văn bằng số

Đối chiếu tính toán vẹn, trạng thái phát hành và bằng chứng blockchain của chứng thư Blockcerts V3.

CÁCH THỰC HIỆN

Ba bước xác minh

1

Chọn chứng thư

Tải lên tệp JSON do người nhận cung cấp.

2

Kiểm tra nội dung

Có thể xem lại hoặc dán trực tiếp JSON vào biểu mẫu.

3

Nhận kết quả

Hệ thống đối chiếu Merkle proof và trạng thái thu hồi.

Xác minh độc lập

Không cần tài khoản và không làm thay đổi nội dung chứng thư.

CHỨNG THƯ BLOCKCERTS

Nội dung cần xác minh

Chọn tệp chứng thư JSON

Tối đa 5 MB - định dạng json

Chọn tệp

hoặc dán nội dung

Nội dung JSON

{

"@context": [...],

"type": "BlockcertsCredential",

...

}

Đã ký tự

Dữ liệu được gửi qua kết nối nội nhiệm tại

Xác minh văn bằng

Toàn vẹn chứng thư

Kiểm tra nội dung JSON

Merkle proof

Đối chiếu bằng chứng lỗ

Trạng thái hiệu lực

Kiểm tra dữ liệu thu hồi

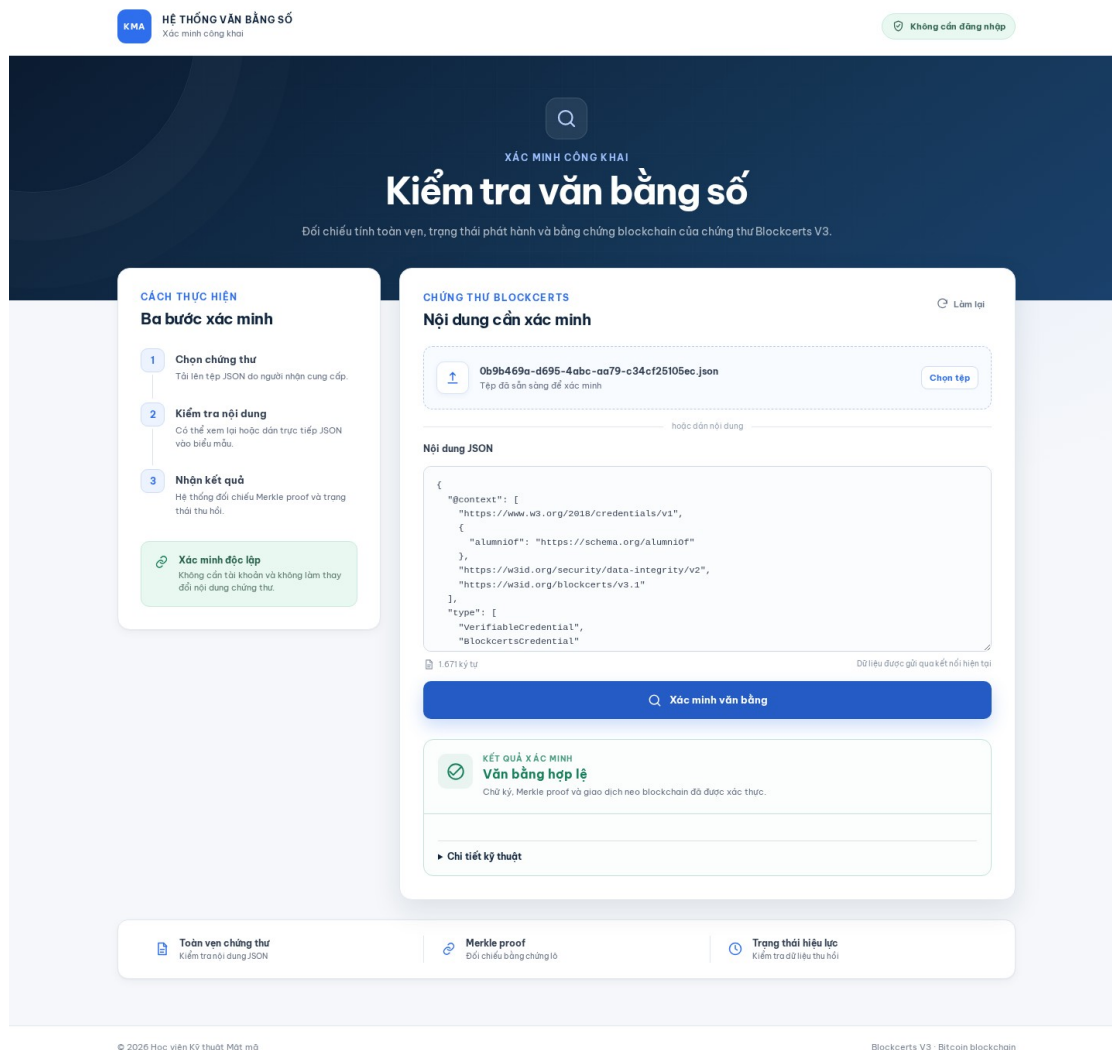
© 2026 Học viện Kỹ thuật Mật mã

Blockcerts V3 - Bitcoin blockchain

Hình 3.31. Cổng Verify công khai trước khi tải chứng thư

Bước 2. Khi tất cả điều kiện đạt, kết quả là “Văn bằng hợp lệ”; giao diện hiển thị certUid, txid, Merkle root và các bước kiểm tra. Không kết luận hợp lệ nếu một nguồn dữ liệu bắt buộc không truy cập được; trường hợp đó phải trả INDETERMINATE.

91



Hình 3.32. Kết quả xác minh hợp lệ trước khi chứng thư bị thu hồi

3.13.5. Bước quy trình 5 – Phát hiện chứng thư bị sửa

Tạo một bản sao của JSON, thay đổi một trường nghiệp vụ như credentialSubject.name nhưng giữ nguyên receipt, rồi tải bản sao lên Verify. Không sửa tệp gốc. Có thể thao tác bằng trình soạn thảo hoặc lệnh jq:

```
jq '.credentialSubject.name = "TEN_DA_BI_SUA"' certificate.json > certificate-tampered.json
```

Kết quả bắt buộc là “Không thể xác thực”/INVALID vì target hash và Merkle proof không còn khớp. Ảnh dưới đây được tạo từ bản sao bị sửa của chính chứng thư ở Kịch bản 4.

KMA

HỆ THỐNG VĂN BẰNG SỐ

Xác minh công khai

Không cần đăng nhập

XÁC MINH CÔNG KHAI

Kiểm tra văn bằng số

Đối chiếu tính toàn vẹn, trạng thái phát hành và bằng chứng blockchain của chứng thư Blockcerts V3.

CÁCH THỰC HIỆN

Ba bước xác minh

1 Chọn chứng thư

Tải lên tệp JSON do người nhận cung cấp.

2 Kiểm tra nội dung

Có thể xem lại hoặc dán trực tiếp JSON vào biểu mẫu.

3 Nhận kết quả

Hệ thống đối chiếu Merkle proof và trạng thái thu hồi.

Xác minh độc lập

Không cần tài khoản và không làm thay đổi nội dung chứng thư.

CHỨNG THƯ BLOCKCERTS

Nội dung cần xác minh

tampered-0b9b469a-d695-4abc-aa79-c34cf25105ec.json

Tệp đã sẵn sàng để xác minh

Chọn tệp

hoặc dán nội dung

Nội dung JSON

```
{
  "@context": [
    "https://www.w3.org/2018/credentials/v1",
    {
      "alumniOf": "https://schema.org/alumniOf"
    }
  ],
  "https://w3id.org/security/data-integrity/v2",
  "https://w3id.org/blockcerts/v3.1"
],
  "type": [
    "VerifiableCredential",
    "BlockcertsCredential"
  ]
}
```

1.652 ký tự

Dữ liệu được gửi qua kết nối an toàn

Xác minh văn bằng

KẾT QUẢ XÁC MINH

Không thể xác thực

Nội dung chứng thư không hợp lệ hoặc bằng chứng blockchain không khớp.

Chi tiết kỹ thuật

Toàn vẹn chứng thư

Kiểm tra nội dung JSON

Merkle proof

Đối chiếu bằng chứng

Trạng thái hiệu lực

Kiểm tra dữ liệu thu hồi

© 2026 Học viện Kỹ thuật Mật mã

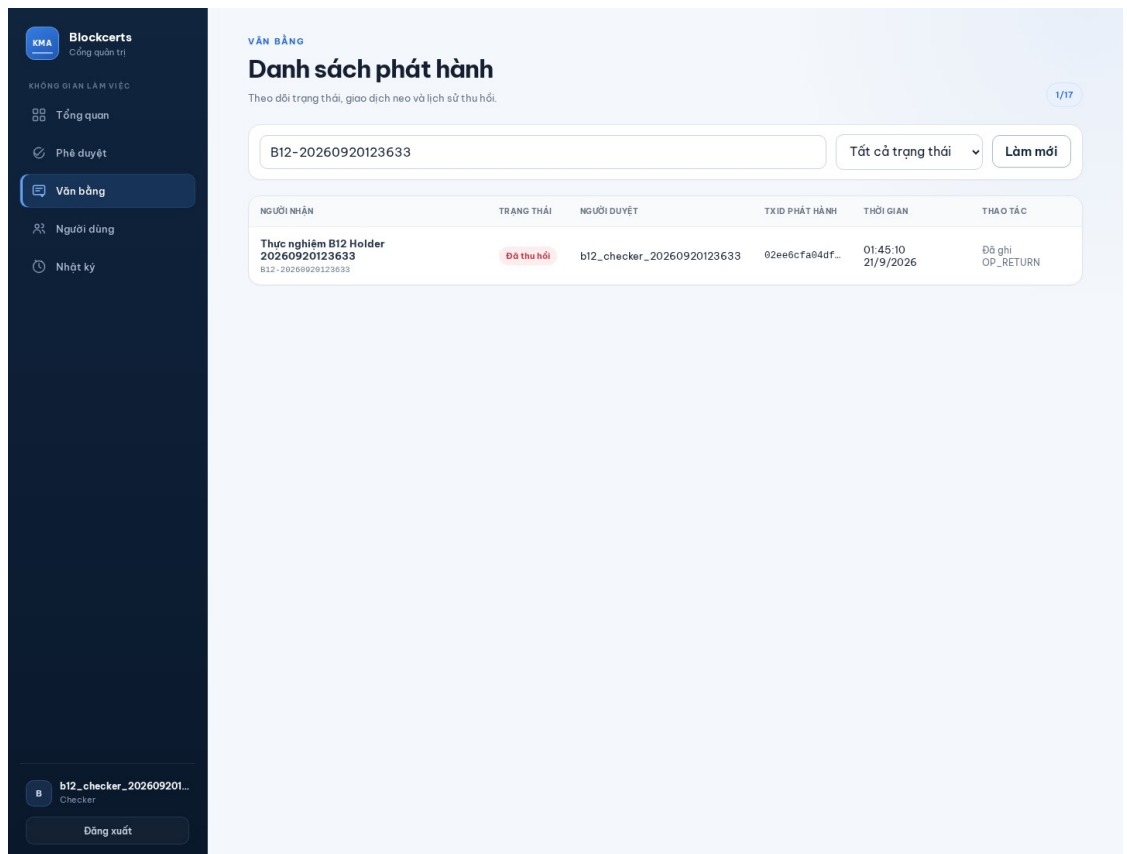
Blockcerts V3 · Bitcoin blockchain

Hình 3.33. Verifier phát hiện nội dung chứng thư đã bị thay đổi

3.13.6. Bước quy trình 6 – Checker thu hồi và xác minh lại

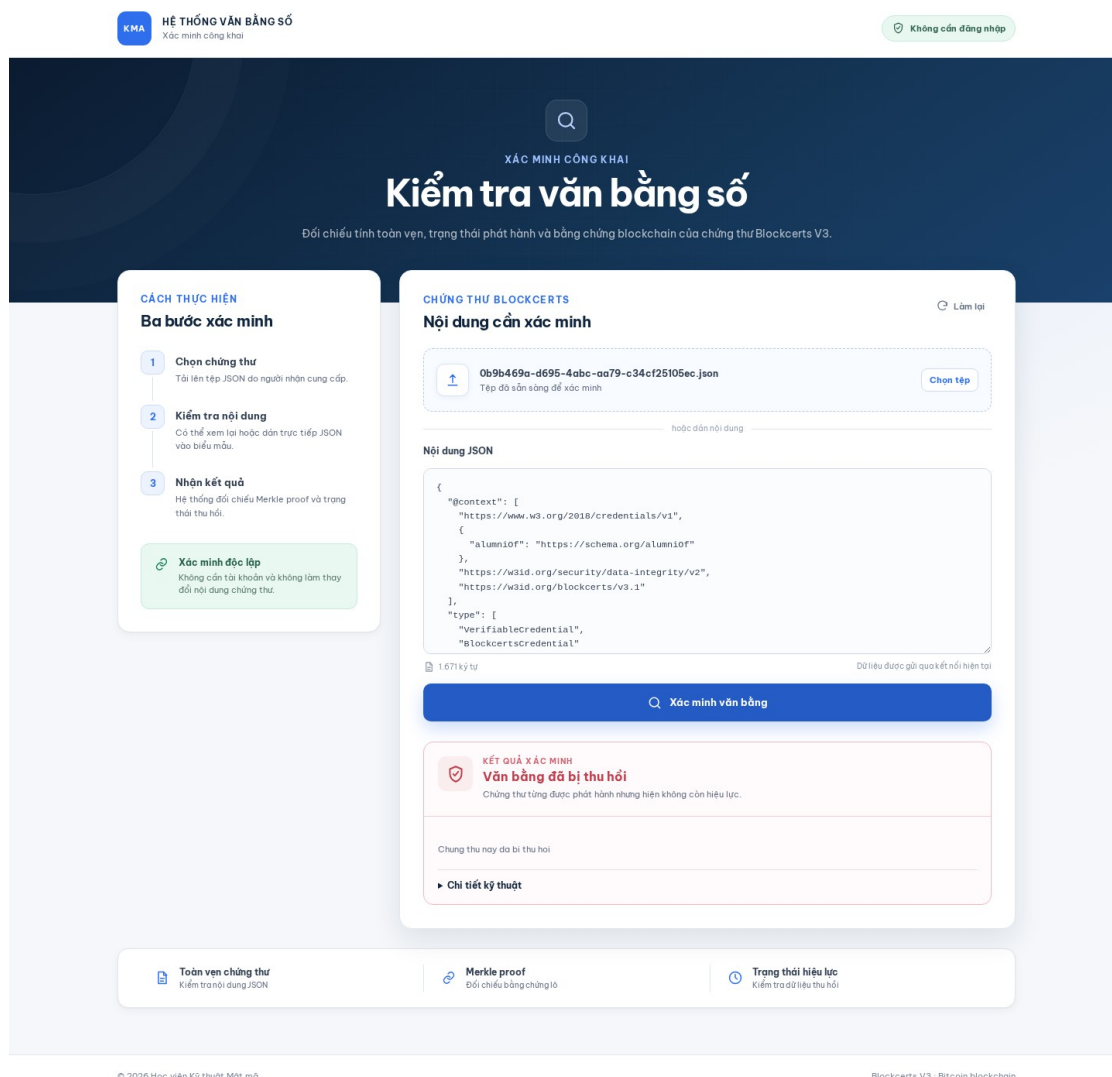
Bước 1. Checker vào “Văn bằng”, tìm theo studentCode, chọn “Thu hồi”, nhập lý do và xác nhận. Hệ thống tạo giao dịch OP_RETURN REVOKE:<certUid>, đào block regtest rồi cập nhật DB. Trong lần chạy minh họa, revocation txid là e30509406db61ab6aa877ced58933c68e50e7549777eeb8d6eff4d679c9f8548.

93



Hình 3.34. Checker thu hồi chứng thư và hệ thống ghi nhận trạng thái

Bước 2. Tải lại tệp gốc chưa sửa lên Verify. Bằng chứng mật mã vẫn đúng nhưng kết quả cuối phải là “Văn bằng đã bị thu hồi”/REVOKED; không được trả VALID.



Hình 3.35. Cổng Verify trả trạng thái đã thu hồi cho chứng thư gốc

3.13.7. Bước quy trình 7 – Kiểm tra RBAC, isolation và SSRF

Chạy bộ E2E với tài khoản thử nghiệm cục bộ. Kịch bản phải xác nhận Maker không tự duyệt, Student không truy cập API quản trị, hai Student khác mã không đọc chứng thư của nhau và verifier từ chối URL loopback/private/link-local/metadata. Không ghi tài khoản hoặc mật khẩu thử nghiệm vào báo cáo.

```
cd ~/DATN_work/datn-blockcerts/experiments
node scripts/e2e-preflight.mjs
cd ../apps/verifier-service
npm test
# Kết quả xác minh cuối: E2E PASS; verifier 10/10 test PASS.
```

3.13.8. Bước quy trình 8 – Đối chiếu trạng thái và bằng chứng sau thử nghiệm

Sau mỗi phiên demo, kiểm tra health, trạng thái container, service và nhánh Git. Các lệnh này không in secret:

```
cd ~/DATN_work/datn-blockcerts
./scripts/health-check-b12.sh
docker compose ps
systemctl --user is-active datn-blockcerts-backend.service datn-blockcerts-
nginx.service
git status --branch --short
```

Kết quả mong đợi trên bản triển khai đã xác minh là HEALTH_OK; PostgreSQL, Redis và Bitcoin Core healthy; hai service active; working tree sạch hoặc chỉ có đúng artifact cục bộ đã chủ động tạo và đang bị ignore. Nếu khác, phải lưu output thực tế và xử lý trước khi dùng số liệu trong báo cáo.

3.14. Hỏi đáp và tự phản biện

Phần này tổng hợp các câu hỏi có khả năng xuất hiện khi đọc, phản biện hoặc bảo vệ đề án. Câu trả lời phân biệt rõ năng lực đã được chứng minh trong môi trường regtest với các yêu cầu bổ sung trước khi triển khai production; vì vậy chúng không được hiểu là cam kết hệ thống hiện tại đã sẵn sàng vận hành thực tế.

3.14.1. Chi phí, Bitcoin và mô hình vận hành

Q1: Nếu triển khai production ngoài đời thì chi phí như thế nào? Có mất Bitcoin thật không?

A: Có. Trên Bitcoin mainnet, tổ chức phải sở hữu một lượng bitcoin để trả phí giao dịch neo dữ liệu; hệ thống không cần chuyển một bitcoin nguyên vẹn cho người nhận. Chi phí mỗi lô xấp xỉ kích thước giao dịch nhân với mức phí satoshi/vbyte tại thời điểm phát hành, cộng chi phí máy chủ, lưu trữ, giám sát, sao lưu, tên miền, TLS, nhân sự vận hành và quản lý khóa. Merkle batching giúp nhiều văn bản dùng chung một giao dịch nên chi phí bình quân trên mỗi văn bản giảm mạnh.

Q2: Có phải mỗi văn bản tốn một giao dịch Bitcoin không?

A: Không. Hệ thống băm các chứng thư thành cây Merkle rồi chỉ neo một Merkle root cho cả lô. Mỗi chứng thư giữ Merkle proof riêng để chứng minh nó thuộc lô, do đó một giao dịch có thể phục vụ nhiều văn bản.

Q3: Phí phát hành có cố định không?

A: Không. Phí mainnet biến động theo nhu cầu không gian khối, kích thước giao dịch và chính sách chọn mức phí. Khi vận hành thật cần cơ chế ước lượng phí, ngưỡng ngân sách, lịch phát hành theo lô và chính sách tăng phí hoặc phát hành lại khi giao dịch bị treo.

Q4: Nguyên mẫu hiện tại có tiêu tiền thật không?

A: Không. Thực nghiệm dùng Bitcoin regtest, nơi block và coin được tạo cục bộ, không có giá trị kinh tế. Kết quả regtest chứng minh luồng kỹ thuật nhưng không đại diện cho phí, độ trễ, thanh khoản hay rủi ro vận hành của mainnet.

Q5: Ngoài phí blockchain còn những khoản nào?

A: Các khoản chính gồm hạ tầng ứng dụng và cơ sở dữ liệu, Bitcoin node hoặc nhà cung cấp RPC, lưu trữ bản sao, giám sát và cảnh báo, HSM/KMS, kiểm thử xâm nhập, chứng thư TLS, tên miền, trực vận hành, hỗ trợ người dùng và tuân thủ pháp lý. Trong nhiều triển khai, chi phí con người và an toàn khóa lớn hơn phí neo blockchain.

Q6: Có thể dùng testnet để vận hành thật nhằm tiết kiệm không?

A: Không nên. Testnet không bảo đảm giá trị kinh tế, độ ổn định hay tính chống viết lại tương đương mainnet và có thể được reset hoặc gián đoạn. Testnet phù hợp tích hợp trước triển khai; dữ liệu cần giá trị xác minh lâu dài phải dùng mạng có mô hình tin cậy phù hợp.

Q7: Có thể giảm phí bằng cách chờ gom đủ lô không?

A: Có, nhưng phải đánh đổi với thời gian chờ phát hành. Production nên đặt ngưỡng theo số lượng hoặc thời gian, ví dụ phát hành khi đủ lô hoặc đến hạn dịch vụ, đồng thời cho phép lô khẩn cấp với mức phí cao hơn.

3.14.2. Lý do chọn Blockchain và Bitcoin

Q8: Tại sao cần Blockchain, cơ sở dữ liệu ký số thông thường có đủ không?

A: Cơ sở dữ liệu kết hợp chữ ký số có thể đủ nếu mọi bên luôn tin tưởng và truy cập được tổ chức phát hành. Blockchain bổ sung mốc thời gian và cam kết mật mã trên một hạ tầng độc lập, giúp bên thứ ba kiểm tra mà không chỉ dựa vào bản ghi nội bộ. Tuy nhiên Blockchain không tự bảo đảm dữ liệu đầu vào đúng và không thay thế quản trị nghiệp vụ.

Q9: Tại sao chọn Bitcoin thay vì Ethereum?

A: Blockcerts có mô hình neo Merkle root phù hợp với giao dịch Bitcoin và không cần smart contract. Bitcoin cung cấp lịch sử giao dịch lâu dài và mô hình xác minh đơn giản; Ethereum có thể phù hợp khi cần logic on-chain phong phú nhưng làm tăng bề mặt hợp đồng, phí và độ phức tạp mà bài toán này chưa cần.

Q10: Tại sao không dùng Blockchain riêng hoặc Hyperledger Fabric?

A: Mạng cấp quyền cho hiệu năng và quản trị thành viên tốt hơn, nhưng người xác minh phải tin vào liên minh vận hành mạng. Bitcoin công khai tạo điểm neo độc lập hơn. Nếu yêu cầu chủ quyền dữ liệu, thông lượng và quản trị nội bộ quan trọng hơn tính kiểm chứng công khai, Fabric có thể là lựa chọn hợp lý.

Q11: Blockchain có làm văn bằng trở thành “đúng” tuyệt đối không?

A: Không. Blockchain chỉ giúp phát hiện thay đổi và chứng minh dữ liệu đã được neo bởi khóa của tổ chức tại một thời điểm. Nếu Maker nhập sai hoặc

Checker duyệt sai, bản ghi sai vẫn có thể được neo bất biến; vì vậy quy trình Maker–Checker, đối soát nguồn và thu hồi vẫn bắt buộc.

Q12: Hệ thống có phải ứng dụng phi tập trung hoàn toàn không?

A: Không. Backend, cơ sở dữ liệu, Issuer Profile, khóa phát hành và quy trình phê duyệt vẫn do tổ chức quản lý. Thành phần phi tập trung chủ yếu là lớp neo và kiểm chứng bằng chứng; báo cáo không tuyên bố loại bỏ hoàn toàn điểm tin cậy trung tâm.

Q13: Bitcoin ngừng hoạt động thì sao?

A: Khả năng này thấp nhưng không thể coi bằng không. Chứng thư và dữ liệu nghiệp vụ vẫn còn ngoài chuỗi, song xác minh anchor mới hoặc đọc lịch sử sẽ bị ảnh hưởng. Production cần cache có kiểm soát, nhiều node/nhà cung cấp độc lập, quy trình gián đoạn và kế hoạch chuyển cơ chế neo nếu tiêu chuẩn thay đổi.

Q14: Proof of Work có gây lãng phí năng lượng cho từng văn bản không?

A: Một văn bản không tạo riêng một quá trình đào; nó sử dụng bảo mật chung của mạng. Dù vậy, tác động môi trường của mạng là vấn đề hợp lệ. Merkle batching, tần suất neo hợp lý và đánh giá lựa chọn mạng theo chính sách ESG là các biện pháp cần xem xét.

3.14.3. Tính đúng đắn mật mã và xác minh

Q15: Merkle proof chứng minh điều gì?

A: Nó chứng minh hash của chứng thư thuộc tập lá tạo ra Merkle root đã neo. Proof không tự chứng minh danh tính người học hay tính đúng của nội dung; các yếu tố đó phụ thuộc chữ ký, Issuer Profile và quy trình nghiệp vụ.

Q16: Nếu sửa một ký tự trong chứng thư thì sao?

A: Hash của chứng thư thay đổi nên đường Merkle proof không còn dẫn tới root đã neo và kết quả phải là INVALID. Đây là tính chất chống sửa đổi, không phải cơ chế mã hóa nội dung.

Q17: Hai chứng thư có thể có cùng hash không?

A: Về lý thuyết va chạm hash tồn tại, nhưng với hàm băm mật mã phù hợp xác suất thực tế là cực nhỏ. Production vẫn phải cố định thuật toán được tiêu chuẩn hỗ trợ và có kế hoạch chuyển đổi nếu thuật toán bị suy yếu.

Q18: Xác minh có cần truy cập máy chủ trường không?

A: Để kiểm tra đầy đủ thường cần truy cập Issuer Profile, danh sách thu hồi và nguồn blockchain hoặc node. Một phần kiểm tra cấu trúc/hash có thể làm cục bộ, nhưng kết luận về danh tính tổ chức, trạng thái thu hồi và anchor mới nhất cần dữ liệu đáng tin cậy.

Q19: Tại sao kết quả có trạng thái INDETERMINATE?

A: Trạng thái này dùng khi thiếu bằng chứng để kết luận, chẳng hạn RPC lỗi, Issuer Profile không truy cập được hoặc dịch vụ verifier gián đoạn. Không được biến lỗi hạ tầng thành INVALID vì điều đó có thể kết tội nhầm một chứng thư hợp lệ.

Q20: Bao nhiêu confirmation là đủ?

A: Nguyên mẫu yêu cầu tối thiểu một confirmation để hoàn tất thử nghiệm nhanh. Production phải chọn ngưỡng theo giá trị rủi ro và thời gian chấp nhận; càng nhiều confirmation thì rủi ro reorg càng thấp nhưng người dùng chờ lâu hơn.

Q21: Nếu Bitcoin xảy ra reorg thì chứng thư có mất không?

A: Reorg ngắn có thể làm giao dịch tạm thời mất confirmation hoặc quay lại mempool. Worker và verifier phải theo dõi trạng thái đến ngưỡng xác nhận, không coi broadcast là hoàn tất, và có quy trình phát hành lại/đối soát nếu giao dịch bị loại.

Q22: Làm sao phân biệt tổ chức phát hành thật với kẻ giả mạo?

A: Người xác minh phải kiểm tra Issuer Profile, miền công bố, khóa/địa chỉ được ủy quyền và chuỗi tin cậy quản trị của tổ chức. Blockchain chỉ chứng minh giao dịch đến từ một khóa; việc gắn khóa đó với một trường hợp pháp vẫn cần cơ chế định danh đáng tin cậy.

3.14.4. Khóa, danh tính và quyền sở hữu

Q23: Nếu khóa phát hành bị lộ thì hậu quả gì?

A: Kẻ tấn công có thể neo chứng thư giả hoặc tạo bằng chứng gây nhầm lẫn dưới khóa bị xâm phạm. Cần ngừng phát hành, xoay khóa, công bố sự cố, cập nhật Issuer Profile theo quy trình có kiểm soát, rà soát các giao dịch trong khoảng ảnh hưởng và dùng HSM/KMS cùng phê duyệt nhiều người.

Q24: Tại sao không lưu private key trong cơ sở dữ liệu?

A: Cơ sở dữ liệu là mục tiêu thường xuyên của tấn công và được nhiều thành phần truy cập. Tách khóa khỏi dữ liệu nghiệp vụ, hạn chế quyền đọc và dùng HSM/KMS giúp giảm bán kính ảnh hưởng; bản nguyên mẫu dùng tệp WIF cục bộ chỉ phù hợp thí nghiệm.

Q25: Sinh viên mất khóa thì có mất văn bằng không?

A: Văn bằng và bằng chứng neo không biến mất, nhưng người học có thể mất khả năng chứng minh quyền kiểm soát khóa nếu quy trình yêu cầu. Tổ chức cần chính sách cấp lại hoặc liên kết khóa mới, giữ lịch sử và thu hồi phiên bản cũ thay vì sửa blockchain.

Q26: Public key của sinh viên có phải dữ liệu cá nhân không?

A: Có thể có, đặc biệt khi khóa được liên kết với danh tính và lịch sử học tập. Cần đánh giá pháp lý, tối thiểu hóa dữ liệu, thông báo mục đích sử dụng và tránh tái sử dụng cùng một định danh công khai cho nhiều ngữ cảnh nếu gây khả năng theo dõi.

Q27: Có thể dùng ví Bitcoin thông thường của sinh viên không?

A: Về kỹ thuật có thể dùng loại địa chỉ/khóa được chuẩn hỗ trợ, nhưng trải nghiệm quản lý khóa và khôi phục phải được thiết kế rõ. Không nên yêu cầu sinh viên giữ số dư; khóa nhận văn bằng và ví tài sản nên tách biệt để giảm rủi ro và hiểu nhầm.

Q28: Maker hoặc Checker nghỉ việc thì sao?

A: Tài khoản phải bị vô hiệu hóa, token bị thu hồi, quyền được rà soát và audit được lưu giữ. Khóa phát hành không nên thuộc cá nhân; nó thuộc tổ chức và được quản lý bằng vai trò, phê duyệt nhiều người và quy trình bàn giao.

Q29: Một Checker có thể tự ý cấp văn bằng không?

A: Trong thiết kế hiện tại, Checker duyệt những Maker tạo yêu cầu, giúp tách nhiệm vụ. Tuy nhiên nếu hai người thông đồng hoặc Checker có thêm quyền quản trị dữ liệu ngoài luồng thì kiểm soát có thể bị vượt qua; production cần đối soát hệ thống đào tạo, giới hạn quyền, cảnh báo bất thường và phê duyệt nhiều cấp cho lô nhạy cảm.

3.14.5. Dữ liệu cá nhân, pháp lý và quyền riêng tư

Q30: Thông tin nào được ghi trực tiếp lên Bitcoin?

A: Chỉ cam kết mật mã như Merkle root và dữ liệu kỹ thuật cần thiết của giao dịch; họ tên, mã sinh viên và chi tiết văn bằng không nên ghi trực tiếp lên chuỗi. Chứng thư đầy đủ được lưu và chia sẻ ngoài chuỗi.

Q31: Hash có chắc chắn không phải dữ liệu cá nhân không?

A: Không thể khẳng định trong mọi bối cảnh. Nếu hash có thể liên kết với một cá nhân hoặc bị thử lại từ tập dữ liệu nhỏ, nó vẫn có thể chịu yêu cầu bảo vệ dữ liệu. Cần đánh giá tác động quyền riêng tư và tránh neo dữ liệu có entropy thấp một cách trực tiếp.

Q32: Quyền được xóa dữ liệu được xử lý thế nào khi blockchain bất biến?

A: Không ghi dữ liệu cá nhân trực tiếp lên blockchain là biện pháp chính. Dữ liệu off-chain có thể xóa hoặc hạn chế theo chính sách; cam kết mật mã trên chuỗi không thể xóa nên phải được đánh giá pháp lý trước triển khai và giải thích minh bạch cho chủ thể dữ liệu.

Q33: Văn bằng blockchain có giá trị pháp lý tự động không?

A: Không. Giá trị pháp lý phụ thuộc luật, quy chế văn bằng điện tử, chữ ký số, thẩm quyền tổ chức và quy trình chứng thực tại từng quốc gia. Blockchain là bằng chứng kỹ thuật, không tự thay thế con dấu, chữ ký số chuyên dùng hoặc quy định của cơ quan quản lý.

Q34: Có cần sự đồng ý của sinh viên không?

A: Tùy căn cứ xử lý dữ liệu và quy định áp dụng. Dù dựa trên nghĩa vụ pháp lý hay sự đồng ý, tổ chức vẫn cần thông báo mục đích, phạm vi công bố, thời hạn lưu, quyền của người học và kênh xử lý khiếu nại.

Q35: Công khai cổng Verify có làm rò rỉ dữ liệu không?

A: Có nguy cơ nếu API lưu toàn bộ chứng thư, log quá chi tiết hoặc cho phép dò mã sinh viên. Cần tối thiểu hóa log, giới hạn tốc độ, không cung cấp API liệt kê, đặt thời hạn lưu verification log và chỉ hiển thị dữ liệu cần cho mục đích xác minh.

Q36: Có nên đưa bản scan PDF văn bằng lên blockchain không?

A: Không. Dung lượng, chi phí, quyền riêng tư và tính không thể xóa khiến cách đó không phù hợp. Nên lưu tệp ngoài chuỗi, ký/bấm tệp và chỉ neo cam kết mật mã tối thiểu.

3.14.6. Thu hồi, sửa sai và vòng đời văn bằng

Q37: Blockchain bất biến thì sửa sai bằng cách nào?

A: Không sửa giao dịch cũ. Tổ chức thu hồi chứng thư sai, ghi rõ lý do theo mức công khai phù hợp, tạo chứng thư mới và giữ liên kết audit giữa hai phiên bản. Bất biến giúp lịch sử không bị âm thầm viết lại.

Q38: Thu hồi có xóa văn bằng cũ không?

A: Không. Tệp cũ và anchor vẫn tồn tại nhưng trạng thái xác minh phải trả REVOKED. Người đọc cần kiểm tra trạng thái hiện tại thay vì chỉ nhìn thấy giao dịch đã được xác nhận.

Q39: Nếu dịch vụ danh sách thu hồi bị tắt thì sao?

A: Verifier không nên tự kết luận VALID khi không kiểm tra được nguồn thu hồi; kết quả phù hợp là INDETERMINATE hoặc cảnh báo rõ phần kiểm tra chưa hoàn thành. Production cần nhiều bản sao, cache có thời hạn và cơ chế công bố thu hồi bền vững.

Q40: Lý do thu hồi có nên công khai hoàn toàn không?

A: Không phải lúc nào cũng nên, vì lý do có thể chứa dữ liệu nhạy cảm. Có thể công bố mã lý do tối thiểu và giữ diễn giải chi tiết trong audit nội bộ theo phân quyền.

Q41: Số hiệu văn bản trùng được xử lý thế nào?

A: Phiên cập nhật áp dụng ràng buộc duy nhất trên toàn hệ thống và backend kiểm tra trước khi phát hành. Ràng buộc cơ sở dữ liệu vẫn là lớp quyết định cuối để chống race condition giữa các request đồng thời.

Q42: Nếu đã neo blockchain nhưng cập nhật database thất bại thì sao?

A: Đây là lỗi phân tán không thể giải quyết chỉ bằng transaction cơ sở dữ liệu. Worker phải lưu checkpoint, idempotency key và artifact, sau đó đối soát giao dịch để tiếp tục cập nhật thay vì phát hành trùng; cần job reconciliation và cảnh báo vận hành.

Q43: Có thể cấp lại đúng số hiệu đã thu hồi không?

A: Không nên tái sử dụng định danh nếu nó làm lịch sử mơ hồ. Chính sách an toàn là giữ số hiệu cũ gắn với trạng thái thu hồi và cấp phiên bản hoặc số hiệu mới có tham chiếu đến bản được thay thế.

3.14.7. Bảo mật ứng dụng và mô hình đe dọa

Q44: Ấn nút trên giao diện có đủ để phân quyền không?

A: Không. Người dùng có thể gọi API trực tiếp. Backend phải xác thực JWT, role, quyền trên tài nguyên và trạng thái workflow cho mọi endpoint; frontend chỉ cải thiện trải nghiệm.

Q45: JWT bị đánh cắp thì sao?

A: Kẻ tấn công có thể hành động trong phạm vi token đến khi token hết hạn hoặc bị chặn. Production cần TLS, thời hạn access token ngắn, refresh token xoay vòng, thu hồi phiên, bảo vệ XSS/CSRF theo cách lưu token, MFA cho vai trò nhạy cảm và phát hiện đăng nhập bất thường.

Q46: SSRF nguy hiểm ở đâu trong công Verify?

A: Chứng thư có thể chứa URL do người dùng kiểm soát khiến verifier truy cập loopback, metadata cloud hoặc mạng nội bộ. Cần allowlist giao thức, chặn dải IP riêng/đặc biệt sau phân giải DNS, chống DNS rebinding, giới hạn redirect, thời gian, dung lượng và cô lập tiến trình verifier.

Q47: Tải lên JSON có thể dẫn đến thực thi mã không?

A: JSON thuần không tự thực thi, nhưng parser, thư viện và quy trình gọi công cụ có thể có lỗ hổng. Cần giới hạn kích thước/độ sâu, schema validation, không ghép dữ liệu vào shell command, chạy container không đặc quyền và cập nhật dependency.

Q48: Audit log có thể bị quản trị viên sửa không?

A: Trong nguyên mẫu, audit nằm trong PostgreSQL nên tài khoản đặc quyền có thể tác động. Production nên dùng append-only/WORM hoặc chuyển log có chữ

ký sang hệ thống SIEM độc lập, đồng bộ thời gian và cảnh báo khi có khoảng trống.

Q49: Rate limiting có chống được DDoS không?

A: Chỉ giảm lạm dụng ở mức ứng dụng và không thay thế bảo vệ tầng mạng. Cần reverse proxy, CDN/WAF khi phù hợp, quota theo danh tính, giới hạn kích thước, circuit breaker, autoscaling và phương án hấp thụ hoặc lọc lưu lượng.

Q50: Nếu dependency Blockcerts có lỗi hỏng thì sao?

A: Phải khóa phiên bản, tạo SBOM, quét CVE, kiểm tra chữ ký/nguồn gói và có quy trình cập nhật khẩn cấp. Container và worker cần quyền tối thiểu để lỗi của thư viện không trở thành quyền điều khiển toàn hệ thống.

Q51: CORS có phải cơ chế bảo mật API chính không?

A: Không. CORS chủ yếu giới hạn trình duyệt, không ngăn curl hoặc server độc hại gọi API. Xác thực, phân quyền, validation, rate limit và kiểm soát mạng mới là lớp bảo vệ chính.

3.14.8. Độ tin cậy, mở rộng và vận hành production

Q52: Hệ thống hiện tại đã production-ready chưa?

A: Chưa. Hệ thống chứng minh được luồng nghiệp vụ và kỹ thuật trên regtest, nhưng còn cần HSM/KMS, HA, backup/restore được diễn tập, quan sát hệ thống, mainnet staging, kiểm thử tải đồng thời, kiểm thử xâm nhập, quản trị sự cố và rà soát pháp lý.

Q53: Điểm lỗi đơn hiện tại là gì?

A: Một backend/worker, một PostgreSQL, một Redis, một Bitcoin node và lưu trữ cục bộ đều có thể là điểm lỗi đơn. Production cần nhân bản phù hợp, tách worker, lưu trữ bền vững, health check, failover và mục tiêu RPO/RTO rõ ràng.

Q54: Redis mất dữ liệu thì chứng thư có mất không?

A: PostgreSQL và artifact mới là nguồn trạng thái lâu dài; Redis chỉ điều phối. Tuy nhiên job đang chờ có thể mất hoặc lặp, nên cần cấu hình persistence phù hợp, job id ổn định và tiến trình reconciliation quét bản ghi queued/processing để phục hồi.

Q55: Làm sao tránh phát hành trùng khi worker chạy lại?

A: Mỗi yêu cầu cần idempotency key, khóa/trạng thái chuyển đổi nguyên tử và kiểm tra artifact/giao dịch đã tồn tại trước khi broadcast. Cơ chế retry phải phân biệt lỗi trước và sau broadcast vì retry mà sau broadcast có thể tạo giao dịch thứ hai.

Q56: Một lô quá lớn có vấn đề gì?

A: Tạo proof và artifact tốn CPU, RAM, I/O và làm tăng phạm vi ảnh hưởng nếu lô lỗi. Cần giới hạn kích thước lô, chia chunk, đo thời gian, lưu checkpoint và bảo đảm một chứng thư lỗi không làm trạng thái các chứng thư khác mơ hồ.

Q57: Có thể phục vụ hàng triệu văn bằng không?

A: Về nguyên lý batching giảm số giao dịch, nhưng nguyên mẫu chưa chứng minh quy mô đó. Cần benchmark theo tải thật, partition/index database, object storage, nhiều worker, queue partition, cache xác minh, lifecycle dữ liệu và kiểm thử chi phí trước khi tuyên bố khả năng mở rộng.

Q58: Cần giám sát những chỉ số nào?

A: Nên theo dõi độ trễ và tỷ lệ lỗi API, độ sâu queue, tuổi job, số bản ghi mắc ở queued/processing, thời gian phát hành, confirmation, số dư ví phí, lỗi RPC, tỷ lệ VALID/INVALID/INDETERMINATE, dung lượng đĩa, backup và các sự kiện quyền cao.

Q59: Kế hoạch sao lưu tối thiểu là gì?

A: Sao lưu mã cấu hình không bí mật, PostgreSQL, artifact chứng thư, Issuer Profile, audit và metadata khóa; khóa phải sao lưu riêng bằng cơ chế mã hóa/quorum. Quan trọng hơn là định kỳ phục hồi thử trên môi trường cô lập và đo RPO/RTO.

Q60: Nếu mất toàn bộ máy chủ nhưng blockchain còn thì khôi phục được hết không?

A: Không. Blockchain chỉ chứa cam kết mật mã, không chứa hồ sơ đầy đủ, tài khoản, audit hay tệp chứng thư. Muốn khôi phục dịch vụ phải có backup off-chain và vật liệu khóa; blockchain không thay thế chiến lược sao lưu.

3.14.9. Kiểm thử, bằng chứng và giới hạn thực nghiệm

Q61: Tại sao kết quả regtest không chứng minh mainnet sẽ giống hệt?

A: Regtest cho phép tạo block theo yêu cầu, gần như không có cạnh tranh phí, reorg thực tế hay độ trễ mạng công khai. Nó chứng minh tích hợp API và định dạng giao dịch, nhưng mainnet cần staging/testnet, theo dõi mempool và thử nghiệm vận hành riêng.

Q62: Các bài test tự động chứng minh được điều gì?

A: Chúng chứng minh các trường hợp đã mã hóa đang cho kết quả mong đợi tại phiên bản hiện tại. Chúng không chứng minh không có lỗi ngoài phạm vi, không thay thế review kiến trúc, pentest, kiểm thử tải và đánh giá người dùng.

Q63: Ảnh chụp màn hình có phải bằng chứng đủ mạnh không?

A: Không. Ảnh hỗ trợ tái hiện giao diện nhưng có thể thiếu ngữ cảnh và không chứng minh trạng thái backend. Bằng chứng tốt hơn gồm log đã khử bí mật, JSON kết quả, txid, hash artifact, phiên bản commit, script tái lập và kiểm tra chéo dữ liệu.

Q64: Benchmark trong báo cáo có thể suy rộng không?

A: Chỉ trong cấu hình phần cứng, dữ liệu, tải và điều kiện được mô tả. Không nên dùng số đo đơn máy/regtest để cam kết SLA; cần nhiều lần chạy, percentile, tải đồng thời và phân tích nút thắt ở môi trường gần production.

Q65: Làm sao bảo đảm thí nghiệm tái lập được?

A: Cố định commit và phiên bản dependency/image, công bố cấu hình không chứa bí mật, migration, seed tối thiểu, script chạy, tiêu chí pass/fail và hash artifact. Các dữ liệu nhạy cảm phải được thay bằng fixture giả chứ không đưa credential thật vào repository.

Q66: Tại sao ba kịch bản vẫn chứa tám bước thao tác?

A: Ba kịch bản là ba nhóm mục tiêu đánh giá tổng hợp, còn tám bước là quy trình chi tiết để tái hiện toàn bộ coverage trước đó. Việc nhóm lại nhằm giảm phân mảnh trình bày, không xóa trường hợp kiểm thử.

Q67: Có trường hợp kiểm thử nào còn thiếu?

A: Có. Các khoảng trống đáng chú ý gồm tải đồng thời lớn, reorg dài, mempool fee spike, node lệch chuỗi, crash đúng thời điểm broadcast, phục hồi backup, xoay khóa, tấn công chuỗi cung ứng, kiểm thử trình duyệt/thiết bị và accessibility.

3.14.10. Trải nghiệm người dùng và khả năng tương tác

Q68: Người tuyển dụng có phải cài ví Bitcoin không?

A: Không. Cổng Verify hoặc thư viện xác minh đọc chứng thư và blockchain thay người dùng. Người xác minh chỉ cần tệp/QR hoặc liên kết phù hợp; hệ thống không yêu cầu họ sở hữu bitcoin.

Q69: Tại sao phát hành JSON thay vì chỉ PDF?

A: JSON chứa dữ liệu có cấu trúc, chữ ký/proof và tham chiếu cần cho xác minh máy. PDF dễ đọc nhưng không đủ để mang toàn bộ bằng chứng một cách chuẩn hóa; có thể cung cấp PDF trình bày kèm QR/hash liên kết tới chứng thư JSON.

Q70: QR code có làm giả được không?

A: Có thể thay QR để trở tới trang giả. Người dùng vẫn phải kiểm tra tên miền, HTTPS, danh tính issuer và kết quả verifier; QR chỉ là phương tiện nhập dữ liệu, không phải bằng chứng tin cậy.

Q71: Nếu đổi tên miền Issuer Profile thì chứng thư cũ ra sao?

A: URL cũ trong chứng thư vẫn được tham chiếu. Tổ chức nên duy trì redirect/hosting dài hạn hoặc cơ chế định danh bền vững; thay miền không kế hoạch có thể khiến xác minh thành INDETERMINATE dù anchor còn nguyên.

Q72: Hệ thống có tương tác với ví danh tính số hoặc chuẩn khác không?

A: Blockcerts dựa trên Verifiable Credentials nhưng khả năng tương tác thực tế phụ thuộc profile, proof và verifier. Muốn tích hợp ví khác cần kiểm thử theo từng implementation, ánh xạ schema và tránh tuyên bố tương thích chỉ dựa vào tên chuẩn.

Q73: Người khuyết tật có sử dụng được ba công không?

A: Nguyên mẫu chưa phải bằng chứng tuân thủ accessibility. Production cần kiểm tra bàn phím, screen reader, độ tương phản, nhãn biểu mẫu, thông báo lỗi, responsive và tiêu chuẩn WCAG phù hợp.

Q74: Có thể xác minh hoàn toàn bằng điện thoại không?

A: Có thể nếu giao diện, tải tệp và QR được tối ưu, nhưng cần kiểm thử trên Android/iOS và nhiều trình duyệt. Các tệp lớn, quyền truy cập file và chuyển ứng dụng là các điểm dễ gây lỗi trải nghiệm.

3.14.11. Quản trị, đạo đức và hướng phát triển

Q75: Ai chịu trách nhiệm khi văn bằng sai?

A: Trách nhiệm vẫn thuộc quy trình và tổ chức có thẩm quyền, không chuyển cho blockchain. Audit giúp xác định ai tạo, ai duyệt và hệ thống xử lý thế nào, nhưng chính sách nhân sự/pháp lý phải quy định trách nhiệm và kênh sửa sai.

Q76: Có nguy cơ dùng công nghệ để giám sát người học không?

A: Có nếu cùng một định danh công khai được tái sử dụng hoặc lịch sử xác minh bị lưu quá mức. Cần privacy by design, định danh theo ngữ cảnh, tối thiểu hóa log, thời hạn lưu và quyền truy cập minh bạch.

Q77: Làm sao tránh phụ thuộc một nhà cung cấp?

A: Dùng chuẩn mở, giữ dữ liệu và artifact ở định dạng có thể xuất, tự vận hành node hoặc hỗ trợ nhiều RPC, container hóa có kiểm soát và diễn tập chuyển nhà cung cấp. Tuy nhiên vẫn phải quản lý rủi ro phụ thuộc thư viện Blockcerts và phiên bản chuẩn.

Q78: Nếu Blockcerts ngừng được duy trì thì sao?

A: Tập và anchor vẫn có thể được kiểm tra nếu đặc tả, mã nguồn và dependency được lưu trữ. Tổ chức cần archive verifier, SBOM, test vector và có kế hoạch chuyển đổi sang chuẩn/chữ ký khác mà vẫn duy trì xác minh chứng thư cũ.

Q79: Đóng góp mới của đề án nằm ở đâu?

A: Giá trị không phải phát minh blockchain mới mà là tích hợp có kiểm soát: ba cổng tách biệt, Maker–Checker, RBAC, pipeline bất đồng bộ, Merkle batching, xác minh/thu hồi, dữ liệu nghiệp vụ văn bằng và bộ thực nghiệm tái lập trong một hệ thống hoàn chỉnh.

Q80: Hạn chế lớn nhất của đề án là gì?

A: Hạn chế lớn nhất là khoảng cách giữa nguyên mẫu regtest một máy và production có giá trị pháp lý: quản lý khóa, HA, tải lớn, mainnet, quyền riêng tư, vận hành sự cố và đánh giá pháp lý chưa được chứng minh đầy đủ.

Q81: Ưu tiên phát triển tiếp theo nên là gì?

A: Ưu tiên theo rủi ro: HSM/KMS và xoay khóa; idempotency/reconciliation; backup-restore và HA; quan sát hệ thống; testnet/mainnet staging cùng quản lý phí; pentest; đánh giá pháp lý/quyền riêng tư; sau đó mới tối ưu trải nghiệm và mở rộng tích hợp.

Q82: Tiêu chí nào để quyết định có nên đưa hệ thống vào vận hành?

A: Chỉ nên triển khai khi có chủ sở hữu nghiệp vụ, đánh giá pháp lý, threat model được phê duyệt, kiểm thử xâm nhập, diễn tập mất khóa/khôi phục, SLO và trực vận hành, kiểm soát thay đổi, ngân sách phí, kế hoạch ngừng dịch vụ và chấp nhận rủi ro bằng văn bản.

3.15. Kết luận Chương 3

Chương 3 đã trình bày môi trường, backend, ba frontend, Worker, Blockcerts toolchain, xác minh, thu hồi, định danh Student, outbox/idempotency/reconciliation và safe-fetch chống SSRF; đồng thời bổ sung quy trình triển khai cùng kịch bản thao tác có thể tái lập. Kịch bản gốc đạt 6/6, 9/9 lần chạy với 1.830/1.830 proof và 8/8; vòng xác minh mới đạt 8/8 suite–21/21 test backend, 10/10 test SSRF, build và E2E thành công. Kết quả chỉ chứng minh phạm vi đã kiểm thử trên regtest, không được suy rộng thành mức sẵn sàng vận hành thực tế.

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết quả của đồ án

Đồ án đã nghiên cứu nền tảng blockchain và mô hình Thông tin xác thực có thể kiểm chứng; từ đó phân tích, thiết kế và xây dựng nguyên mẫu quản lý văn bằng số dựa trên Blockcerts V3 và Bitcoin. Hệ thống có ba vai trò Maker–Checker–Student, một tác nhân xác minh công khai, ba cổng web, phát hành theo lô, thu hồi và nhật ký kiểm toán.

Nguyên mẫu bấm từng chứng thư, tạo cây Merkle, neo một gốc chung của lô lên Bitcoin regtest và gắn bằng chứng riêng cho từng chứng thư. Bộ xác minh đối chiếu dữ liệu, bằng chứng Merkle, giao dịch neo, số xác nhận và trạng thái thu hồi. Thực nghiệm hoàn thành 6/6 phép thử nghiệp vụ, 8/8 phép thử xác minh–thu hồi và chín lần đo hiệu năng với 1.830/1.830 chứng thư được kiểm toán. Kết quả chỉ có giá trị trong phạm vi nguyên mẫu; blockchain không tự chứng minh dữ liệu đầu vào và không thay thế trách nhiệm quản trị của cơ sở đào tạo.

2. Đóng góp chính

Đóng góp chính gồm: mô hình phát hành có phân tách Maker–Checker; cơ chế phát hành Blockcerts V3 theo lô với bằng chứng Merkle và giao dịch neo Bitcoin; ba cổng giao diện theo nhóm người dùng; quy trình xác minh kết hợp dữ liệu trên chuỗi, trạng thái thu hồi và nhật ký kiểm toán; cùng bộ migration, kịch bản và tệp bằng chứng phục vụ tái lập.

3. Kiến nghị

Trước khi triển khai thực tế, cần đánh giá pháp lý và quy trình nghiệp vụ; quản lý khóa bằng HSM/KMS; bổ sung chữ ký tổ chức; xây dựng bộ lập chỉ mục để phục hồi dữ liệu thu hồi; tăng độ phủ kiểm thử và thực hiện kiểm thử bảo mật độc lập; hoàn thiện quy trình sao lưu, giám sát, luân chuyển khóa và ứng cứu sự cố. Chỉ nên chuyển sang testnet hoặc mainnet sau khi đáp ứng các điều kiện trên và phê duyệt chính sách về chi phí, quyền riêng tư, lưu trữ dữ liệu.

- [1] D. Yaga, P. Mell, N. Roby, K. Scarfone, “Blockchain Technology Overview,” NISTIR 8202, National Institute of Standards and Technology, 2018. DOI: 10.6028/NIST.IR.8202. <https://doi.org/10.6028/NIST.IR.8202>.
- [2] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. <https://bitcoin.org/bitcoin.pdf> (truy cập ngày 13/09/2026).
- [3] R. C. Merkle, “Secrecy, Authentication, and Public Key Systems,” doctoral dissertation, Stanford University, 1979.
- [4] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4. <https://doi.org/10.6028/NIST.FIPS.180-4>.
- [5] National Institute of Standards and Technology, “Digital Signature Standard (DSS),” FIPS PUB 186-5, 2023. DOI: 10.6028/NIST.FIPS.186-5. <https://doi.org/10.6028/NIST.FIPS.186-5>.
- [6] M. Castro, B. Liskov, “Practical Byzantine Fault Tolerance,” Proceedings of the Third Symposium on Operating Systems Design and Implementation, 1999, pp. 173–186. <https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance>.
- [7] D. Ongaro, J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” 2014 USENIX Annual Technical Conference, 2014, pp. 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>.
- [8] E. Androulaki và cộng sự, “Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains,” Proceedings of the Thirteenth EuroSys Conference, 2018. DOI: 10.1145/3190508.3190538. <https://doi.org/10.1145/3190508.3190538>.
- [9] M. Sporny, D. Longley, D. Chadwick (biên tập), “Verifiable Credentials Data Model v2.0,” W3C Recommendation, 15/05/2025. <https://www.w3.org/TR/vc-data-model-2.0/>.
- [10] T. T. A. Dinh, R. Liu, M. Zhang, G. Chen, B. C. Ooi, J. Wang, “Untangling Blockchain: A Data Processing View of Blockchain Systems,” IEEE Transactions on Knowledge and Data Engineering, vol. 30, no. 7, pp. 1366–1385, 2018. DOI: 10.1109/TKDE.2017.2781227.
- [11] Blockcerts, “cert-schema: JSON schemas and context for Blockcerts,” commit 1e390ac, 31/07/2025. <https://github.com/blockchain-certificates/cert-schema/tree/1e390ace91fb2b09518e45760168f344ef3a4409> (truy cập ngày 13/09/2026).
- [12] D. Longley, G. Kellogg, M. Sporny, M. Lanthaler, P.-A. Champin, N. Lindström (biên tập), “JSON-LD 1.1,” W3C Recommendation, 16/07/2020. <https://www.w3.org/TR/json-ld11/>.
- [13] M. Sporny, D. Longley (biên tập), “Verifiable Credential Data Integrity 1.0,” W3C Recommendation, 15/05/2025. <https://www.w3.org/TR/vc-data-integrity/>.
- [14] Blockcerts, “jsonld-signatures-merkleproof2019: MerkleProof2019 suite,” commit 26a9c30, 25/08/2026. <https://github.com/blockchain-certificates/jsonld-signatures-merkleproof2019/tree/26a9c3071355a9ccd83b0b90fc375a4d8cf1d4b8> (truy cập ngày 13/09/2026).
- [15] M. Jones, J. Bradley, N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, 2015. <https://www.rfc-editor.org/rfc/rfc7519>.

- [16] OWASP Foundation, “Authorization Cheat Sheet.”
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (truy cập ngày 13/09/2026).
- [17] OWASP Foundation, “Password Storage Cheat Sheet.”
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (truy cập ngày 13/09/2026).
- [18] Bitcoin Project, “Bitcoin Developer Guide — Testing Applications.”
<https://developer.bitcoin.org/examples/testing.html> (truy cập ngày 13/09/2026).
- [19] NestJS, “NestJS Documentation.” <https://docs.nestjs.com/> (truy cập ngày 13/09/2026).
- [20] PostgreSQL Global Development Group, “PostgreSQL 17 Documentation — Transactions.” <https://www.postgresql.org/docs/17/tutorial-transactions.html> (truy cập ngày 13/09/2026).
- [21] BullMQ, “BullMQ Documentation.” <https://docs.bullmq.io/> (truy cập ngày 13/09/2026).
- [22] TypeORM, “TypeORM Documentation — Migrations and Transactions.”
<https://typeorm.io/docs/advanced-topics/migrations/>; <https://typeorm.io/docs/advanced-topics/transactions/> (truy cập ngày 13/09/2026).
- [23] OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet.”
https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html (truy cập ngày 13/09/2026).

PHỤ LỤC 89

Phụ lục A. Nhóm API chính 89

Nhóm	Đường dẫn tiêu biểu	Quyền
Xác thực	POST /api/auth/login; POST /api/auth/register	Đăng nhập công khai; đăng ký chỉ Checker
Tài khoản	/api/admin/users	Checker
Phát hành	/api/issue/*	Maker/Checker theo hành động
Holder	/api/issue/holder/certificates	Student
Xác minh	/api/verify	Công khai
Thu hồi	/api/revoke/:id	Checker
Issuer	/api/blockcerts/issuers/kma/*	Công khai
Audit	/api/audit	Maker, Checker
Health	/health	Công khai

Phụ lục B. Bảng chứng thực nghiệm chính thức 89

Bộ kết quả chính thức nằm trong `experiments/results/`, gồm tóm tắt kịch bản 1 và 3, chín kết quả benchmark, bảng tổng hợp hiệu năng, kiểm toán artifact, biểu đồ và SHA256SUMS. Freeze dùng cho báo cáo là `experiments/freeze/b12-freeze-r2.json`. Dữ liệu credential, private key, WIF, token, mật khẩu và tài khoản bootstrap không thuộc phạm vi công khai.

Phụ lục C. Hướng dẫn tái lập tóm tắt 89

1. Tạo biến môi trường từ `.env.example` và thay toàn bộ giá trị bí mật.
2. Khởi động PostgreSQL, Redis và Bitcoin Core regtest bằng Docker Compose.
3. Chạy migration của backend; tạo ví regtest và cấp coin thử nghiệm.
4. Build, khởi động backend/worker và ba frontend; chạy health check.
5. Tạo tài khoản thử nghiệm không chứa dữ liệu cá nhân thật.
6. Chạy preflight, đóng băng cấu hình và thực thi các kịch bản.
7. Kiểm toán Merkle proof, tạo checksum và chỉ công bố artifact đã khử dữ liệu nhạy cảm.